计算机组成原理

实验教程

西北工业大学出版社

前　言

计算机组成原理实验的目标是是学生真正了解计算机硬件的基本组成原理，掌握计算机系统的设计技术，培养学生分析和解决数字系统问题的能力，为后续相关专业课程的学习打下良好的基础。

本书是《计算机组成原理》的配套实验教材，用于指导学生完成计算机组成原理的一系列实验。实验装置采用西安唐都科教仪器公司和西北工业大学联合研发的cm3+计算机组成原理实验系统。

本书分两部分：第一部分为验证性实验，包括部件实验和模型机实验，介绍了几个实验，用来干什么。第二部分为设计性实验，介绍了几个实验，用来干什么。每个实验包括实验目的、实验设备选择、实验内容（实验原理和实验步骤）和实验参考结果，并附有思考题。

本书实验内容紧扣教学大纲，理论联系实际，有利于培养学生的动手能力，帮助其深入掌握计算机硬件电路。

由于编者水平有限，错误在所难免，殷切希望广大读者批评指正。

编　者

目 录

第 1 部分 验证性实验.....1
1.1 基本运算器实验.....1
1.2 静态随机存储器实验.....7
1.3 微程序控制器实验.....11
1.4 基本模型机设计实验.....21
1.5 复杂模型机设计实验.....30
1.6 带中断处理能力的模型机设计实验.....44
1.7 典型I/O接口8253 扩展设计实验.....59
第 2 部分 设计性实验.....63
2.1 超前进位加法器设计实验.....63
2.2 阵列乘法器设计实验.....70
2.3 Cache控制器设计实验.....72
2.4 硬布线控制器设计实验.....78
附录1 软件使用说明.....81
附录2 时序单元介绍.....90
附录3 系统布局图.....92
附录4 实验用芯片介绍.....93

第1部分 验证性实验

1.1 基本运算器实验

1.1.1 实验目的

(1) 了解运算器的组成结构。

(2) 掌握运算器的工作原理。

1.1.2 实验设备

PC 机一台，TD-CM3+实验系统一套。

1.1.3 实验原理

本实验的原理如图1-1-1 所示。

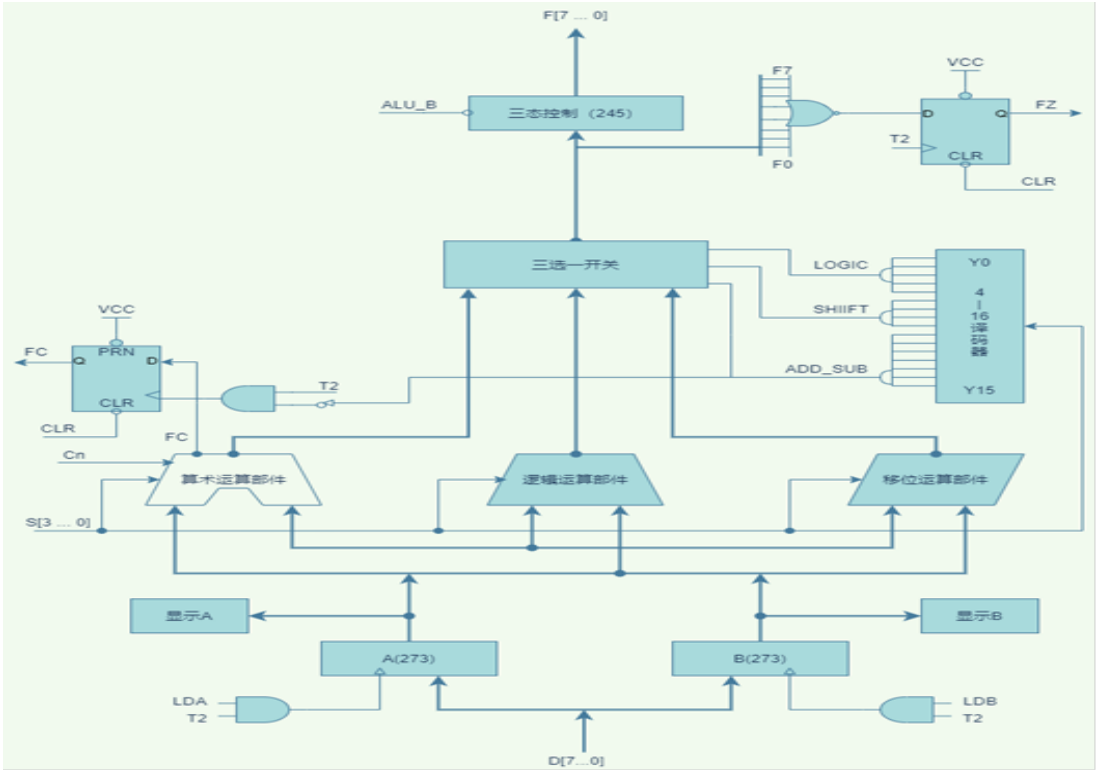


图1-1-1 运算器原理图

运算器内部含有三个独立运算部件，分别为算术、逻辑和移位运算部件，要处理的数据存于暂存器A 和暂存器B，三个部件同时接受来自A 和B 的数据（有些处理器体系结构把移位运算器放于算术和逻辑运算部件之前，如ARM），各部件对操作数进行何种运算由控制信号S3…S0来决定，任何时候，多路选择开关只选择三部件中一个部件的结果作为ALU 的输出。如果是算术运算，还将置进位标志FC，在运算结果输出前，置ALU 零标志。ALU 中所有模块集成在一片CPLD(MAXII EPM240)中。

逻辑运算部件由逻辑门构成，较为简单，而后面又有专门的算术运算部件设计实验，在此对这两个部件不再赘述。移位运算采用的是桶形移位器，一般采用交叉开关矩阵来实现，

交叉开关的原理如图1-1-2 所示。图中显示的是一个4X4 的矩阵（系统中是一个8X8 的矩阵）。每一个输入都通过开关与一个输出相连，把沿对角线的开关导通，就可实现移位功能，即：

(1) 对于逻辑左移或逻辑右移功能，将一条对角线的开关导通，这将所有的输入位与所使用的输出分别相连,而没有同任何输入相连的则输出连接0。

(2) 对于循环右移功能，右移对角线同互补的左移对角线一起激活。例如，在4 位矩阵中使用‘右1’和‘左3’对角线来实现右循环1 位。

(3) 对于未连接的输出位，算术右移使用符号扩展而不是0 填充。使用另外的逻辑进行移位总量译码和符号判别。

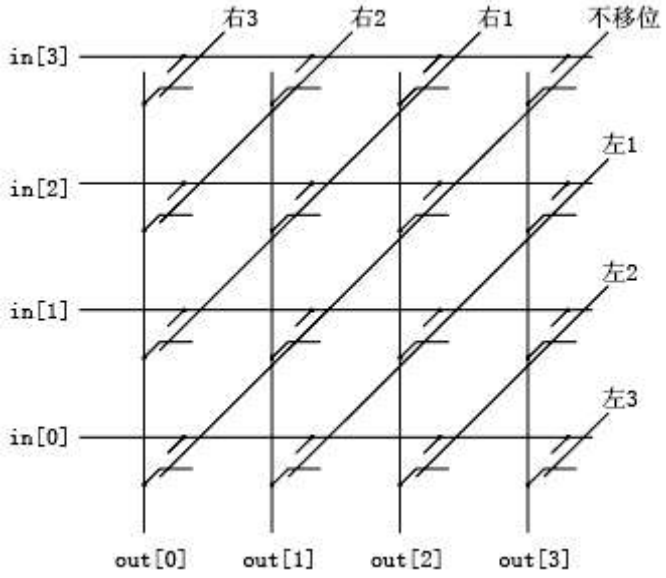


图1-1-2 交叉开关桶形移位器原理图

运算器单元由以下部分构成：一片CPLD 实现的ALU，四片74LS245 构成的保护电路。ALU的输出通过三态门74LS245 连到CPU 内总线上，CPU 内总线和数据总线已连通，数据总线指示灯和数据总线相连,用来显示数据总线的内容(实验系统中所有的LED 显示灯均为正逻辑，亮为‘1’，灭为‘0’)，ALU 的数据输入以排针形式引出B7’…B0’，另外还有进位标志FC 和零标志FZ 指示灯。图1-1-1 中有三部分不在CPLD 中实现，而是在外围电路中实现，这三部分为图中的‘显示A’、‘显示B’和ALU 的输出控制‘三态控制245’，请注意：实验箱上凡丝印标注有马蹄形标记‘⌴’，表示这两根排针之间是连通的。图中除T2 和CLR，其余信号均来自于ALU单元的排线座，实验箱中所有单元的T1、T2 都连接至MC 单元的T1、T2，CLR 都连接至CON 单元的CLR 按钮。T2 由时序单元的TS2 提供（时序单元的介绍见附录二），其余控制信号均由CON 单元的二进制数据开关模拟给出。控制信号中除T2 为脉冲信号外，其余均为电平信号，其中ALU_B 为低有效，其余为高有效。

暂存器A 和暂存器B 的数据能在LED 灯上实时显示，原理如图1-1-3 所示（以A0 为例，其它相同）。进位标志FC、零标志FZ 和数据总线D7…D0 的显示原理也是如此。

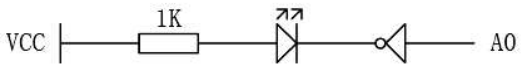


图1-1-3 A0 显示原理图

ALU 和外围电路的连接如图1-1-4 所示，由于ALU 的工作电压为3.3V，所以在所有用户操作的I/O 脚都加上74LS245 加以隔离保护，以防误操作烧坏ALU 芯片，图中的小方框代表排针座。

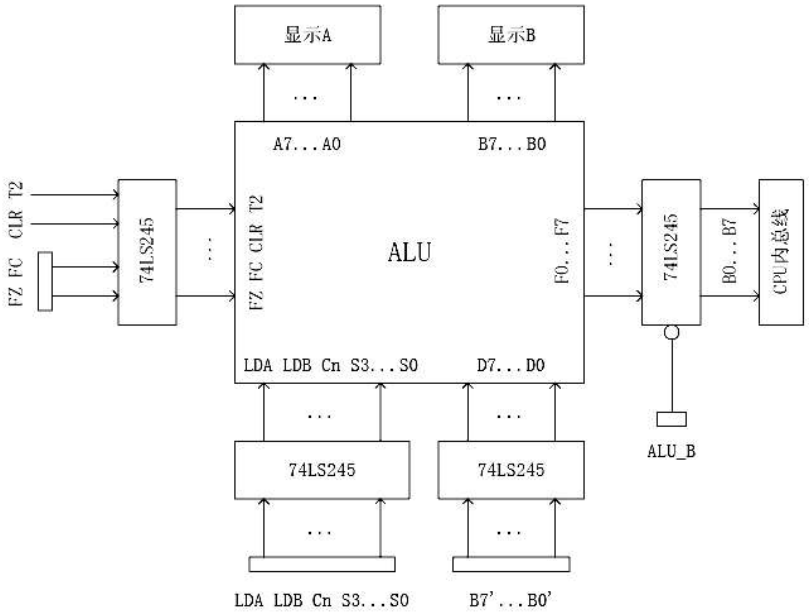


图1-1-4 ALU 和外围电路连接原理图

运算器的逻辑功能表如表1-1-1 所示，其中S3 S2 S1 S0 为4 位控制信号，Cn 为来自低位的进位信号，FC 为向高位的进位标志，FZ 为运算器零标志，表中功能栏内的FC、FZ 表示当前运算会影响到该标志。

表1-1-1 运算器逻辑功能表

运算类型	S3S2S1S0	功能
逻辑运算	0 0 0 0	F=A （直通）
	0 0 0 1	F=B （直通）
	0 0 1 0	F=AB （FZ）
	0 0 1 1	F=A+B （FZ）
	0 1 0 0	F=/A （FZ）
移位运算	0 1 0 1	F=A逻辑右移B（取低3位）位 （FZ）
	0 1 1 0	F=A逻辑左移B（取低3位）位 （FZ）
	0 1 1 1	F=A算术右移B（取低3位）位 （FZ）
	1 0 0 0	F=A循环右移B（取低3位）位 （FZ）
算术运算	1 0 0 1	F=A加B （FC，FZ）
	1 0 1 0	F=A加B加Cn （FC，FZ）
	1 0 1 1	F=A减B （FC，FZ）
	1 1 0 0	F=A减1 （FC，FZ）
	1 10 1	F=A加1 （FC，FZ）
	1 1 1 0	（保留）
	1 1 1 1	（保留）

1.1.4 实验步骤

(1) 按图1-1-5 连接实验电路，并检查无误。图中将用户需要连接的信号用圆圈标明（其它实验相同）。

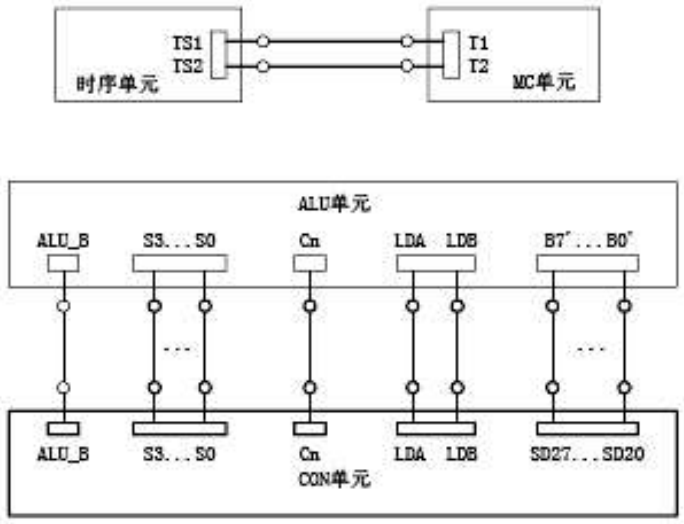


图1-1-5 实验接线图

(2) 将时序单元的状态开关置为‘单步’档（时序单元的介绍见附录二）,MEM 单元的编程开关置为‘运行’档。

(3) 打开电源开关，如果听到有‘嘀’报警声，说明有总线竞争现象，应立即关闭电源，重新检查接线，直到错误排除。然后按动CON 单元的CLR 按钮，将运算器的A、B 和FC、FZ 清零。

(4) 用输入开关向暂存器A 置数。

① 拨动CON 单元的SD27…SD20 数据开关，形成二进制数01100101（或其它数值），数据显示亮为‘1’，灭为‘0’。

② 置LDA=1,LDB=0,按动时序单元的TS 按钮,产生一个T2 上沿,则将二进制数01100101 置入暂存器A 中，暂存器A 的值通过ALU 单元的A7…A0 八位LED 灯显示。

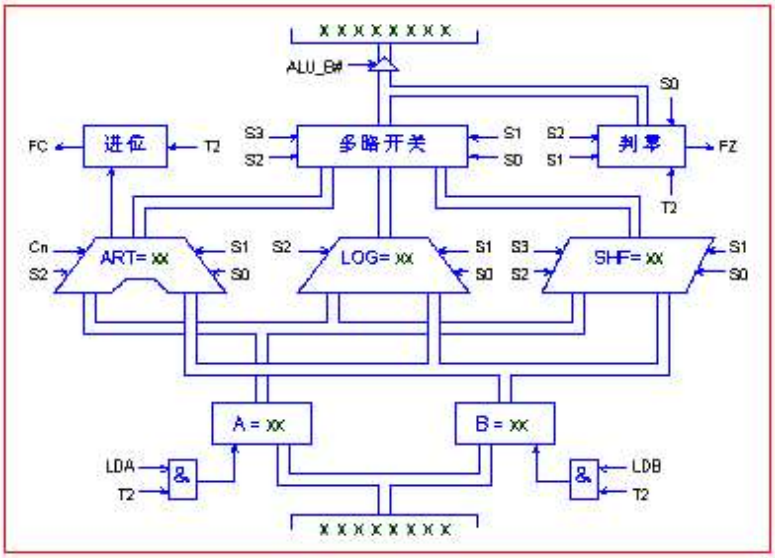


图1-1-6 数据通路图

(5) 用输入开关向暂存器B 置数。

① 拨动CON 单元的SD27…SD20 数据开关，形成二进制数10100111（或其它数值）。

② 置LDA=0,LDB=1,按动时序单元的TS 按钮,产生一个T2 上沿,则将二进制数10100111置入暂存器B 中，暂存器B 的值通过ALU 单元的B7…B0 八位LED 灯显示。

表1-1-2 运算结果表

运算类型	A	B	S3S2S1S0	结果
逻辑运算	65	A7	0 0 0 0	F=（ 65 ） FC=（ ） FZ（ ）
	65	A7	0 0 0 1	F=（ A7 ） FC=（ ） FZ（ ）
			0 0 1 0	F=（ ） FC=（ ） FZ（ ）
			0 0 1 1	F=（ ） FC=（ ） FZ（ ）
			0 1 0 0	F=（ ） FC=（ ） FZ（ ）
移位运算			0 1 0 1	F=（ ） FC=（ ） FZ（ ）
			0 1 1 0	F=（ ） FC=（ ） FZ（ ）
			0 1 1 1	F=（ ） FC=（ ） FZ（ ）
			1 0 0 0	F=（ ） FC=（ ） FZ（ ）
算术运算			1 0 0 1	F=（ ） FC=（ ） FZ（ ）
			1 0 1 0（Cn=0）	F=（ ） FC=（ ） FZ（ ）
			1 0 1 0（Cn=1）	F=（ ） FC=（ ） FZ（ ）
			1 0 1 1	F=（ ） FC=（ ） FZ（ ）
			1 1 0 0	F=（ ） FC=（ ） FZ（ ）
			1 1 0 1	F=（ ） FC=（ ） FZ（ ）

(6) 改变运算器的功能设置，观察运算器的输出。置ALU_B=0、LDA=0、LDB=0，然后按表1-1-1 置S3、S2、S1、S0 和Cn 的数值，并观察数据总线LED 显示灯显示的结果。如置S3、S2、S1、S0 为0010，运算器作逻辑与运算，置S3、S2、S1、S0 为1001，运算器作加法运算。

如果实验箱和PC 联机操作，则可通过软件中的数据通路图来观测实验结果（软件使用说明请看附录一），方法是：打开软件，选择联机软件的“【实验】—【运算器实验】”，打开运算器实验的数据通路图，如图1-1-6 所示。进行上面的手动操作，每按动一次TS 按钮，数据通路图会有数据的流动，反映当前运算器所做的操作，或在软件中选择“【调试】—【单周期】”，其作用相当于将时序单元的状态开关置为‘单步’档后按动了一次TS 按钮，数据通路图也会反映当前运算器所做的操作。

重复上述操作，并完成表1-1-2。然后改变A、B 的值，验证FC、FZ 的锁存功能。

思考题：

1. 利用上述运算器能否实现大于8位二进制数的算术运算？如果能，需要采取什么样的措施？
2. 给出一组数据，验证桶形移位器的功能。

实验报告要求：

1. 实验记录：所有的运算结果，故障现象及排出经过；
2. 试解答思考题的问题；
3. 通过本次实验的收获及想法。

1.2 静态随机存储器实验

1.2.1 实验目的

掌握静态随机存储器RAM 工作特性及数据的读写方法。

1.2.2 实验设备

PC 机一台，TD-CM3+实验系统一套。

1.2.3 实验原理

实验所用的静态存储器由一片6116（2K×8bit）构成（位于MEM 单元），如图1-2-1 所示。6116 有三个控制线：CS（片选线）、OE（读线）、WE（写线），其功能如表1-2-1 所示，当片选有效（CS=0）时，OE=0 时进行读操作，WE=0 时进行写操作，本实验将CS 常接地。

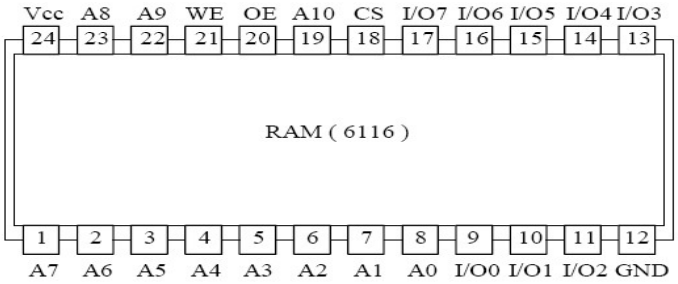


图1-2-1 SRAM 6116 引脚图

由于存储器（MEM）最终是要挂接到CPU 上，所以其还需要一个读写控制逻辑，使得CPU 能控制MEM 的读写，实验中的读写控制逻辑如图1-2-2 所示，由于T2 的参与，可以保证MEM 的写脉宽与T2 一致，T2 由时序单元的TS2 给出（时序单元的介绍见附录2）。IOM 用来选择是对I/O 还是对MEM 进行读写操作，RD=1 时为读，WR=1 时为写。

表1-2-1 SRAM 6116 功能表

$\overline{CS}$	$\overline{WE}$	$\overline{OE}$	功能
1	X	X	不选择
0	1	0	读
0	0	1	写
0	0	0	写

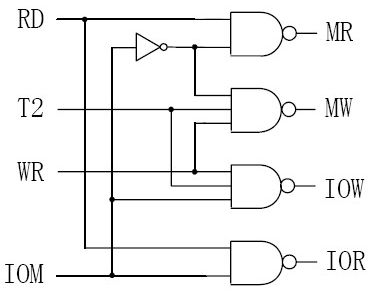


图1-2-2 读写控制逻辑

实验原理图如图1-2-3 所示，存储器数据线接至数据总线，数据总线上接有8个LED 灯

显示D7…D0 的内容。地址线接至地址总线，地址总线上接有8 个LED 灯显示A7…A0 的内容，地址由地址锁存器（74LS273，位于PC&AR 单元）给出。数据开关（位于IN 单元）经一个三态门（74LS245）连至数据总线，分时给出地址和数据。地址寄存器为8 位，接入6116 的地址A7…A0，6116 的高三位地址A10…A8 接地，所以其实际容量为256 字节。

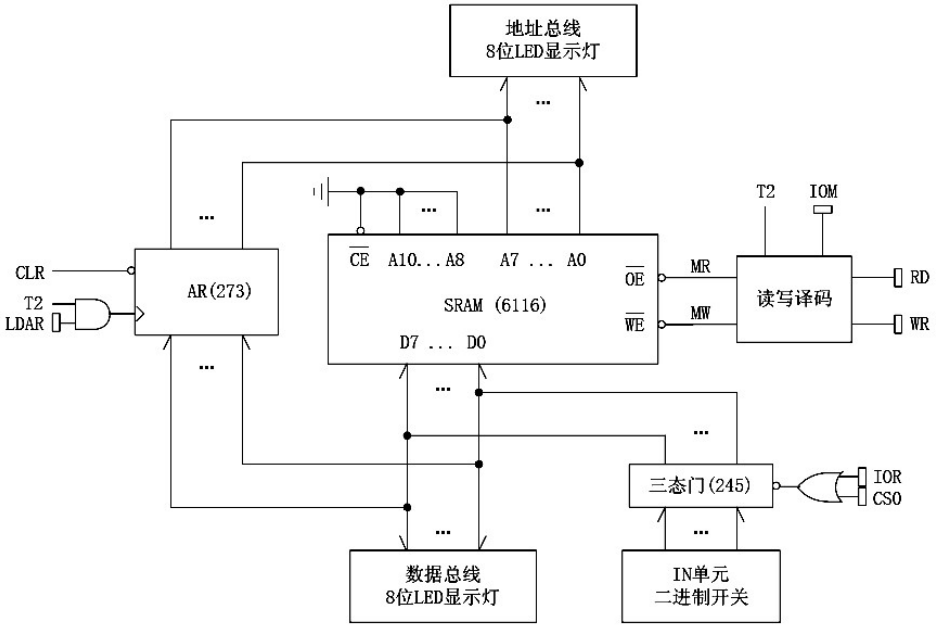


图1-2-3 存储器实验原理图

实验箱中所有单元的T1、T2 都连接至MC 单元的T1、T2，CLR 都连接至CON 单元的CLR 按钮。实验时T2 由时序单元给出，其余信号由CON 单元的二进制开关模拟给出，其中IOM 应为低（即MEM 操作），RD、WR 高有效，MR 和MW 低有效，LDAR 高有效。

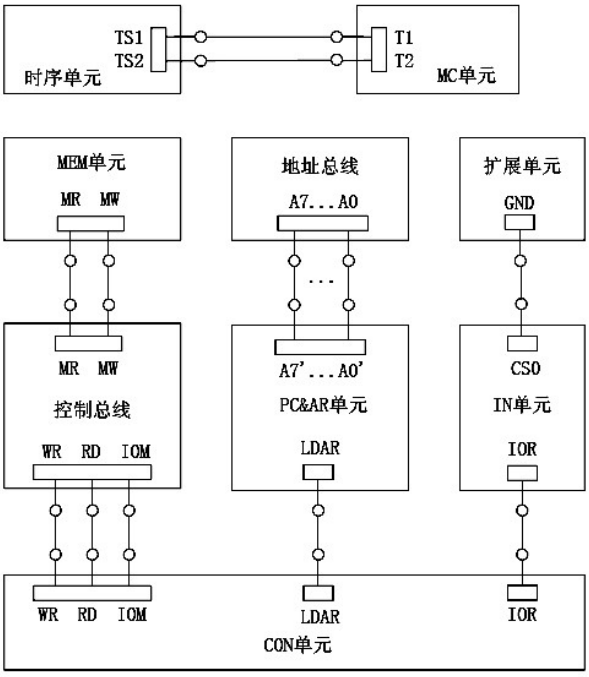


图1-2-4 实验接线图

1.2.4 实验步骤

(1) 关闭实验系统电源，按图1-2-4 连接实验电路，并检查无误，图中将用户需要连接的信号用圆圈标明。

(2) 将时序单元的状态开关置为‘单步’档（时序单元的介绍见附录二）,MEM 单元的编程开关置为‘运行’档。

(3) 将CON 单元的IOR 开关置为1（使IN 单元无输出），打开电源开关，如果听到有‘嘀’报警声，说明有总线竞争现象，应立即关闭电源，重新检查接线，直到错误排除。

(4) 给存储器的00H、01H、02H、03H、04H 地址单元中分别写入数据11H、12H、13H、14H、15H。由前面的存储器实验原理图（图1-2-3）可以看出，由于数据和地址由同一个数据开关给出，因此数据和地址要分时写入，先写地址，具体操作步骤为：先关掉存储器的读写（WR=0，RD=0），然后利用数据开关设定地址，输出地址到数据总线（IOR=0），最后打开地址寄存器门控信号（LDAR=1），按动TS 产生T2 脉冲，即将地址打入到AR 中。再写数据，具体操作步骤为：先关掉存储器的读写（WR=0，RD=0）和地址寄存器门控信号（LDAR=0），然后利用数据开关给出要写入的数据，输出数据到数据总线（IOR=0），最后使存储器处于写状态（WR=1，RD=0，IOM=0），按动TS 产生T2 脉冲，即将数据打入到存储器中。写存储器的流程如图1-2-5 所示（以向00 地址单元写入11H 为例）：

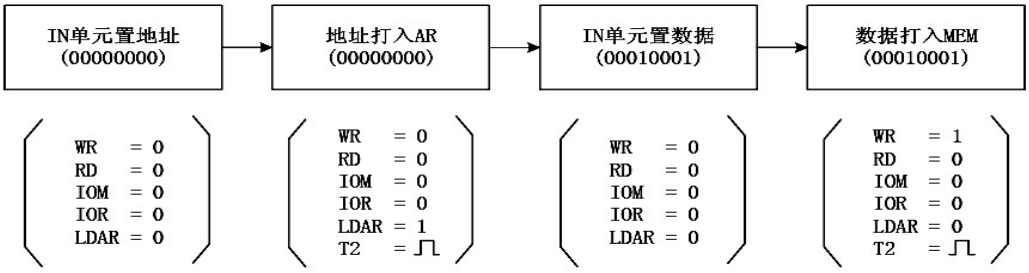


图1-2-5 写存储器流程图

(5) 依次读出第00、01、02、03、04 号单元中的内容，观察上述各单元中的内容是否与前面写入的一致。同写操作类似，也要先给出地址，然后进行读，地址的给出和前面一样，而在进行读操作时，应先关闭IN 单元的输出（IOR=1），然后使存储器处于读状态（WR=0，RD=1， IOM=0），此时数据总线上的数即为从存储器当前地址中读出的数据内容。读存储器的流程如图1-2-6 所示（以从00 地址单元读出11H 为例）：

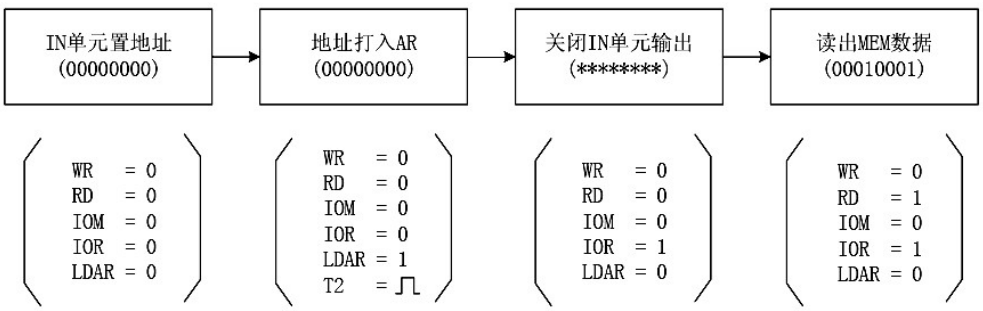


图1-2-6 读存储器流程图

如果实验箱和PC 联机操作，则可通过软件中的数据通路图来观测实验结果（软件使用

说明请看附录1），方法是：打开软件，选择联机软件的“【实验】—【存储器实验】”，打开存储器实验的数据通路图，如图1-2-7 所示。

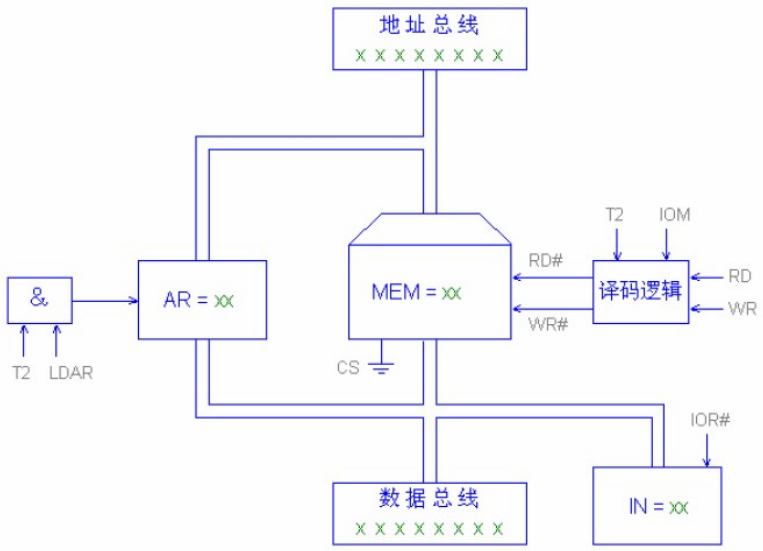


图1-2-7 数据通路图

进行上面的手动操作，每按动一次TS 按钮，数据通路图会有数据的流动，反映当前存储器所做的操作（即使是对存储器进行读，也应按动一次TS 按钮，数据通路图才会有数据流动），或在软件中选择“【调试】—【单周期】”，其作用相当于将时序单元的状态开关置为‘单步’档后按动了一次TS 按钮，数据通路图也会反映当前存储器所做的操作，借助于数据通路图，仔细分析SRAM 的读写过程。

思考题：

1. 随机存储器和只读存储器的区别是什么，能否通过外加电路实现用随机存储器代替只读存储器？
2. 动态随机存储器和静态随机存储器的区别是什么，与静态随机存储器相比，动态随机存储器在电路设计需要考虑什么问题？

实验报告要求：

1. 实验记录：所有的运算结果，故障现象及排出经过；
2. 试解答思考题的问题；
3. 通过本次实验的收获及想法。

1.3 微程序控制器实验

1.3.1 实验目的

(1) 掌握微程序控制器的组成原理。

(2) 掌握微程序的编制、写入，观察微程序的运行过程。

1.3.2 实验设备

PC 机一台，TD-CM3+实验系统一套。

1.3.3 实验原理

微程序控制器的基本任务是完成当前指令的翻译和执行，即将当前指令的功能转换成可以控制的硬件逻辑部件工作的微命令序列，完成数据传送和各种处理操作。它的执行方法就是将控制各部件动作的微命令的集合进行编码，即将微命令的集合仿照机器指令一样，用数字代码的形式表示，这种表示称为微指令。这样就可以用一个微指令序列表示一条机器指令，这种微指令序列称为微程序。微程序存储在一种专用的存储器中，称为控制存储器，微程序控制器原理框图如图1-4-1 所示。

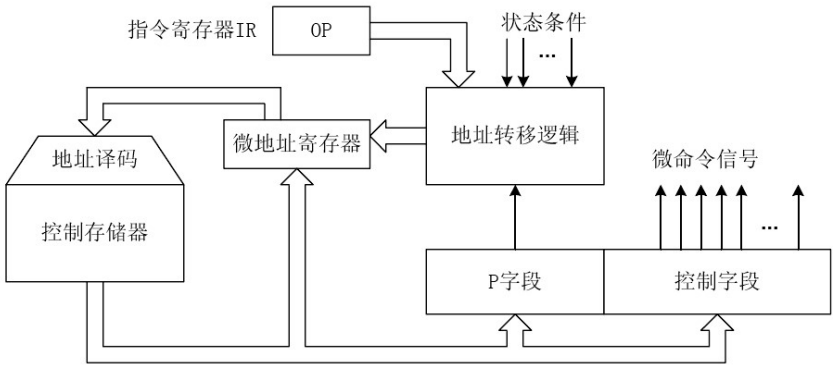


图1-3-1 微程序控制器组成原理框图

控制器是严格按照系统时序来工作的，因而时序控制对于控制器的设计是非常重要的，从前面的实验可以很清楚地了解时序电路的工作原理，本实验所用的时序由时序单元来提供，分为两拍TS1 和TS2，时序单元的介绍见附录2。

微程序控制器的组成见图1-3-2，其中控制存储器采用3 片2816 的E^2PROM，具有掉电保护功能，微命令寄存器18 位，用两片8D触发器（273）和一片4D（175）触发器组成。微地址寄存器6 位，用三片正沿触发的双D触发器（74）组成，它们带有清“0”端和预置端。在不判别测试的情况下，T1 时刻打入微地址寄存器的内容即为下一条微指令地址。当T2 时刻进行测试判别时，转移逻辑满足条件后输出的负脉冲通过强置端将某一触发器置为“1”状态，完成地址修改。

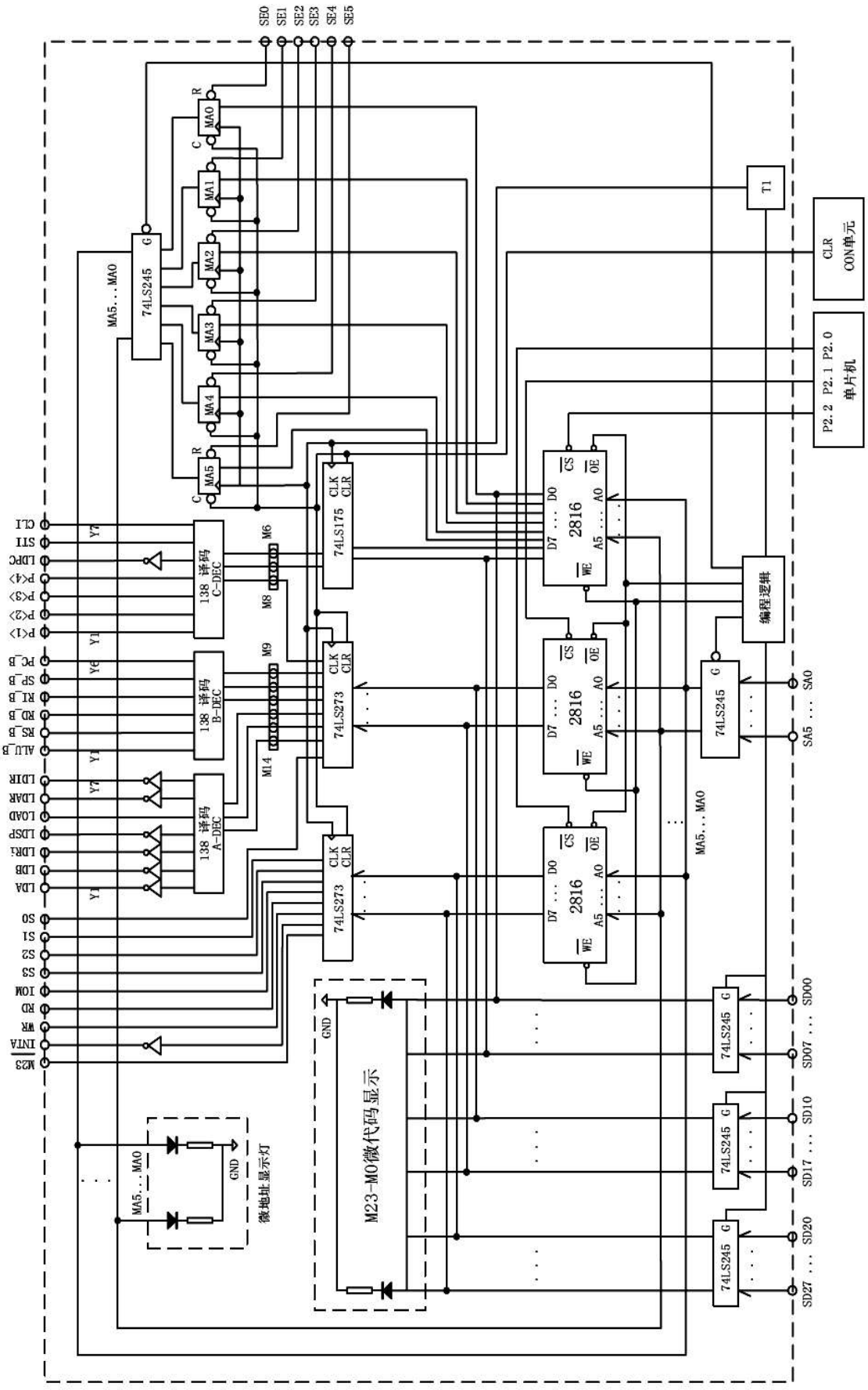


图1-3-2 微程序控制器原理图

在该实验电路中设有一个编程开关（位于实验平台左边中部），它具有三种状态：编程、校验、运行。处于‘编程’状态时，可根据微地址和微指令格式将微指令二进制代码写入到控制存储器2816 中。处于‘校验’状态时，可对写入控制存储器中的二进制代码进行验证，判断写入的二进制代码是否正确。处于‘运行’状态时，给出微程序的入口微地址，可根据微程序流程图自动执行微程序。图中微地址寄存器输出端增加了一组三态门，目的是隔离触发器的输出，增加抗干扰能力，并用来驱动微地址显示灯。

微指令字长共24 位，控制位顺序如表1-3-1：

表1-3-1 微指令格式

23	22	21	20	19	18-15	14-12	11-9	8-6	5-0
M23	M22	WR	RD	IOM	S3-S0	A字段	B字段	C字段	MA5-MA0

A字段

14	13	12	选择
0	0	0	NOP
0	0	1	LDA
0	1	0	LDB
0	1	1	LDR0
1	0	0	保留
1	0	1	保留
1	1	0	保留
1	1	1	LDIR

B字段

14	13	12	选择
0	0	0	NOP
0	0	1	ALU-B
0	1	0	R0-B
0	1	1	保留
1	0	0	保留
1	0	1	保留
1	1	0	保留
1	1	1	保留

C字段

14	13	12	选择
0	0	0	NOP
0	0	1	P<1>
0	1	0	保留
0	1	1	保留
1	0	0	保留
1	0	1	保留
1	1	0	保留
1	1	1	保留

其中MA5…MA0为6位的后续微地址，A、B、C 为三个译码字段，分别由三个控制位译码出多位。C 字段中的P<1>为测试字位。其功能是根据机器指令及相应微代码进行译码，使微程序转入相应的微地址入口，从而实现完成对指令的识别，并实现微程序的分支，本系统上的指令译码原理如图1-3-3 所示，图中I7…I2 为指令寄存器的第7…2 位输出，SE5…SE0 为微控器单元微地址锁存器的强置端输出，指令译码逻辑在IR 单元的INS_DEC（GAL20V8）中实现。

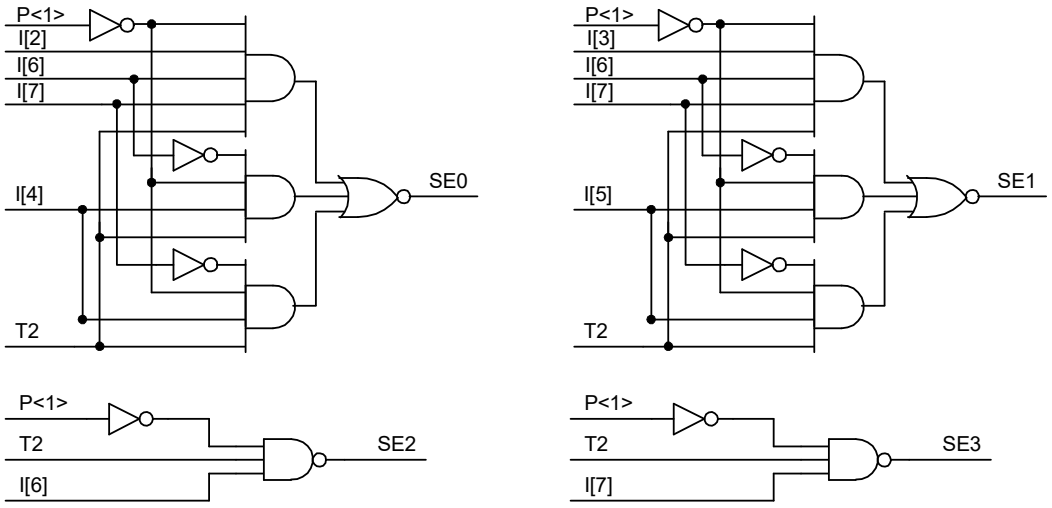


图1-3-3 指令译码原理图

从图1-3-2 中也可以看出，微控器产生的控制信号比表1-3-1 中的要多，这是因为实验的不同，所需的控制信号也不一样，本实验只用了部分的控制信号。

本实验除了用到指令寄存器（IR）和通用寄存器R0 外，还要用到IN 和OUT 单元，从微

控器出来的信号中只有IOM、WR 和RD 三个信号，所以对这两个单元的读写信号还应先经过译码，其译码原理如图1-3-4 所示。IR 单元的原理图如图1-3-5 所示，R0 单元原理如图1-3-7 所示， IN 单元的原理图见图1-2-3 所示，OUT 单元的原理图见图1-3-6 所示。

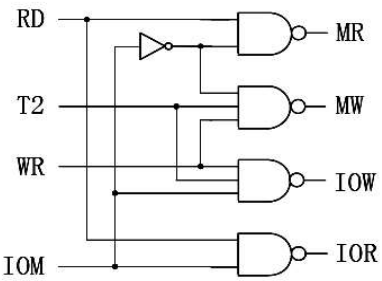


图1-3-4 读写控制逻辑

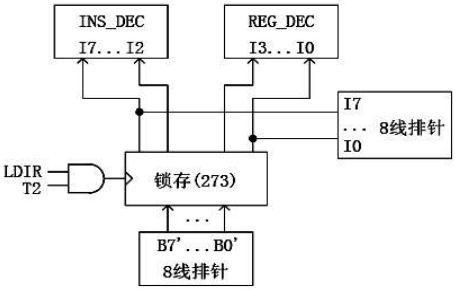


图1-3-5 IR 单元原理图

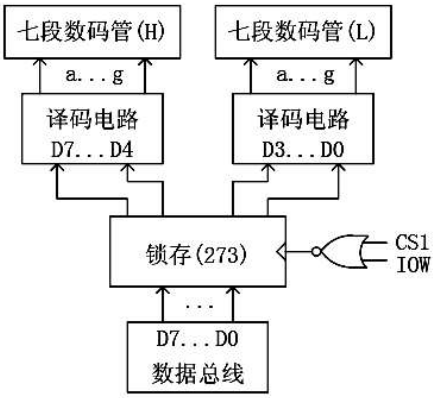


图1-3-6 OUT 单元原理图

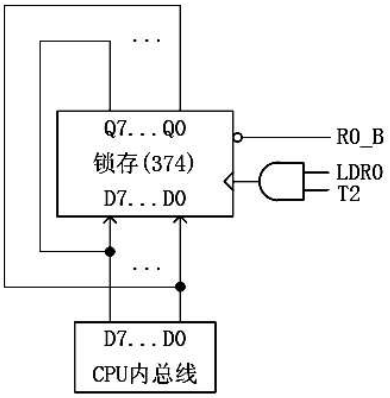


图1-3-7 R0 原理图

本实验安排了四条机器指令，分别为ADD（0000 0000）、IN（0010 0000）、OUT（0011 0000）和HLT（0101 0000），括号中为各指令的二进制代码，指令格式如下：

助记符	机器指令码	说明
IN	0010 0000	IN→R0
ADD	0000 0000	R0+R0→R0
OUT	0011 0000	R0→OUT
HLT	0101 0000	停机

实验中机器指令由CON 单元的二进制开关手动给出，其余单元的控制信号均由微程序控制器自动产生，为此可以设计出相应的数据通路图，见图1-3-8 所示。

几条机器指令对应的参考微程序流程图如图1-3-9 所示。图中一个矩形方框表示一条微指令，方框中的内容为该指令执行的微操作，右上角的数字是该条指令的微地址，右下角的数字是该条指令的后续微地址，所有微地址均用16 进制表示。向下的箭头指出了下一条要执行的指令。P<1>为测试字，根据条件使微程序产生分支。

将全部微程序按微指令格式变成二进制微代码，可得到表1-3-2 的二进制代码表。

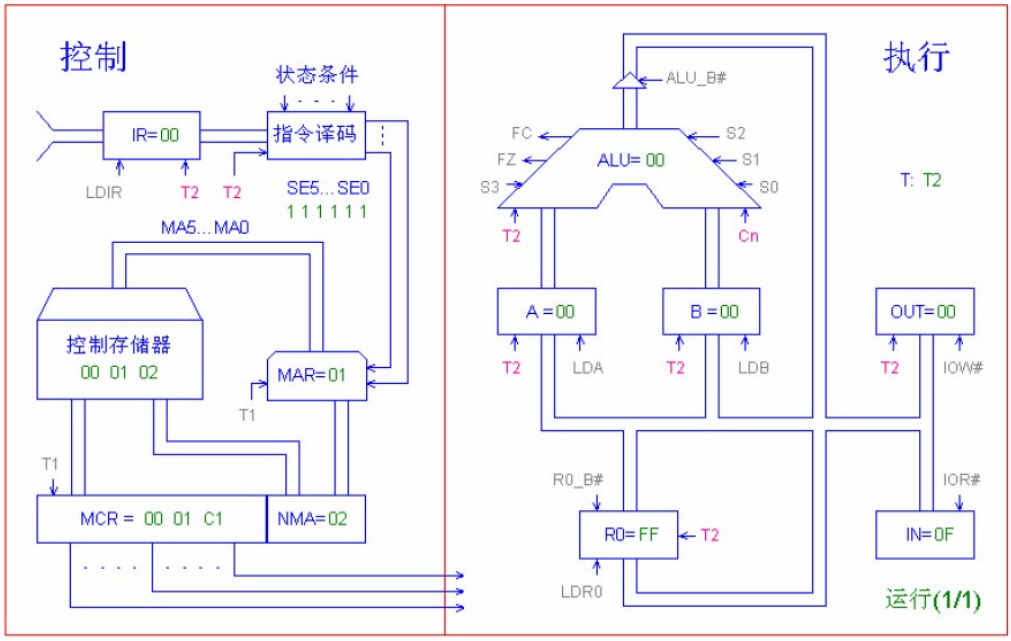
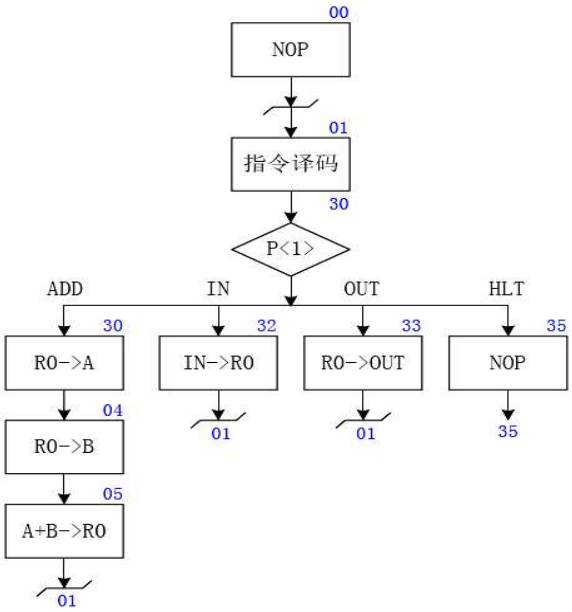


图1-3-8 数据通路图



1-3-9 微程序流程图

表1-3-2 二进制微代码表

地址	十六进制	高五位	S3-S0	A字段	B字段	C字段	MA5-MA0
00	00 00 01	00000	0000	000	000	000	000001
01	00 70 70	00000	0000	111	000	001	110000
04	00 24 05	00000	0000	010	010	000	000101
05	04 B2 01	00000	1001	011	001	000	000001
30	00 14 04	00000	0000	001	010	000	000100
32	18 30 01	00011	0000	011	000	000	000001
33	28 04 01	00101	0000	000	010	000	000001
35	00 00 35	00000	0000	000	000	000	110101

3.2.4 实验步骤

1. 按图1-3-10 所示连接实验线路，仔细查线无误后接通电源。如果有‘滴’报警声，说明总线有竞争现象，应关闭电源，检查接线，直到错误排除。

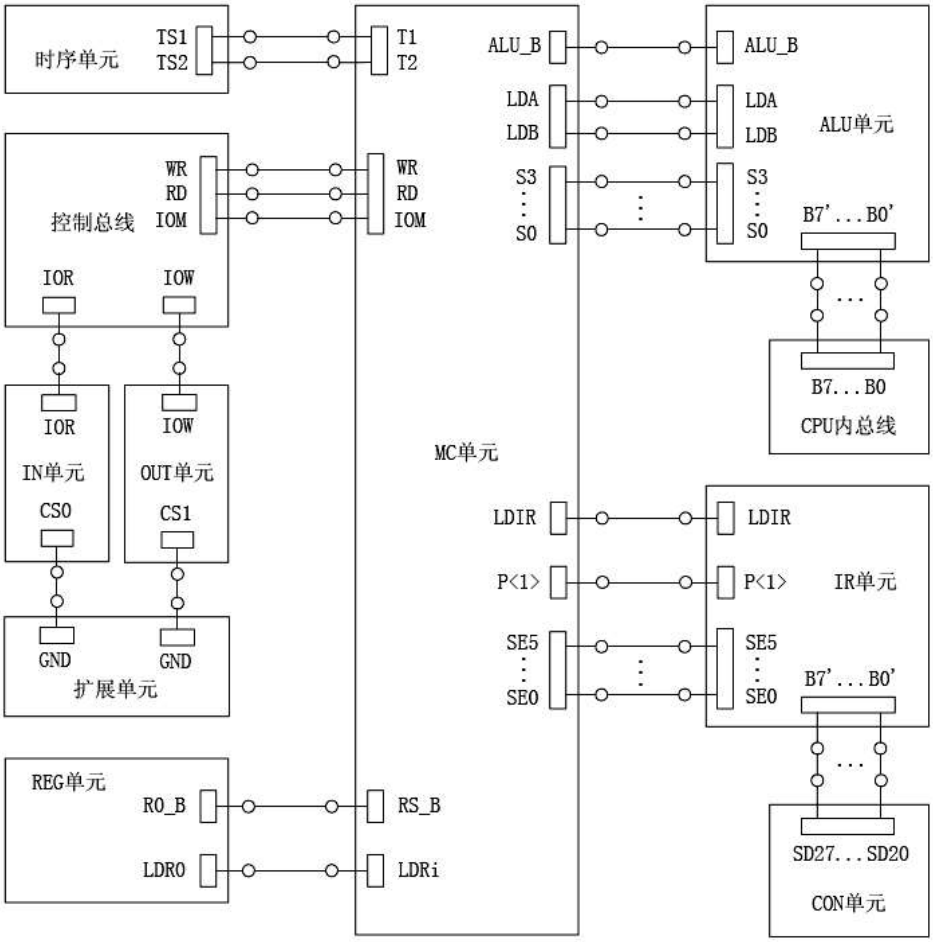


图1-3-10 实验接线图

2. 对微控器进行读写操作，分两种情况：手动读写和联机读写。

1）手动读写

进行手动读或是写，都需要手动给出地址，系统专门安排了一个ADDR 单元，做为地址输入。ADDR 单元原理如图1-3-11 所示，可以看出本单元实为一个加减计数器。当开关为‘加1’档时，在T2 的下沿计数器进行加1 计数，当开关为‘减1’档时，在T2 的下沿计数器进行减1计数，当开关置为‘置数’档时，计数器置初值，其作用相当于直通，SA7…SA0 的输出值就是二进制开关组的值。

在实验中选择什么档位，取决于写入数据的地址是否连续，如果是连续地址，选择‘加1’或是‘减1’档会方便一些。如果是离散地址，选择‘置数’档会方便一些。

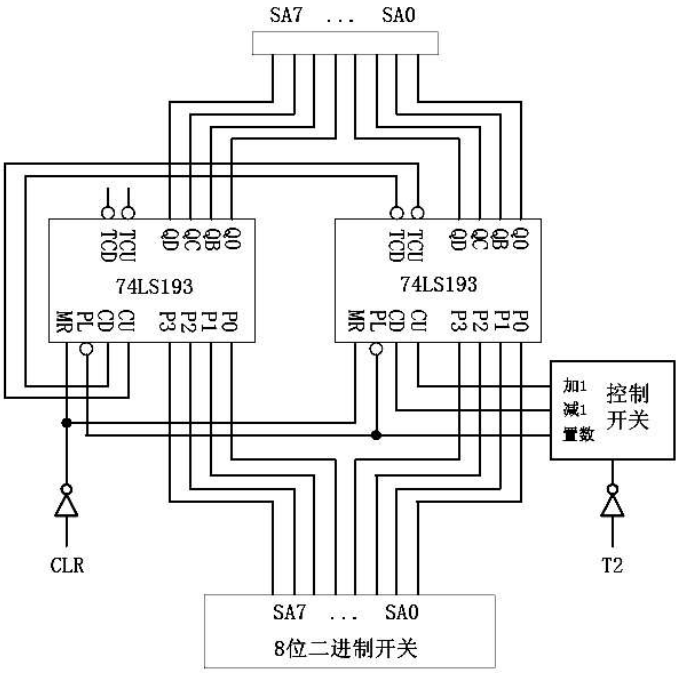


图1-3-11 ADDR 单元原理图

(1) 手动对微控器进行编程（写）

① 按图1-3-12 接好线（部分实验接线被改变，手动读写微控器完成后应予恢复） 。

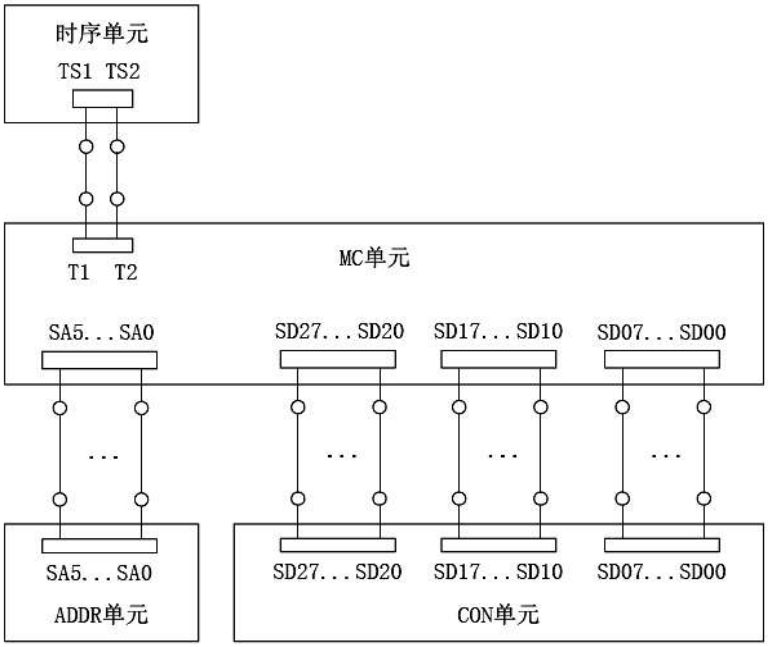


图1-3-12 手动读写微控器接线图

② 将MC 单元编程开关置为‘编程’档，时序单元状态开关置为‘单步’档，ADDR 单元状态开关置为‘置数’档。

③ 使用ADDR 单元的低6 位SA5…SA0 给出微地址MA5…MA0，微地址可以通过MC 单元的MA5…MA0 微地址灯显示。

④ CON 单元SD27…SD20、SD17…SD10、SD07…SD00 开关上置24 位微代码，待写入值由MC 单元的M23…M0 二十四位LED 灯显示。

⑤ 启动时序电路（按动一次TS 按钮），即将微代码写入到E PROM 2816 的相应地址对应的单元中。

⑥ 重复③、④、⑤三步，将表1-3-2 的微代码写入2816 芯片中。

(2) 手动对微控器进行校验（读）

① 接线方法和编程一样。

② 将MC 单元编程开关置为‘校验’档，时序单元状态开关置为‘单步’档，ADDR 单元状态开关置为‘置数’档。

③ 使用ADDR 单元的低6 位SA5…SA0 给出微地址MA5…MA0，微地址可以通过MC 单元的MA5…MA0 微地址灯显示。相应地址单元的数据将会被读出，并在MC 单元的M23…M0 二十四位LED 灯显示。重复本步，检查2816 芯片中相应地址单元的数据是否和表1-3-2 中的二进制数据相同，如果不同，则说明写入操作失败，应重新写入。

2) 联机读写

(1) 将微程序写入文件

联机软件提供了微程序下载功能，以代替手动读写微控器，但微程序得以指定的格式写入到以TXT 为后缀的文件中，微程序的格式如下：

注意，$M XX XXXXXX之间间隔必须为四个空格，不能用TAB键



如$M 1F 112233，表示微指令的地址为1FH，微指令值为11H（高）、22H（中）、33H（低），本次实验的微程序如下，其中分号‘;’为注释符，分号后面的内容在下载时将被忽略掉。

```
; //****************************************//
; //                                        //
; //          微控器实验指令文件            //
; //                                        //
; //****************************************//
; //******* Start Of MicroController Data ********//
  $M 00 000001   ; NOP
  $M 01 007070   ; CON(INS)->IR, P<1>
  $M 04 002405   ; R0->B
  $M 05 04B201   ; A 加 B->R0
  $M 30 001404   ; R0->A
  $M 32 183001   ; IN->R0
  $M 33 280401   ; R0->OUT
  $M 35 000035   ; NOP
; //******* End Of MicroController Data ******** //
```

(2) 写入微程序

用联机软件的“【转储】—【装载】”功能将该格式（*.TXT）文件装载入实验系统。

装入过程中，在软件的输出区的‘结果’栏会显示装载信息，如当前正在装载的是机器指令还是微指令，还剩多少条指令等。

(3) 校验微程序

选择联机软件的“【转储】—【刷新指令区】”可以读出下位机所有的机器指令和微指令，并在指令区显示。检查微控器相应地址单元的数据是否和表1-3-2 中的十六进制数据相同，如果不同，则说明写入操作失败，应重新写入，可以通过联机软件单独修改某个单元的微指令，先用鼠标左键单击指令区的‘微存’TAB 按钮，然后再单击需修改单元的数据，此时该单元变为编辑框，输入6 位数据并回车，编辑框消失，并以红色显示写入的数据。

3. 运行微程序

运行时也分两种情况：本机运行和联机运行。

1) 本机运行

① 将MC 单元的编程开关置为‘运行’档，MEM 单元的编程开关也置为‘运行’档，按动CON 单元的CLR 按钮，将微地址寄存器（MAR）清零，同时也将指令寄存器（IR）、ALU 单元的暂存器A 和暂存器B 清零。

② 将时序单元的状态开关置为‘单拍’档，然后按动TS 按钮，体会系统在T1 和T2 节拍中各做的工作。T1 节拍微控器将后续微地址（下条执行的微指令的地址）打入微地址寄存器，当前微指令打入微指令寄存器，并产生执行部件相应的控制信号；T2 节拍根据T1 节拍产生的控制信号做出相应的执行动作，如果测试位有效，还要根据机器指令及当前微地址寄存器中的内容进行译码，使微程序转入相应的微地址入口，实现微程序的分支。

③ 按动CON 单元的CLR 按钮，清零微地址寄存器（MAR）等，并将时序单元的状态开关置为‘单步’档。

④ 置IN 单元数据为00100011，按动TS 按钮，当MC 单元后续微地址显示为000001 时，在CON 单元的SD27…SD20 模拟给出IN 指令00100000 并继续单步执行，当MC 单元后续微地址显示为000001 时，说明当前指令已执行完；在CON 单元的SD27…SD20 给出ADD 指令00000000，该指令将会在下个T2 被打入指令寄存器（IR），它将R0 中的数据和其自身相加后送R0;接下来在CON 单元的SD27…SD20 给出OUT 指令00110000 并继续单步执行，在MC 单元后续微地址显示为000001 时，观查OUT 单元的显示值是否为01000110。

2) 联机运行

联机运行时，进入软件界面，在菜单上选择【实验】—【微控器实验】，打开本实验的数据通路图，也可以通过工具栏上的下拉框打开数据通路图，数据通路图如图1-3-8 所示。

将MC 单元的编程开关置为‘运行’档，MEM 单元的编程开关也置为‘运行’档，按动CON 单元的总清开关后，按动软件中单节拍按钮，当后续微地址（通路图中的MAR）为000001 时，置CON 单元SD27…SD20，产生相应的机器指令，该指令将会在下个T2 被打入指令寄存器（IR），在后面的节拍中将执行这条机器指令。仔细观察每条机器指令的执行过程，体会后续微地址被强置转换的过程，这是计算机识别和执行指令的根基。也可以打开微程序流程图，跟踪显示每条机器指令的执行过程。

按本机运行的顺序给出数据和指令，观查最后的运算结果是否正确。

思考题：

1. 微指令的编码格式有几种，本设计采用了那种编码格式？
2. 根据图1-3-3 中指令译码原理，列出指令操作码I7…I2和为微地址锁存器MAR3…MAR0的逻辑关系表。

实验报告要求：

1. 实验记录：所有的运算结果，故障现象及排出经过；
2. 试解答思考题的问题；
3. 通过本次实验的收获及想法。

1.4 CPU 与简单模型机设计实验

1.4.1 实验目的

(1) 掌握一个简单CPU 的组成原理。

(2) 在掌握部件单元电路的基础上，进一步将其构造一台基本模型计算机。

(3) 为其定义五条机器指令，编写相应的微程序，并上机调试掌握整机概念。

1.4.2 实验设备

PC 机一台，TD-CM3+实验系统一套。

1.4.3 实验原理

本实验要实现一个简单的CPU，并且在此CPU 的基础上，继续构建一个简单的模型计算机。CPU 由运算器（ALU）、微程序控制器（MC）、通用寄存器（RO），指令寄存器（IR）、程序计数器（PC）和地址寄存器（AR）组成,如图2-1-1 所示。这个CPU 在写入相应的微指令后，就具备了执行机器指令的功能，但是机器指令一般存放在主存当中，CPU 必须和主存挂接后，才有实际的意义，所以还需要在该CPU 的基础上增加一个主存和基本的输入输出部件，以构成一个简单的模型计算机。

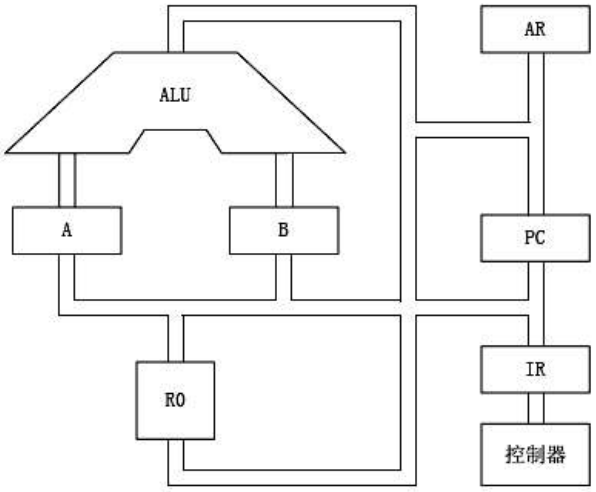


图1-4-1 基本CPU 构成原理图

除了程序计数器（PC），其余部件在前面的实验中都已用到，在此不再讨论。系统的程序计数器（PC）由两片74LS161 和一片74LS245 构成，其原理如图1-4-2 所示。PC_B 为三态门的输出使能端，CLR 连接至CON 单元的总清端CLR，按下CLR 按钮，将使PC 清零，LDPC 和T2 相与后作为计数器的计数时钟，当LOAD 为低时，计数时钟到来后将CPU 内总线上的数据打入PC。

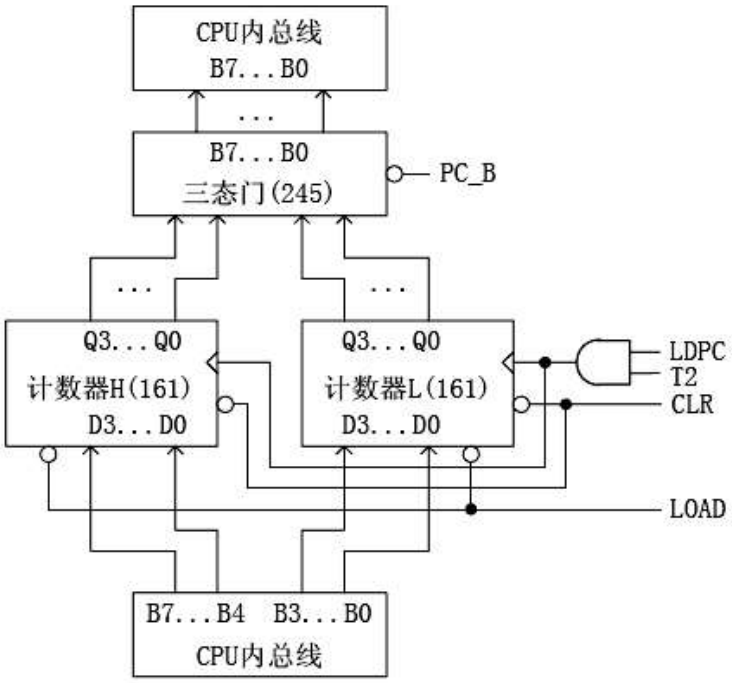


图1-4-2 程序计数器(PC)原理图

本模型机和前面微程序控制器实验相比，新增加一条跳转指令JMP，共有五条指令：IN（输入）、ADD（二进制加法）、OUT（输出）、JMP（无条件转移），HLT（停机），其指令格式如下（高 4 位为操作码）：

助记符	机器指令码	说明
IN	0010 0000	IN→R0
ADD	0000 0000	R0 + R0→R0
OUT	0011 0000	R0→OUT
JMP addr	1100 0000 ********	addr→ PC
HLT	0101 0000	停机

其中JMP 为双字节指令，其余均为单字节指令，********为addr 对应的二进制地址码。微程序控制器实验的指令是通过手动给出的，现在要求CPU 自动从存储器读取指令并执行。根据以上要求，设计数据通路图，如图1-4-3 所示。

本实验在前一个实验的基础上增加了三个部件，一是PC（程序计数器），另一个是AR（地址寄存器），还有就是MEM（主存）。因而在微指令中应增加相应的控制位，其微指令格式如表1-4-1 所示。

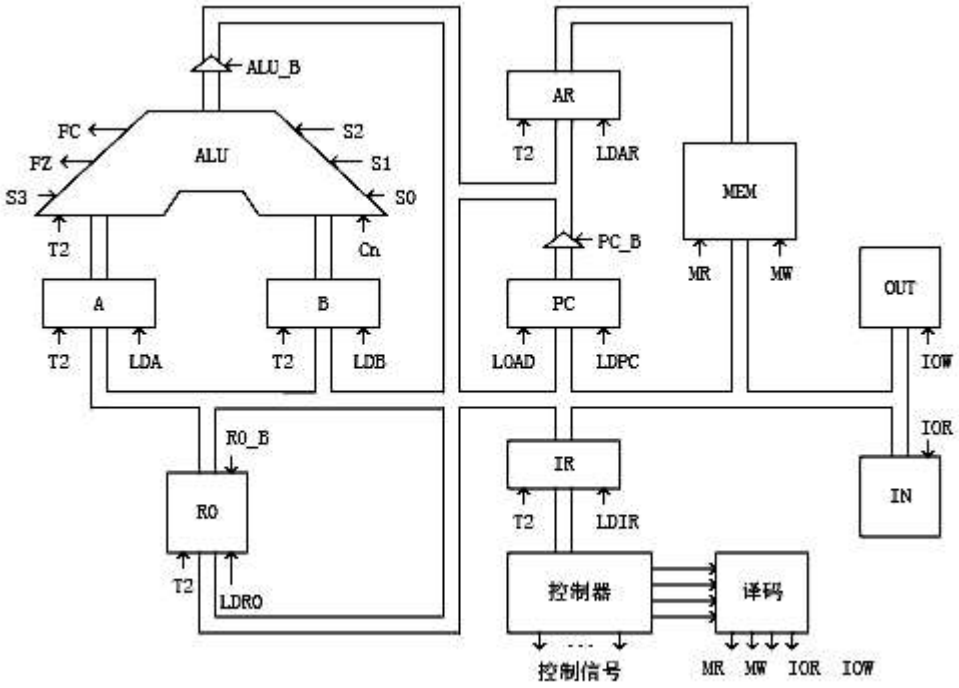


图1-4-3 数据通路图

表1-4-1 微指令格式

23	22	21	20	19	18-15	14-12	11-9	8-6	5-0
M23	M22	WR	RD	IOM	S3-S0	A字段	B字段	C字段	MA5-MA0

A字段

14	13	12	选择
0	0	0	NOP
0	0	1	LDA
0	1	0	LDB
0	1	1	LDR0
1	0	0	保留
1	0	1	LOAD
1	1	0	LDAR
1	1	1	LDIR

B字段

14	13	12	选择
0	0	0	NOP
0	0	1	ALU-B
0	1	0	R0-B
0	1	1	保留
1	0	0	保留
1	0	1	保留
1	1	0	PC-B
1	1	1	保留

C字段

14	13	12	选择
0	0	0	NOP
0	0	1	P<1>
0	1	0	保留
0	1	1	保留
1	0	0	保留
1	0	1	LDPC
1	1	0	保留
1	1	1	保留

系统涉及到的微程序流程见图1-4-4 所示，当拟定“取指”微指令时，该微指令的判别测试字段为P<1>测试。指令译码原理见图1-3-3 所示，由于“取指”微指令是所有微程序都使用的公用微指令，因此P<1> 的测试结果出现多路分支。本机用指令寄存器的高6 位（IR7—IR2）作为测试条件，出现 5 路分支，占用 5 个固定微地址单元，剩下的其它地方就可以一条微指令占用控存一个微地址单元随意填写，微程序流程图上的单元地址为16 进制。

当全部微程序设计完毕后，应将每条微指令代码化，表1-4-2 即为将图1-4-2 的微程序流程图按微指令格式转化而成的“二进制微代码表”。

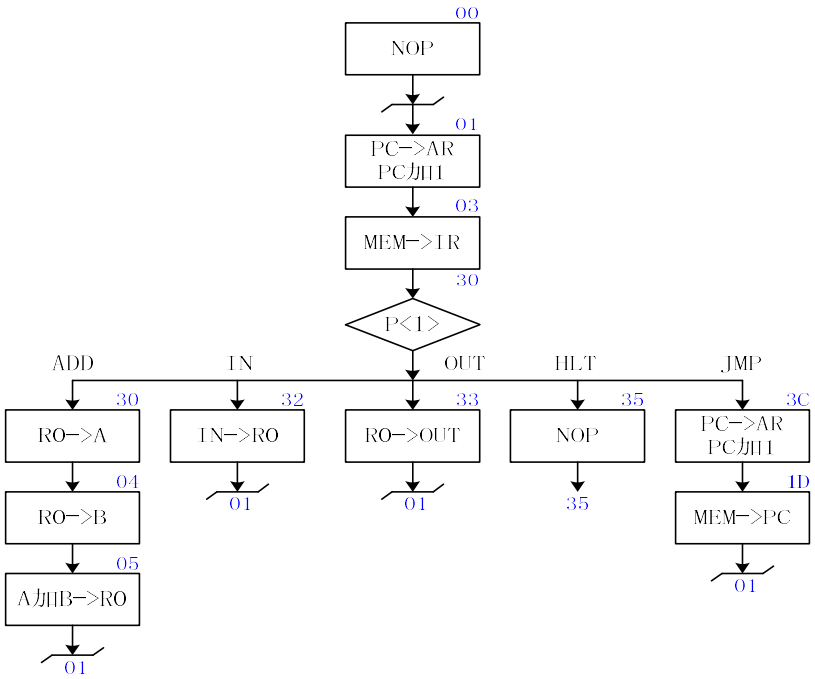


图1-4-4 简单模型机微程序流程图

表1-4-2 二进制微代码表

地址	十六进制	高五位	S3-S0	A字段	B字段	C字段	MA5-MA0
00	00 00 01	00000	0000	000	000	000	000001
01	00 6D 43	00000	0000	110	110	101	000011
03	10 70 70	00010	0000	111	000	001	110000
04	00 24 05	00000	0000	010	010	000	000101
05	04 B2 01	00000	1001	011	001	000	000001
1D	10 51 41	00010	0000	101	000	101	000001
30	00 14 04	00000	0000	001	010	000	000100
32	18 30 01	00011	0000	011	000	000	000001
	28 04 01	00101	0000	000	010	000	000001
35	00 00 35	00000	0000	000	000	000	110101
3C	00 6D 5D	00000	0000	110	110	101	011101

设计一段机器程序，要求从IN 单元读入一个数据，存于R0，将R0 和自身相加，结果存于R0，再将R0 的值送OUT 单元显示。根据要求可以得到如下程序，地址和内容均为二进制数。

地址	内容	助记符	说明
00000000	00100000	; START: IN R0	从 IN 单元读入数据送 R0
00000001	00000000	; ADD R0,R0	R0 和自身相加，结果送 R0
00000010	00110000	; OUT R0	R0 的值送 OUT 单元显示
00000011	11100000	; JMP START	跳转至 00H 地址
00000100	00000000		
00000101	01010000	; HLT	停机

1.5.4 实验步骤

1. 按图1-4-5 连接实验线路。

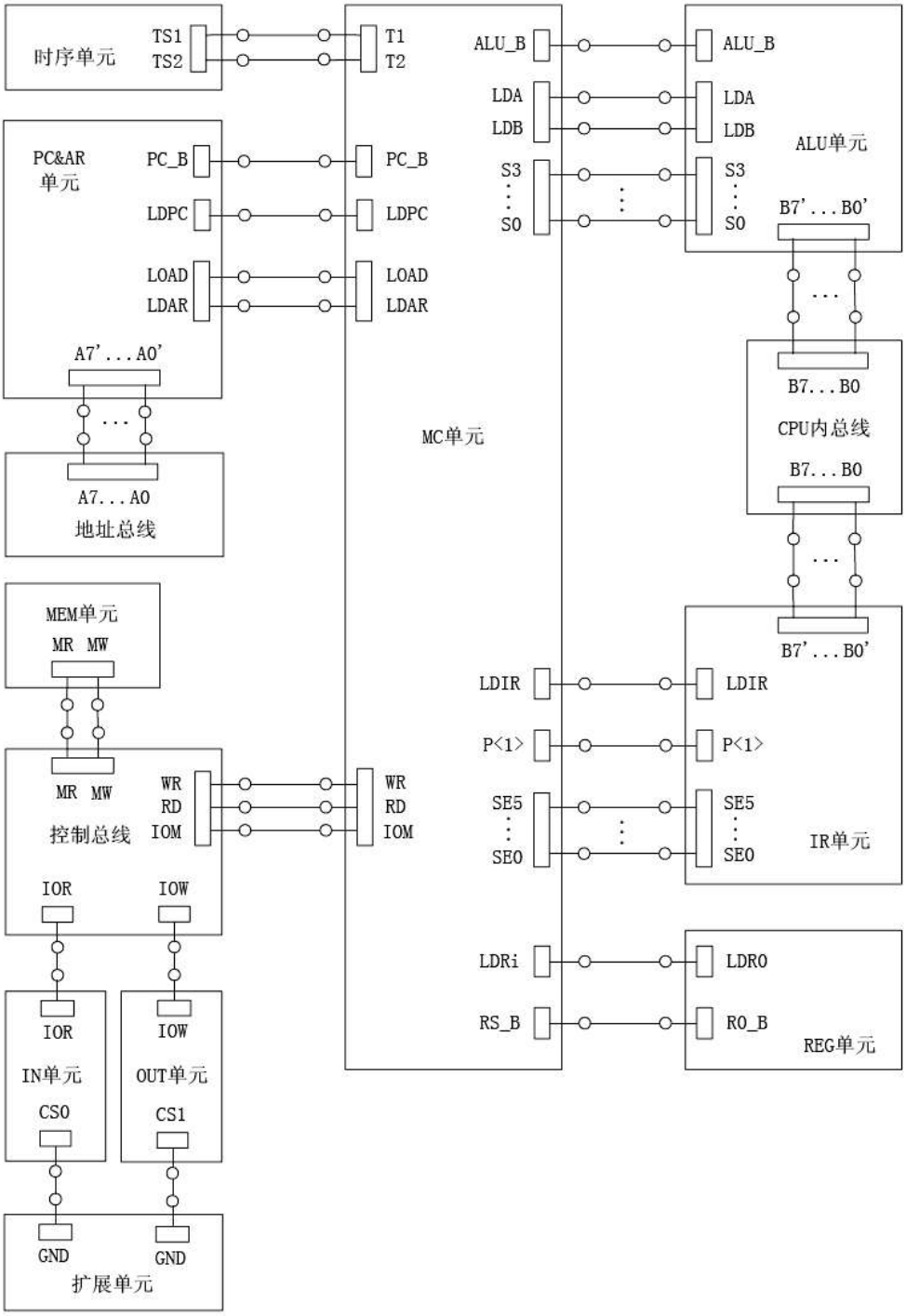


图1-4-5 实验接线图

2. 写入实验程序，并进行校验，分两种方式，手动写入和联机写入。

1）手动写入和校验

(1) 手动写入微程序

① 按图1-3-12 接好线。

② 将MC 单元编程开关置为‘编程’档，时序单元状态开关置为‘单步’档，ADDR 单元状态开关置为‘置数’档。

③ 使用ADDR 单元的低6 位SA5…SA0 给出微地址MA5…MA0，微地址可以通过MC 单元的MA5…MA0 微地址灯显示。

④ CON 单元SD27…SD20、SD17…SD10、SD07…SD00 开关上置24 位微代码，待写入值由MC 单元的M23…M0 二十四位LED 灯显示。

⑤ 启动时序电路（按动一次TS 按钮），即将微代码写入到E PROM 2816 的相应地址对应的单元中。

⑥ 重复③、④、⑤三步，将表1-4-2 的微代码写入2816 芯片中。

(2) 手动校验微程序

① 接线方法和编程一样。

② 将MC 单元编程开关置为‘校验’档，时序单元状态开关置为‘单步’档，ADDR 单元状态开关置为‘置数’档。

③ 使用ADDR 单元的低6 位SA5…SA0 给出微地址MA5…MA0，微地址可以通过MC 单元的MA5…MA0 微地址灯显示。相应地址单元的数据将会被读出，并在MC 单元的M23…M0二十四位LED 灯显示。重复本步，检查2816 芯片中相应地址单元的数据是否和表1-4-2 中的二进制数据相同，如果不同，则说明写入操作失败，应重新写入。

(3) 手动写入机器程序

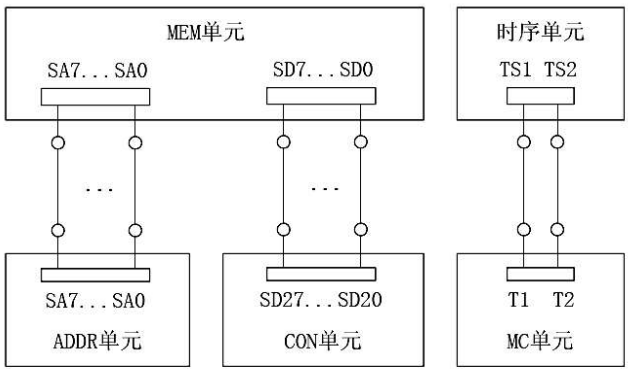


图1-4-6 手动读写存储器接线图

② 将MEM 单元编程开关置为‘编程’档，时序单元状态开关置为‘单步’档，ADDR 单元状态开关置为‘置数’档。

③ 在ADDR 单元给出地址SA7…SA0，地址值通过ADDR 单元的8 位LED 灯观测。

④ 在CON 单元SD27…SD20 开关置待写入数据，值通过数据总线的D7…D0 八位LED 灯观测。

⑤ 启动时序电路（按动一次TS 按钮），将数据写入SRAM 6116 地址对应的单元中。

⑥ 重复③、④、⑤三步，将本实验机器程序写入主存。

(4) 手动校验机器程序

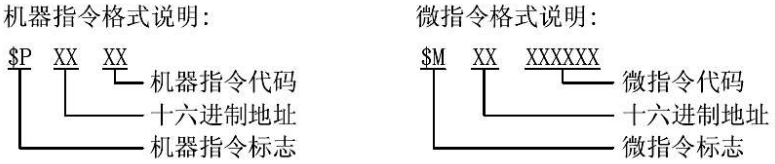
① 接线方法和编程一样。

② 将MEM 单元编程开关置为‘校验’档，时序单元状态开关置为‘单步’档，ADDR 单元状态开关置为‘置数’档。

③ 在ADDR 单元给出地址SA7…SA0，地址值通过ADDR 单元的8 位LED 观测。相应地址单元的数据将会被读出，且在数据总线的D7…D0 八位LED 灯显示。

2) 联机写入和校验

联机软件提供了微程序和机器程序下载功能，以代替手动读写微程序和机器程序，但是微程序和机器程序得以指定的格式写入到以TXT 为后缀的文件中，微程序和机器程序的格式如下：



本次实验程序如下，程序中分号‘；’为注释符，分号后面的内容在下载时将被忽略掉：

```
; //****************************************//
; //                                        //
; //     CPU与简单模型机实验指令文件        //
; //                                        //
; //****************************************//

; //***** Start Of Main Memory Data *****//

  $P 00 20    ; START: IN  R0    从IN单元读入数据送R0
  $P 01 00    ; ADD R0,R0        R0和自身相加，结果送R0
  $P 02 30    ; OUT R0           R0的值送OUT单元显示
  $P 03 E0    ; JMP START        跳转至00H地址
  $P 04 00    ;
  $P 05 50    ; HLT              停机

; //***** End Of Main Memory Data *****//


; //** Start Of MicroController Data **//

  $M 00 000001    ; NOP
  $M 01 006D43    ; PC->AR,PC加1
  $M 03 107070    ; MEM->IR, P<1>
  $M 04 002405    ; R0->B
  $M 05 04B201    ; A加B->R0
  $M 1D 105141    ; MEM->PC
  $M 30 001404    ; R0->A
  $M 32 183001    ; IN->R0
```

```
$M 33 280401     ; RO->OUT
$M 35 000035     ; NOP
$M 3C 006D5D     ; PC->AR,PC加1

; //** End Of MicroController Data **//
```

选择联机软件的“【转储】—【装载】”功能，在打开文件对话框中选择上面所保存的文件，软件自动将机器程序和微程序写入指定单元。

选择联机软件的“【转储】—【刷新指令区】”可以读出下位机所有的机器指令和微指令，并在指令区显示，对照文件检查微程序和机器程序是否正确，如果不正确，则说明写入操作失败，应重新写入，可以通过联机软件单独修改某个单元的指令，以修改微指令为例，先用鼠标左键单击指令区的‘微存’TAB 按钮，然后再单击需修改单元的数据，此时该单元变为编辑框，输入6 位数据并回车，编辑框消失，并以红色显示写入的数据。

3. 运行程序

方法一：本机运行

将MC 单元的编程开关置为‘运行’档，MEM 单元的编程开关也置为‘运行’档，按动CON 单元的总清按钮CLR，将使程序计数器PC、地址寄存器AR 和微程序地址为OOH，程序可以从头开始运行，暂存器A、B，指令寄存器IR 和OUT 单元也会被清零。

将时序单元的状态开关置为‘单步’档，每按动一次TS 按钮，即可单步运行一条微指令，对照微程序流程图，观察微地址显示灯是否和流程一致。每运行完一条微指令，观测一次数据总线和地址总线，对照数据通路图，分析总线上的数据是否正确。

当模型机执行完JMP 指令后，检查OUT 单元显示的数是否为IN 单元值的2 倍，按下CON 单元的总清按钮CLR，改变IN 单元的值，再次执行机器程序，从OUT 单元显示的数判别程序执行是否正确。

方法二：联机运行

将MC 单元的编程开关置为‘运行’档，MEM 单元的编程开关也置为‘运行’档，进入软件界面，选择菜单命令“【实验】—【简单模型机】”，打开简单模型机数据通路图。

按动CON 单元的总清按钮CLR，然后通过软件运行程序，选择相应的功能命令，即可联机运行、监控、调试程序，当模型机执行完JMP 指令后，检查OUT 单元显示的数是否为IN 单元值的2 倍。在数据通路图和微程序流中观测指令的执行过程，并观测软件中地址总线、数据总线以及微指令显示和下位机是否一致。

思考题：

1. 试修改现有的指令系统，将加法指令的功能修改为RO的内容和某个存储单元的内容相加;增加存数、取数和减法三条机器指令，指令助记符分别为 STA、LAD 和SUB，指令操作码分别为十六进制的60、70和80。
2. 利用修改后的指令系统编写一段程序，完成十六位二进制数的加减法运算。

实验报告要求：

1. 实验记录：所有的运算结果，故障现象及排出经过；

2. 试解答思考题的问题；
3. 通过本次实验的收获及想法。

1.5 复杂模型机设计实验

1.5.1 实验目的

综合运用所学计算机组成原理知识，设计并实现较为完整的计算机。

1.5.2 实验设备

PC 机一台，TD-CM3+实验系统一套。

1.5.3 实验原理

下面讲述一下模型计算机的数据格式及指令系统。

1. 数据格式

模型机规定采用定点补码表示法表示数据，字长为 8 位，8 位全用来表示数据（最高位不表示符号），数值表示范围是： $0 \leqslant X \leqslant 2^8 - 1$。

2. 指令设计

模型机设计三大类指令共十五条，其中包括运算类指令、控制转移类指令，数据传送类指令。运算类指令包含三种运算，算术运算、逻辑运算和移位运算，设计有6 条运算类指令，分别为：ADD、AND、INC、SUB、OR、RR，所有运算类指令都为单字节，寻址方式采用寄存器直接寻址。控制转移类指令有三条HLT、JMP、BZC，用以控制程序的分支和转移，其中HLT 为单字节指令，JMP 和BZC 为双字节指令。数据传送类指令有IN、OUT、MOV、LDI、LAD、STA 共6 条，用以完成寄存器和寄存器、寄存器和I/O、寄存器和存储器之间的数据交换，除MOV 指令为单字节指令外，其余均为双字节指令。

3. 指令格式

所有单字节指令（ADD、AND、INC、SUB、OR、RR、HLT 和MOV）格式如下：

7 6 5 4	3 2	1 0
OP-CODE	RS	RD

其中，OP-CODE 为操作码，RS 为源寄存器，RD 为目的寄存器，并规定：

RS或RD	选定的寄存器
00	R0
01	R1
10	R2
11	R3

IN 和OUT 的指令格式为：

7 6 5 4 （1）	3 2（1）	1 0（1）	7—0（2）
OP-CODE	RS	RD	P

其中括号中的1 表示指令的第一字节，2 表示指令的第二字节，OP-CODE 为操作码，RS 为源寄存器，RD 为目的寄存器，P 为I/O 端口号，占用一个字节，系统的I/O 地址译码原

理见图1-5-1（在地址总线单元）。

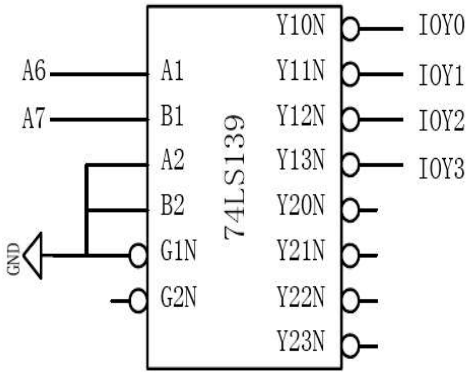


图1-5-1 I/O 地址译码原理图

由于用的是地址总线的高两位进行译码，I/O 地址空间被分为四个区，如表1-5-1 所示：

表1-5-1 I/O 地址空间分配

A7 A6	选定	地址空间
00	IOY0	00-3F
01	IOY1	40-7F
10	IOY2	80-BF
11	IOY3	C0-FF

系统设计五种数据寻址方式，即立即、直接、间接、变址和相对寻址，LDI 指令为立即寻址，LAD、STA、JMP 和BZC 指令均具备直接、间接、变址和相对寻址能力。

LDI 的指令格式如下，第一字节同前一样，第二字节为立即数。

7 6 5 4 （1）	3 2（1）	1 0（1）	7—0（2）
OP-CODE	RS	RD	data

LAD、STA、JMP 和BZC 指令格式如下。

7 6 5 4 （1）	3 2（1）	1 0（1）	7—0（2）
OP-CODE	M	RD	D

其中M 为寻址模式，具体见表1-5-2，以R2 做为变址寄存器RI。

表1-5-2 寻址方式

寻址模式M	有效地址E	说明
00	E=D	直接寻址
01	E=（D）	间接寻址
10	E=（RI）+D	RI变址寻址
11	E=（PC）+D	相对寻址

4．指令系统

本模型机共有15 条基本指令，表1-5-3 列出了各条指令的格式、汇编符号、指令功能。

表1-5-3 指令描述

助记符号	指令格式	指令功能
MOV RD,RS	0100 RS RD	RS → RD
ADD RD,RS	0000 RS RD	RD + RS → RD
SUB RD,RS	1000 RS RD	RD - RS → RD
AND RD,RS	0001 RS RD	RD∧RS → RD
OR RD,RS	1001 RS RD	RD∨RS → RD
RR RD,RS	1010 RS RD	RS右环移 → RD
INC RD	0111 ** RD	RD+1 → RD
LAD M D,RD	1100 M RD D	E → RD
STA M D,RS	1101 M RD D	RD → E
JMP M D	1110 M ** D	E → PC
BZC M D	1111 M ** D	当FC或FZ=1时, E → PC
IN RD,P	0010 ** RD P	[P] → RD
OUT P,RS	0011 RS ** P	RS → [P]
LDI RD,D	0110 ** RD D	D → RD
HALT	0101 ** **	停机

1.6.4 总体设计

本模型机的数据通路框图如图1-5-2 所示。

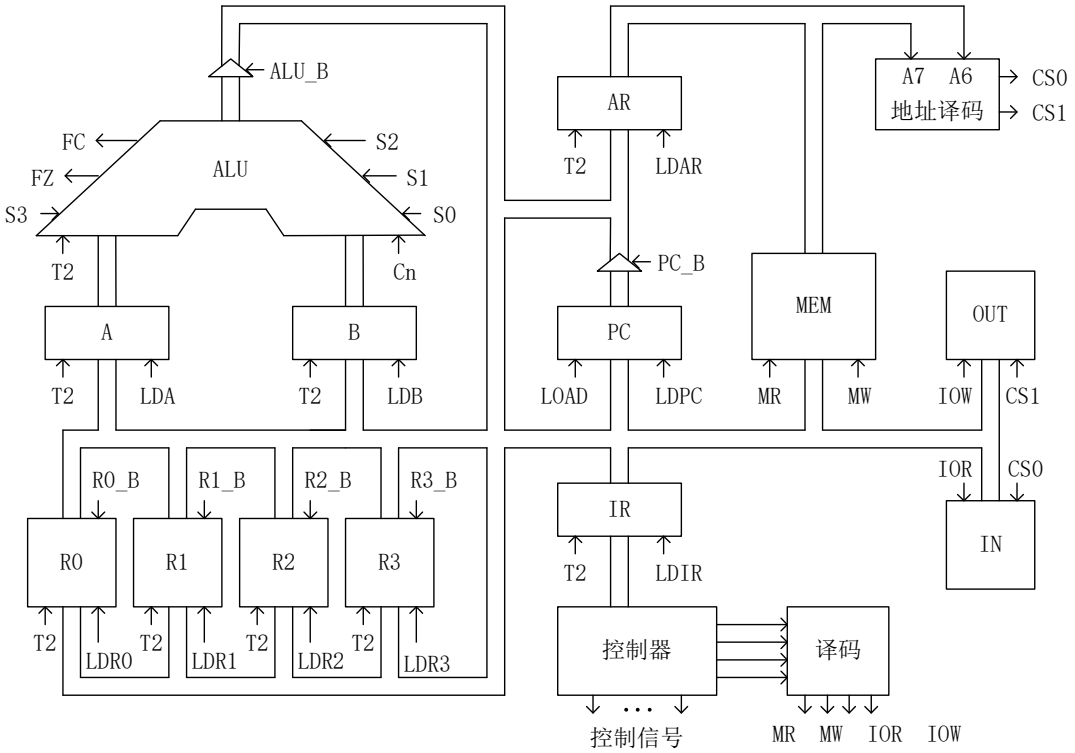


图1-5-2 数据通路框图

和前面的实验相比，复杂模型机实验指令多，寻址方式多，只用一种测试已不能满足设计要求，为此指令译码电路需要重新设计。如图1-5-3 所示在IR 单元的INS_DEC 中实现。

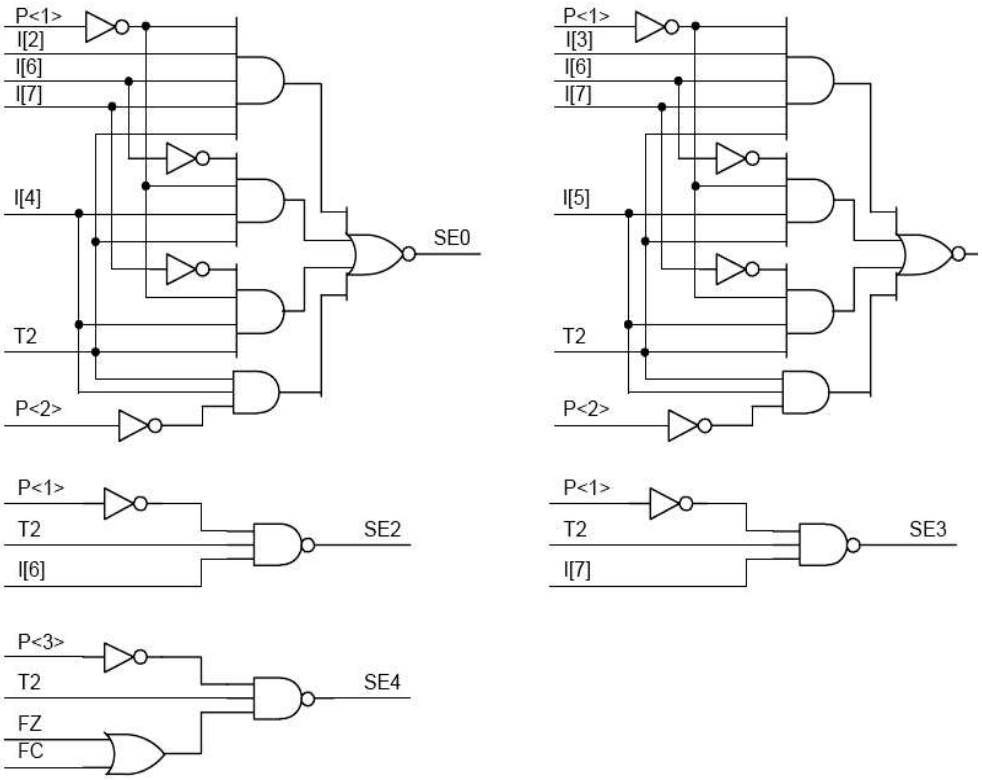


图1-5-3 指令译码原理图

本实验中要用到四个通用寄存器R3…R0，而对寄存器的选择是通过指令的低四位，为此还得设计一个寄存器译码电路，在IR 单元的REG_DEC（GAL16V8）中实现，如图1-5-4 所示。

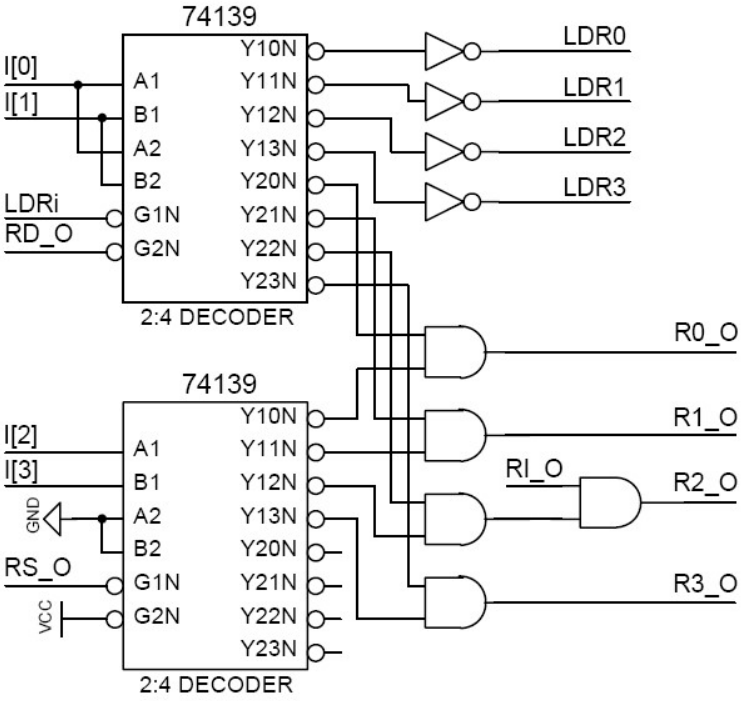


图1-5-4 寄存器译码原理图

根据机器指令系统要求，设计微程序流程图及确定微地址，如图1-5-5 所示。

按照系统建议的微指令格式，见表1-5-4，参照微指令流程图，将每条微指令代码化，译成二进制代码表，见表1-5-5，并将二进制代码表转换为联机操作时的十六进制格式文件。

表1-5-4 微指令格式

23	22	21	20	19	18-15	14-12	11-9	8-6	5-0
M23	M22	WR	RD	IOM	S3-S0	A字段	B字段	C字段	MA5-MA0

A字段

14	13	12	选择
0	0	0	NOP
0	0	1	LDA
0	1	0	LDB
0	1	1	LDR0
1	0	0	保留
1	0	1	LOAD
1	1	0	LDAR
1	1	1	LDIR

B字段

14	13	12	选择
0	0	0	NOP
0	0	1	ALU-B
0	1	0	RS-B
0	1	1	RD-B
1	0	0	RI-B
1	0	1	保留
1	1	0	PC-B
1	1	1	保留

C字段

14	13	12	选择
0	0	0	NOP
0	0	1	P<1>
0	1	0	P<2>
0	1	1	P<3>
1	0	0	保留
1	0	1	LDPC
1	1	0	保留
1	1	1	保留

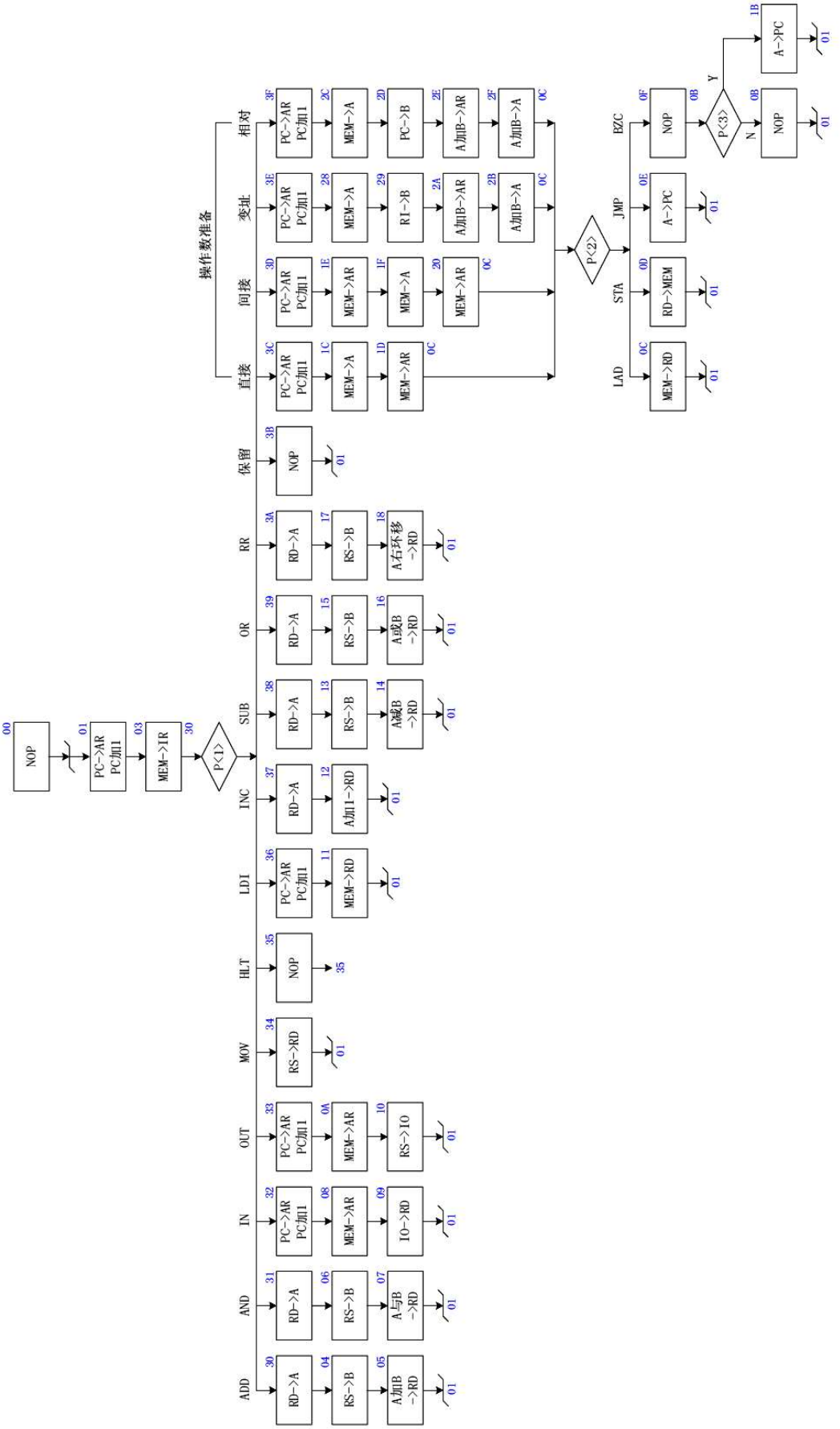

NOP
PC->AR PC加1
MEM->IR
P<1>
操作数准备
ADD
AND
IN
OUT
MOV
HLT
LDI
INC
SUB
OR
RR
保留
直接
间接
变址
相对
RD->A
RS->B
A加B ->RD
A与B ->RD
IO->RD
RS->IO
RS->RD
NOP
MEM->RD
A加1->RD
A减B ->RD
A或B ->RD
A右环移 ->RD
MEM->AR
MEM->A
RI->B
A加B->AR
A加B->A
PC->B
P<2>
LAD
STA
JMP
BZC
MEM->RD
RD->MEM
A->PC
P<3>
Y
N
A->PC

图1-5-5 微程序流程图

表1-5-5 二进制代码表

地址	十六进制表示	高五位	S3-S0	A 字段	B 字段	C 字段	UA5-uA0
00	00 00 01	00000	0000	000	000	000	000001
01	00 6D 43	00000	0000	110	110	101	000011
03	10 70 70	00010	0000	111	000	001	110000
04	00 24 05	00000	0000	010	011	000	000101
05	04 B2 01	00000	1001	011	001	000	000001
06	00 24 07	00000	0000	010	011	000	000111
07	01 32 01	00000	0010	011	001	000	000001
08	10 60 09	00010	0000	110	000	000	001001
09	18 30 01	00011	0000	011	000	000	000001
0A	10 60 10	00010	0000	110	000	000	010000
0B	00 00 01	00000	0000	000	000	000	000001
0C	10 30 01	00010	0000	011	000	000	000001
0D	20 06 01	00100	0000	000	001	100	000001
0E	00 53 41	00000	0000	101	001	101	000001
0F	00 00 CB	00000	0000	000	000	011	001011
10	28 04 01	00101	0000	000	010	000	000001
11	10 30 01	00010	0000	011	000	000	000001
12	06 B2 01	00000	1101	011	001	000	000001
13	00 24 14	00000	0000	010	011	000	010100
14	05 B2 01	00000	1011	011	001	000	000001
15	00 24 16	00000	0000	010	011	000	010110
16	01 B2 01	00000	0011	011	001	000	000001
17	00 24 18	00000	0000	010	011	000	011000
18	04 32 01	00000	1000	011	001	000	000001
1B	00 53 41	00000	0000	101	001	101	000001
1C	10 10 1D	00010	0000	001	000	000	011101
1D	10 60 8C	00010	0000	110	000	010	001100
1E	10 60 1F	00010	0000	110	000	000	011111
1F	10 10 20	00010	0000	110	000	000	100000
20	10 60 8C	00010	0000	110	000	010	001100
28	10 10 29	00010	0000	001	000	000	101001
29	00 28 2A	00000	0000	010	100	000	101010
2A	04 E2 2B	00000	1001	110	001	000	101011
2B	04 92 8C	00000	1001	001	001	010	001100
2C	10 10 2D	00010	0000	001	000	000	101101

根据现有指令，在模型机上实现以下运算：从IN 单元读入一个数据，根据读入数据的低4位值X，求1+2+…+X 的累加和，01H 到0FH 共15 个数据存于60H 到6EH 单元。

根据要求可以得到如下程序，地址和内容均为二进制数。

地 址	内 容	助记符	说 明
00000000	00100000 ；	START: IN R0,00H	从IN 单元读入计数初值

```
00000001 00000000
00000010 01100001 ;   LDI R1,0FH                立即数0FH 送R1
00000011 00001111
00000100 00010100 ;   AND R0,R1                 得到R0 低四位
00000101 01100001 ;   LDI R1,00H                装入和初值00H
00000110 00000000
00000111 11110000 ;   BZC RESULT                计数值为0 则跳转
00001000 00010110
00001001 01100010 ;    LDI R2,60H              读入数据始地址
00001010 01100000
00001011 11001011 ;    LOOP: LAD R3,[RI],00H   从MEM 读入数据送R3，
                                                变址寻址，偏移量为00H
00001100 00000000
00001101 00001101 ;   ADD R1,R3                 累加求和
00001110 01110010 ;   INC RI                    变址寄存加1，指向下一数据
00001111 01100011 ;   LDI R3,01H                装入比较值
00010000 00000001
00010001 10001100 ;   SUB R0,R3
00010010 11110000 ;   BZC RESULT                相减为0，表示求和完毕
00010011 00010110
00010100 11100000 ;   JMP LOOP                  未完则继续
00010101 00001011
00010110 11010001 ;   RESULT: STA 70H,R1        和存于MEM 的70H 单元
00010111 01110000
00011000 00110100 ;   OUT 40H,R1 和在OUT        单元显示
00011001 01000000
00011010 11100000 ;  JMP START                  跳转至START
00011011 00000000
00011100 01010000 ;   HLT                       停机
01100000 00000001 ;   数据
01100001 00000010
01100010 00000011
01100011 00000100
01100100 00000101
01100101 00000110
01100110 00000111
01100111 00001000
01101000 00001001
01101001 00001010
01101010 00001011
01101011 00001100
```

01101100 00001101

01101101 00001110

01101110 00001111

1.6.5 实验步骤

1. 按图1-5-6 连接实验线路，仔细检查接线后打开实验箱电源。

2. 写入实验程序，并进行校验，分两种方式，手动写入和联机写入。

1) 手动写入和校验

(1) 手动写入微程序

① 按图1-3-12 接好线。

② 将MC 单元编程开关置为‘编程’档，时序单元状态开关置为‘单步’档，ADDR 单元状态开关置为‘置数’档。

③ 使用ADDR 单元的低6 位SA5…SA0 给出微地址MA5…MA0，微地址可以通过MC 单元的MA5…MA0 微地址灯显示。

④ CON 单元SD27…SD20、SD17…SD10、SD07…SD00 开关上置24 位微代码，待写入值由MC 单元的M23…M0 二十四位LED 灯显示。

⑤ 启动时序电路（按动一次TS 按钮），即将微代码写入到E PROM 2816 的相应地址对应的单元中。

⑥ 重复③、④、⑤三步，将表1-5-5 的微代码写入2816 芯片中。

(2) 手动校验微程序

① 接线方法和编程一样。

② 将MC 单元编程开关置为‘校验’档，时序单元状态开关置为‘单步’档，ADDR 单元状态开关置为‘置数’档。

③ 使用ADDR 单元的低6 位SA5…SA0 给出微地址MA5…MA0，微地址可以通过MC 单元的MA5…MA0 微地址灯显示。相应地址单元的数据将会被读出，并在MC 单元的M23…M0二十四位LED 灯显示。重复本步，检查2816 芯片中相应地址单元的数据是否和表1-5-5 中的二进制数据相同，如果不同，则说明写入操作失败，应重新写入。

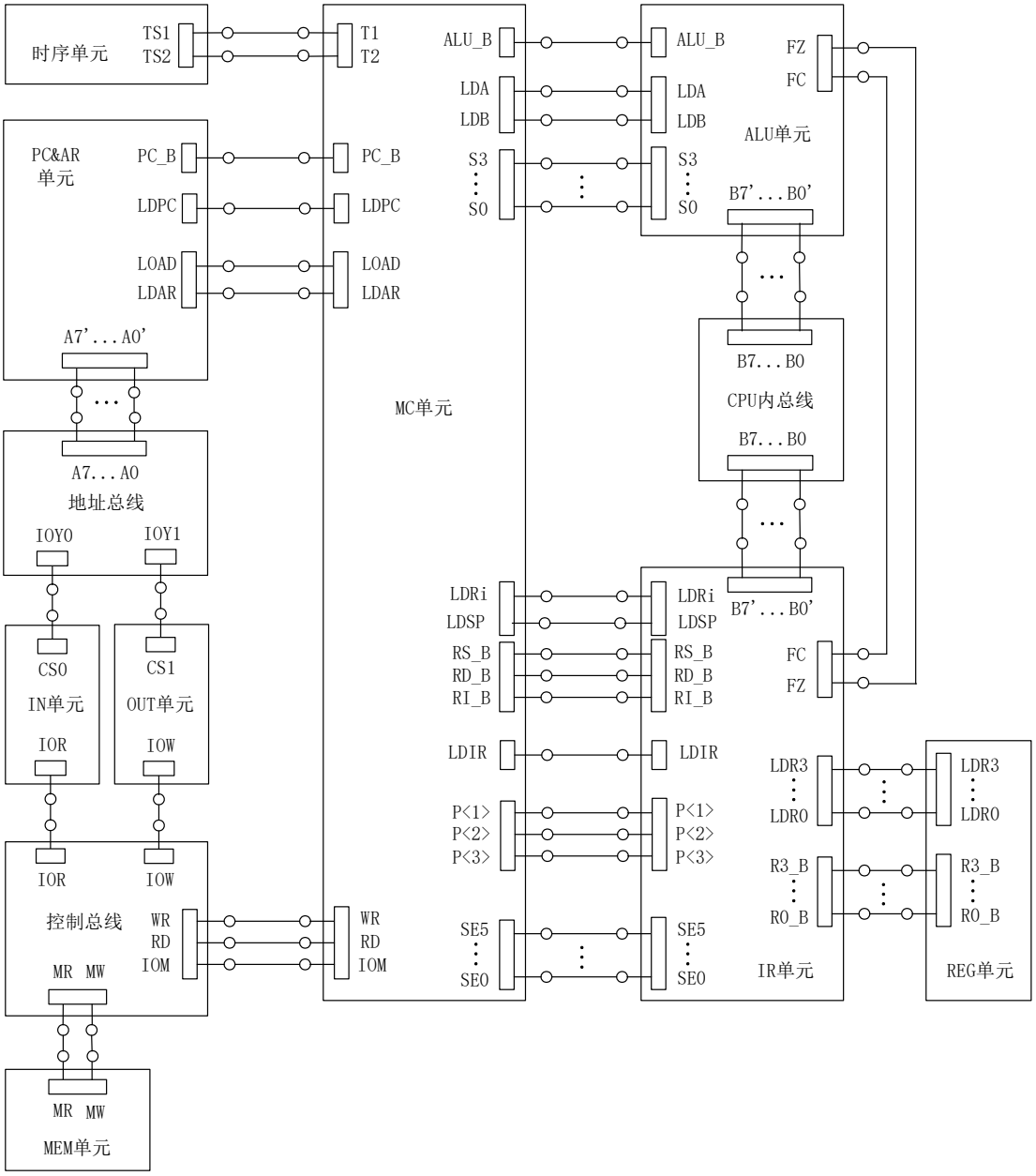


图1-5-6　实验接线图

(3) 手动写入机器程序

① 按图1-4-6 接好线。

② 将MEM 单元编程开关置为‘编程’档，时序单元状态开关置为‘单步’档，ADDR 单元状态开关置为‘置数’档。

③ 在ADDR 单元给出地址SA7…SA0，地址值通过ADDR 单元的8 位LED 灯观测。

④ 在CON 单元SD27…SD20 开关置待写入数据，值通过数据总线的D7…D0 八位LED 灯观测。

⑤ 启动时序电路（按动一次TS 按钮），将数据写入SRAM 6116 地址对应的单元中。

⑥ 重复③、④、⑤三步，将本实验机器程序写入主存。

(4) 手动校验机器程序

① 接线方法和编程一样。

② 将MEM 单元编程开关置为‘校验’档，时序单元状态开关置为‘单步’档，ADDR 单元状态开关置为‘置数’档。

③ 在ADDR 单元给出地址SA7…SA0，地址值通过ADDR 单元的8 位LED 观测。相应地址单元的数据将会被读出，且在数据总线的D7…D0 八位LED 灯显示。

2) 联机写入和校验

联机软件提供了微程序和机器程序下载功能，以代替手动读写微程序和机器程序，但是微程序和机器程序得以指定的格式写入到以TXT 为后缀的文件中，本次实验程序如下，程序中分号‘；’为注释符，分号后面的内容在下载时将被忽略掉。

```
; //****************************************** //
; //                                          //
; // 复杂模型机实验指令文件                   //
; //                                          //
; //****************************************** //
; //****** Start Of Main Memory Data ****** //
    $P 00 20    ; START: IN R0,00H          从IN 单元读入计数初值
    $P 01 00
    $P 02 61    ; LDI R1,0FH                立即数0FH 送R1
    $P 03 0F
    $P 04 14    ; AND R0,R1                 得到R0 低四位
    $P 05 61    ; LDI R1,00H                装入和初值00H
    $P 06 00
    $P 07 F0    ; BZC RESULT                计数值为0 则跳转
    $P 08 16
    $P 09 62    ; LDI R2,60H                读入数据始地址
    $P 0A 60
    $P 0B CB    ; LOOP: LAD R3,[RI],00H     从MEM 读入数据送R3，
                                            变址寻址，偏移量为00H
    $P 0C 00
    $P 0D 0D    ; ADD R1,R3                 累加求和
    $P 0E 72    ; INC RI                    变址寄存加1，指向下一数据
    $P 0F 63    ; LDI R3,01H                装入比较值
    $P 10 01
    $P 11 8C    ; SUB R0,R3
    $P 12 F0    ; BZC RESULT                相减为0，表示求和完毕
    $P 13 16
    $P 14 E0    ; JMP LOOP                  未完则继续
    $P 15 0B
    $P 16 D1    ; RESULT: STA 70H,R1        和存于MEM 的70H 单元
    $P 17 70
```

```
    $P 18 34    ; OUT 40H,R1                  和在OUT 单元显示
    $P 19 40
    $P 1A E0    ; JMP START                   跳转至START
    $P 1B 00
    $P 1C 50    ; HLT                         停机
    $P 60 01    ; 数据
    $P 61 02
    $P 62 03
    $P 63 04
    $P 64 05
    $P 65 06
    $P 66 07
    $P 67 08
    $P 68 09
    $P 69 0A
    $P 6A 0B
    $P 6B 0C
    $P 6C 0D
    $P 6D 0E
    $P 6E 0F
; //***** End Of Main Memory Data *****//
; //** Start Of MicroController Data **//
    $M 00 000001    ; NOP
    $M 01 006D43    ; PC->AR, PC 加1
    $M 03 107070    ; MEM->IR, P<1>
    $M 04 002405    ; RS->B
    $M 05 04B201    ; A 加B->RD
    $M 06 002407    ; RS->B
    $M 07 013201    ; A 与B->RD
    $M 08 106009    ; MEM->AR
    $M 09 183001    ; IO->RD
    $M 0A 106010    ; MEM->AR
    $M 0B 000001    ; NOP
    $M 0C 103001    ; MEM->RD
    $M 0D 200601    ; RD->MEM
    $M 0E 005341    ; A->PC
    $M 0F 0000CB    ; NOP, P<3>
    $M 10 280401    ; RS->IO
    $M 11 103001    ; MEM->RD
    $M 12 06B201    ; A 加1->RD
    $M 13 002414    ;  RS->B
```

```
    $M 14 05B201    ;  A 减B->RD
    $M 15 002416    ;  RS->B
    $M 16 01B201    ;  A 或B->RD
    $M 17 002418    ;  RS->B
    $M 18 043201    ; A 右环移->RD
    $M 1B 005341    ; A->PC
    $M 1C 10101D    ; MEM->A
    $M 1D 10608C    ; MEM->AR, P<2>
    $M 1E 10601F    ; MEM->AR
    $M 1F 101020    ; MEM->A
    $M 20 10608C    ; MEM->AR, P<2>
    $M 28 101029    ; MEM->A
    $M 29 00282A    ; RI->B
    $M 2A 04E22B    ; A 加B->AR
    $M 2B 04928C    ; A 加B->A, P<2>
    $M 2C 10102D    ; MEM->A
    $M 2D 002C2E    ; PC->B
    $M 2E 04E22F    ; A 加B->AR
    $M 2F 04928C    ; A 加B->A, P<2>
    $M 30 001604    ; RD->A
    $M 31 001606    ; RD->A
    $M 32 006D48    ; PC->AR, PC 加1
    $M 33 006D4A    ; PC->AR, PC 加1
    $M 34 003401    ; RS->RD
    $M 35 000035    ; NOP
    $M 36 006D51    ; PC->AR, PC 加1
    $M 37 001612    ; RD->A
    $M 38 001613    ; RD->A
    $M 39 001615    ; RD->A
    $M 3A 001617    ; RD->A
    $M 3B 000001    ; NOP
    $M 3C 006D5C    ; PC->AR, PC 加1
    $M 3D 006D5E    ; PC->AR, PC 加1
    $M 3E 006D68    ; PC->AR, PC 加1
    $M 3F 006D6C    ; PC->AR, PC 加1
; //** End Of MicroController Data **//
```

选择联机软件的“【转储】—【装载】”功能，在打开文件对话框中选择上面所保存的文件，软件自动将机器程序和微程序写入指定单元。

选择联机软件的“【转储】—【刷新指令区】”可以读出下位机所有的机器指令和微指令，并在指令区显示，对照文件检查微程序和机器程序是否正确，如果不正确，则说明写入

操作失败，应重新写入，可以通过联机软件单独修改某个单元的指令，以修改微指令为例，先用鼠标左键单击指令区的‘微存’TAB 按钮，然后再单击需修改单元的数据，此时该单元变为编辑框输入6 位数据并回车，编辑框消失，并以红色显示写入的数据。

3. 运行程序

方法一：本机运行

将MC 单元的编程开关置为‘运行’档，MEM 单元的编程开关也置为‘运行’档，按动CON 单元的总清按钮CLR，将使程序计数器PC、地址寄存器AR 和微程序地址为00H，程序可以从头开始运行，暂存器A、B，指令寄存器IR 和OUT 单元也会被清零。

将时序单元的状态开关置为‘单步’档，每按动一次TS 按钮，即可单步运行一条微指令，对照微程序流程图，观察微地址显示灯是否和流程一致。每运行完一条微指令，观测一次数据总线和地址总线，对照数据通路图，分析总线上的数据是否正确。

当模型机执行完OUT 指令后，检查OUT 单元显示的数是否正确，按下CON 单元的总清按钮CLR，改变IN 单元的值，再次执行机器程序，从OUT 单元显示的数判别程序执行是否正确。

方法二：联机运行（软件使用说明请看附录1）

进入软件界面，选择菜单命令“【实验】—【综合性实验】”，打开综合性实验数据通路图，选择相应的功能命令，即可联机运行、监控、调试程序。按动CON 单元的总清按钮CLR，然后通过软件运行程序，当模型机执行完OUT 指令后，检查OUT 单元显示的数是否正确。在数据通路图和微程序流中观测指令的执行过程，并观测软件中地址总线、数据总线以及微指令显示和下位机是否一致。

思考题：

1. 利用上述模型机能否通过编写程序实现两个8位二进制数的乘法运算，为什么？

实验报告要求：

1. 实验记录：所有的运算结果，故障现象及排出经过；
2. 试解答思考题的问题；
3. 通过本次实验的收获及想法。

1.6 带中断处理能力的模型机设计实验

1.6.1 实验目的

(1) 掌握中断原理及其响应流程。

(2) 掌握8259 中断控制器原理及其应用编程。

1.6.2 实验设备

PC 机一台，TD-CM3+实验系统一套。

1.6.3 实验原理

8259 的引脚分配图如图1-6-1 所示。

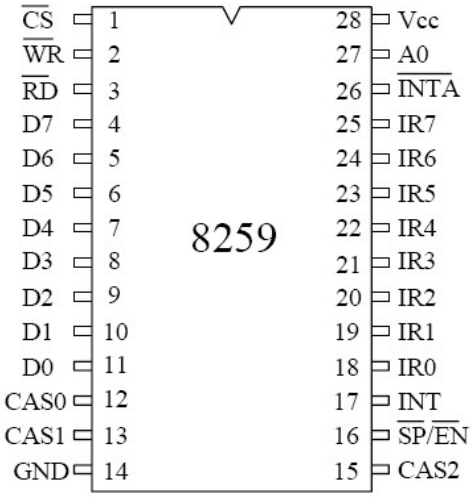


图1-6-1 8259 芯片引脚说明

8259 芯片引脚说明如下。

- D7～D0 为双向三态数据线
- $\overline{CS}$ 片选信号线
- A0 用来选择芯片内部不同的寄存器，通常接至地址总线的 A0。
- $\overline{RD}$ 读信号线，低电平有效，其有效时控制信息从 8259 读至 CPU。
- $\overline{WR}$ 写信号线，低电平有效，其有效时控制信息从 CPU 写入至 8259。
- $\overline{SP}/\overline{EN}$ 从程序/允许缓冲
- $\overline{INTA}$ 中断响应输入
- INT 中断输出
- IR0～IR7 8 条外界中断请求输入线。
- CAS2～CAS0 级连信号线。

$\overline{CS}$、A0、$\overline{RD}$ 、$\overline{WR}$ 、D4、D3 位的电平与 8259 操作关系如表 1-6-1 所示。

CPU 必须有一个中断使能寄存器，并且可以通过指令对该寄存器进行操作，其原理如图1-6-2 所示。CPU 开中断指令STI 对其置1，而CPU 关中断指令CLI 对其置0。

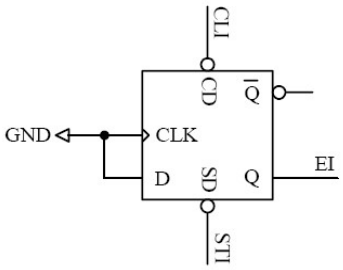


图1-6-2 中断使能寄存器原理图

表1-6-1 8259A 的读/写操作

A0	D4	D3	$\overline{RD}$	$\overline{WR}$	$\overline{CS}$	操作
						输入操作(读)
0			0	1	0	IRR,ISR或中断级别→数据总线
1			0	1	0	IMR→数据总线
						输出操作(写)
0	0	0	1	0	0	数据总线→OCW2
0	0	1	1	0	0	数据总线→OCW3
0	1	X	1	0	0	数据总线→OCW1
1	X	X	1	0	0	数据总线→ICW1, ICW2, ICW3, ICW4
						断开功能
	X	X	1	1	0	数据总线→三态(无操作)
	X	X	X	X	1	数据总线→三态(无操作)

8259 的数据线D7…D0 挂接到数据总线，地址线A0 挂接到地址总线的A0 上，片选信号 $\overline{CS}$（实验箱上丝印为CS3）接控制总线的IOY3，IOY3 由地址总线的高2 位译码产生，其地址方配见表1-6-2，$\overline{RD}$ 、$\overline{WR}$（实验箱上丝印为IOR 和IOW）接CPU 给出的读写信号，8259 和系统的连接如图1-6-3 所示。

表1-6-2 I/O 地址空间分配

A7 A6	选定	地址空间
00	IOY0	00-3F
01	IOY1	40-7F
10	IOY2	80-BF
11	IOY3	C0-FF

本实验要求设计的模型计算机具备有类X86 的中断功能，当外部中断请求有效、CPU 允许中断，且在一条指令执行完时，CPU 将响应中断。当CPU 响应中断时，将会向8259 发送两个连续的 $\overline{INTA}$ 信号，请注意，8259 是在接收到第一个 $\overline{INTA}$ 信号后才会锁住向 CPU 的中断请求信号 INTR（高电平有效），并且在第二个 $\overline{INTA}$ 信号到达后将其变为低电平（自动 EOI 方式），所以，中断请求信号 IR0 应该维持一段时间，直到 CPU 发送出第一个 $\overline{INTA}$ 信号，这才是一个有效的中断请求。8259 在收到第二个 $\overline{INTA}$ 信号后，就会将中断向量号发送到数据总线，CPU 读取中断向量号，并转入相应的中断处理程序中。

本系统的指令译码电路是在IR 单元的INS_DEC（GAL20V8）中实现，如图1-6-4 所示。和前面复杂模型机实验指令译码电路相比，主要增加了对中断的支持，当INTR（有中断请求）

和EI（CPU 允许中断）均为1，且P<4>测试有效，那么在T2 节拍时，微程式序就会产生中断响应分支，从而使得CPU 能响应中断。

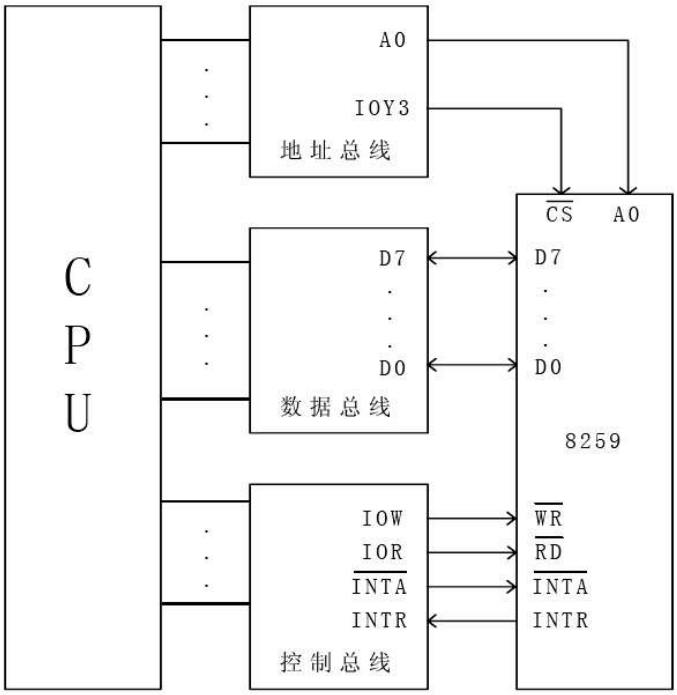


图1-6-3 8259 和CPU 挂接图

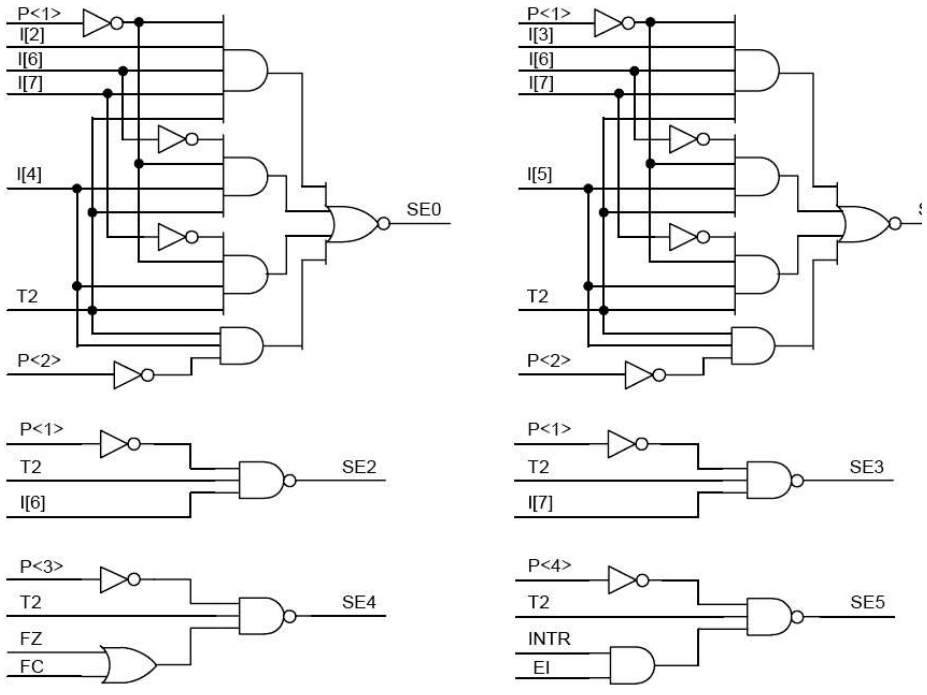


图1-6-4　指令译码电路

在中断过程中需要有现场保护，而且在编程的过程中也需要一些压栈或弹栈操作，所以还需设置一个堆栈，由R3 做堆栈指针。系统的寄存器译码电路如图1-6-5 所示，在IR 单元的REG_DEC（GAL16V8）中实现，和前面复杂模型机实验寄存器译码电路相比，增加一个或门

和一个与门，用以支持堆栈操作。

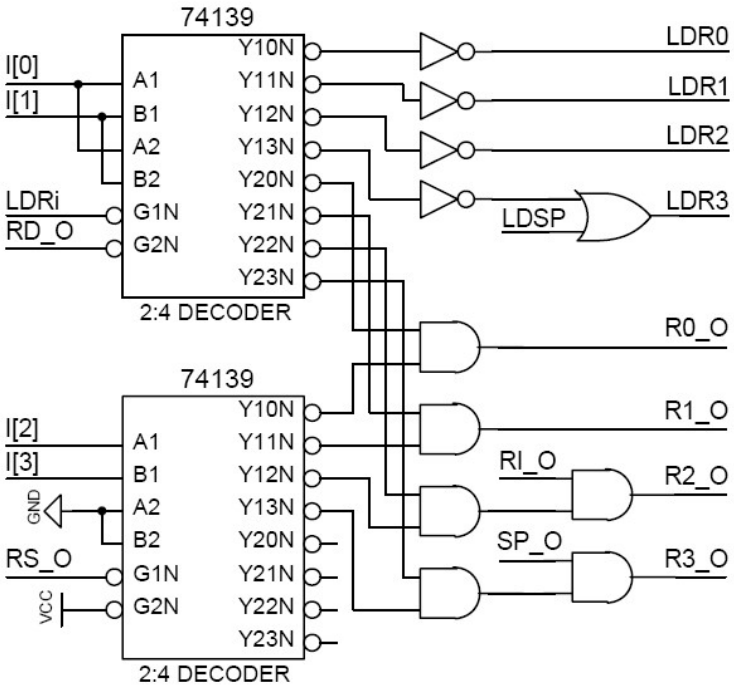


图1-6-5 寄存器译码原理图

本模型机共设计16 条指令,表1-6-3 列出了基本指令的格式、助记符及其功能。

表1-6-3 指令描述

助记符号	指令格式				指令功能
MOV RD,RS	0100	RS	RD		RS → RD
ADD RD,RS	0000	RS	RD		RD + RS → RD
AND RD,RS	0001	RS	RD		RD∧RS → RD
STI	0111	**	**		CPU开中断
CLI	1000	**	**		CPU关中断
PUSH RS	1001	RS	**		RS → 堆栈
POP RD	1010	**	RD		堆栈 → RD
IRET	1011	**	**		中断返回
LAD M D,RD	1100	M	RD	D	E → RD
STA M D,RS	1101	M	RD	D	RD → E
JMP M D	1110	M	**	D	E → PC
BZC M D	1111	M	**	D	当FC或FZ=1时, E → PC
IN RD,P	0010	**	RD	P	[P] → RD
OUT P,RS	0011	RS	**	P	RS → [P]
LDI RD,D	0110	**	RD	D	D → RD
HALT	0101	**	**		停机

其中，D 为立即数，P 为外设的端口地址，RS 为源寄存器，RD 为目的寄存器，并规定如下。

RS 或 RD	选定的寄存器
00	R0
01	R1
10	R2
11	R3

设定微指令格式，如表1-6-4 所示。

根据指令系统要求，设计微程序流程及确定微地址，并得到微程序流程图，如图1-6-6 所示。

参照微程序流程图，将每条微指令代码化，译成二进制微代码表，如表1-6-5 所示。

表1-6-4 微指令格式

23	22	21	20	19	18-15	14-12	11-9	8-6	5-0
M23	$\overline{\text{INTA}}$	WR	RD	IOM	S3-S0	A字段	B字段	C字段	MA5-MA0

A字段

14	13	12	选择
0	0	0	NOP
0	0	1	LDA
0	1	0	LDB
0	1	1	LDR0
1	0	0	保留
1	0	1	LOAD
1	1	0	LDAR
1	1	1	LDIR

B字段

11	10	9	选择
0	0	0	NOP
0	0	1	ALU-B
0	1	0	RS-B
0	1	1	RD-B
1	0	0	RI-B
1	0	1	SP-B
1	1	0	PC-B
1	1	1	保留

C字段

8	7	6	选择
0	0	0	NOP
0	0	1	P<1>
0	1	0	P<2>
0	1	1	P<3>
1	0	0	P<4>
1	0	1	LDPC
1	1	0	STI
1	1	1	CLI

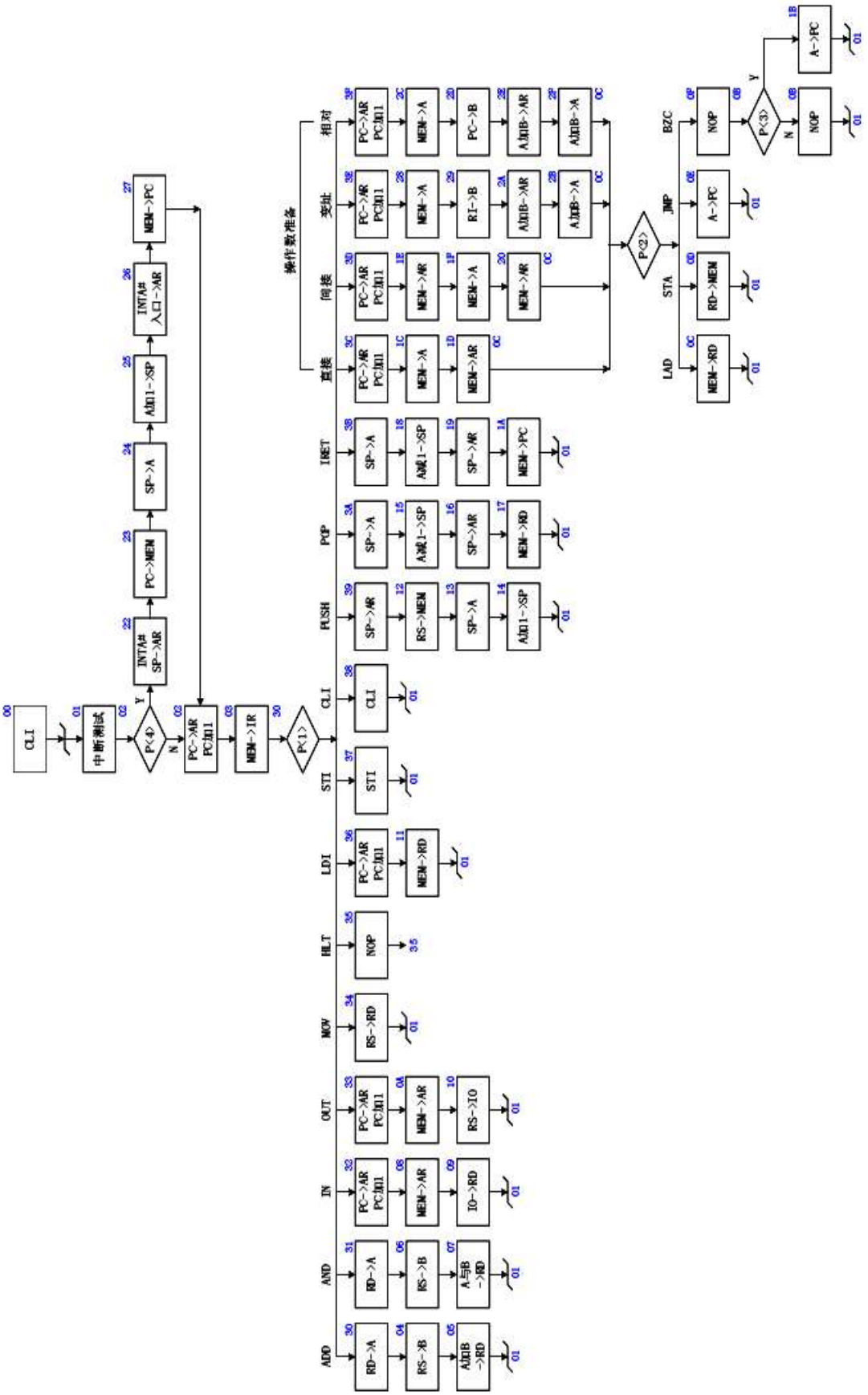

CLI
中断测试
P<4>
INTA#
SP->AR
PC->MEM
SP->A
ADD1->SP
INTA#
入口->AR
MEM->PC
PC->AR
PC加1
MEM->IR
P<1>
操作数准备
直接
间接
变址
相对
CLI
STI
LDI
HLT
NOP
MOV
RS->RD
OUT
PC->AR
PC加1
MEM->AR
RS->IO
IN
IO->RD
AND
RD->A
RS->B
A与B
->RD
ADD
ADDB
->RD
FLUSH
RS->MEM
POP
ADD1->SP
SP->AR
MEM->RD
IRET
MEM->PC
LAD
STA
RD->MEM
JMP
A->PC
BZC
P<3>
P<2>
MEM->A
MEM->AR
PC->B
RI->B
ADDB->AR
ADDB->A

图1-6-6 微程序流程图

地址	十六进制表示	高五位	S3-S0	A 字段	B 字段	C 字段	UA5-uA0
00	00 01 C1	00000	0000	000	000	111	000001
01	00 01 02	00000	0000	000	000	100	000010
02	00 6D 43	00000	0000	110	110	101	000011
03	10 70 70	00010	0000	111	000	001	110000
04	00 24 05	00000	0000	010	011	000	000101
05	04 B2 01	00000	1001	011	001	000	000001
06	00 24 07	00000	0000	010	011	000	000111
07	01 32 01	00000	0010	011	001	000	000001
08	10 60 09	00010	0000	110	000	000	001001
09	18 30 01	00011	0000	011	000	000	000001
0A	10 60 10	00010	0000	110	000	000	010000
0B	00 00 01	00000	0000	000	000	000	000001
0C	10 30 01	00010	0000	011	000	000	000001
0D	20 06 01	00100	0000	000	001	100	000001
0E	00 53 41	00000	0000	101	001	101	000001
0F	00 00 CB	00000	0000	000	000	011	001011
10	28 04 01	00101	0000	000	010	000	000001
11	10 30 01	00010	0000	011	000	000	000001
12	20 04 13	00100	0000	000	010	000	010011
13	00 1A 14	00000	0000	001	101	000	010100
14	06 C2 01	00000	1101	100	001	000	000001
15	06 42 16	00000	1100	100	001	000	010110
16	00 6A 17	00000	0000	110	101	000	010111
17	10 30 01	00010	0000	011	000	000	000001
18	06 42 19	00000	1100	100	001	000	011001
19	00 6A 1A	00000	0000	110	101	000	011010
1A	10 51 41	00010	0000	101	000	101	000001
1B	00 53 41	00000	0000	101	001	101	000001
1C	10 10 1D	00010	0000	001	000	000	011101
1D	10 60 8C	00010	0000	110	000	010	001100
1E	10 60 1F	00010	0000	110	000	000	011111
1F	10 10 20	00010	0000	001	000	000	100000
20	10 60 8C	00010	0000	110	000	010	001100
22	40 6A 23	01000	0000	110	101	000	100011

23	20 0C 24	01000	0000	000	110	000	100100
24	00 1A 25	00000	0000	001	101	000	100101
25	06 C2 26	00000	1101	100	001	000	100110
26	40 60 27	01000	0000	110	000	000	100111
27	10 51 42	00010	0000	101	000	101	000010
28	10 10 29	00010	0000	001	000	000	101001
29	00 28 2A	00000	0000	010	100	000	101010
2A	04 E2 2B	00000	1001	110	001	000	101011
2B	04 92 8C	00000	1001	001	001	010	001100
2C	10 10 2D	00010	0000	001	000	000	101101
2D	00 2C 2E	00000	0000	010	110	000	101110
2E	04 E2 2F	00000	1001	110	001	000	101111
2F	04 92 8C	00000	1001	001	001	010	001100
30	00 16 04	00000	0000	001	010	000	000100
31	00 16 06	00000	0000	001	010	000	000110
32	00 6D 48	00000	0000	110	110	101	001000
33	00 6D 4A	00000	0000	110	110	101	001010
34	00 34 01	00000	0000	011	010	000	000001
35	00 00 35	00000	0000	000	000	000	110101
36	00 6D 51	00000	0000	110	110	101	010001
37	00 01 81	00000	0000	000	000	110	000001
38	00 01 C1	00000	0000	000	000	111	000001
39	00 6A 12	00000	0000	110	101	000	010010
3A	00 1A 15	00000	0000	001	101	000	010101
3B	00 1A 18	00000	0000	001	101	000	011000
3C	00 6D 5C	00000	0000	110	110	101	011100
3D	00 6D 5E	00000	0000	110	110	101	011110
3E	00 6D 68	00000	0000	110	110	101	101000
3F	00 6D 6C	00000	0000	110	110	101	101100

根据现有指令，编写一段程序，在模型机上实现以下功能：从IN 单元读入一个数据X 存于寄存器R0，CPU 每响应一次中断，对R0 中的数据加1，并输出到OUT 单元。

根据要求可以得到如下程序，地址和内容均为二进制数。

```
地 址     内 容              助记符                    说 明
00000000 01100000      ; LDI R0,13H              将立即数13 装入R0
00000001 00010011
00000010 00110000      ; OUT C0H,R0              将R0 中的内容写入端口C0 中，即写
00000011 11000000      ;                         ICW1，边沿触发，单片模式，需ICW4
00000100 01100000      ; LDI R0,30H              将立即数30 装入R0
```

```
00000101 00110000
00000110 00110000       ; OUT C1H,R0              将R0 中的内容写入端口C1 中，即写
00000111 11000001       ; ICW2,                   中断向量为30-37
00001000 01100000       ; LDI R0,03H              将立即数03 装入R0
00001001 00000011
00001010 00110000       ; OUT C1H,R0              将R0 中的内容写入端口C1 中，即写
00001011 11000001       ;                         ICW4，非缓冲，86 模式，自动EOI
00001100 01100000       ; LDI R0,FEH              将立即数FE 装入R0
00001101 00000000
00001110 00110000       ; OUT C1H,R0              将R0 中的内容写入端口C1 中，即写
00001111 11000001       ; OCW1,                   只允许IR0 请求
00010000 01100011       ; LDI SP,A0H              初始化堆栈指针为A0
00010001 10100000
00010010 01110000       ; STI CPU                 开中断
00010011 00100000       ; IN R0,00H               从端口00（IN 单元）读入计数初值
00010100 00000000
00010101 01000001       ; LOOP：MOV R1,R0         移动数据，并等待中断
00010110 11100000       ; JMP LOOP                跳转，并等待中断
00010111 00010101
                  ; 以下为中断服务程序：
00100000 0000000080     ; CLI CPU                 关中断
00100001 0000000061     ; LDI R1,01H              将立即数01 装入R1
00100010 0000000001
00100011 0000000004     ; ADD R0,R1               将R0 和R1 相加，即计数值加1
00100100 0000000030     ; OUT 40H,R0              将计数值输出到端口40（OUT 单元）
00100101 0000000040
00100110 0000000070     ; STI CPU                 开中断
00100111 00000000B0     ; IRET                    中断返回
00110000 0000000020     ;                         IR0 的中断入口地址20
```

1.8.4 实验步骤

1. 按图1-6-6 所示连接实验接线，仔细检查接线后打开实验箱电源。

2. 写入实验程序，并进行校验，分两种方式，手动写入和联机写入。

1) 手动写入和校验

(1) 手动写入微程序

① 按图1-3-12 接好线。

② 将MC 单元编程开关置为‘编程’档，时序单元状态开关置为‘单步’档，ADDR 单元状态开关置为‘置数’档。

③ 使用ADDR 单元的低6 位SA5…SA0 给出微地址MA5…MA0，微地址可以通过MC 单元的MA5…MA0 微地址灯显示。

④ CON 单元SD27…SD20、SD17…SD10、SD07…SD00 开关上置24 位微代码，待写入值由MC 单元的M23…M0 二十四位LED 灯显示。

⑤ 启动时序电路（按动一次TS 按钮），即将微代码写入到E PROM 2816 的相应地址对应的单元中。

⑥ 重复③、④、⑤三步，将表1-6-5 的微代码写入2816 芯片中。

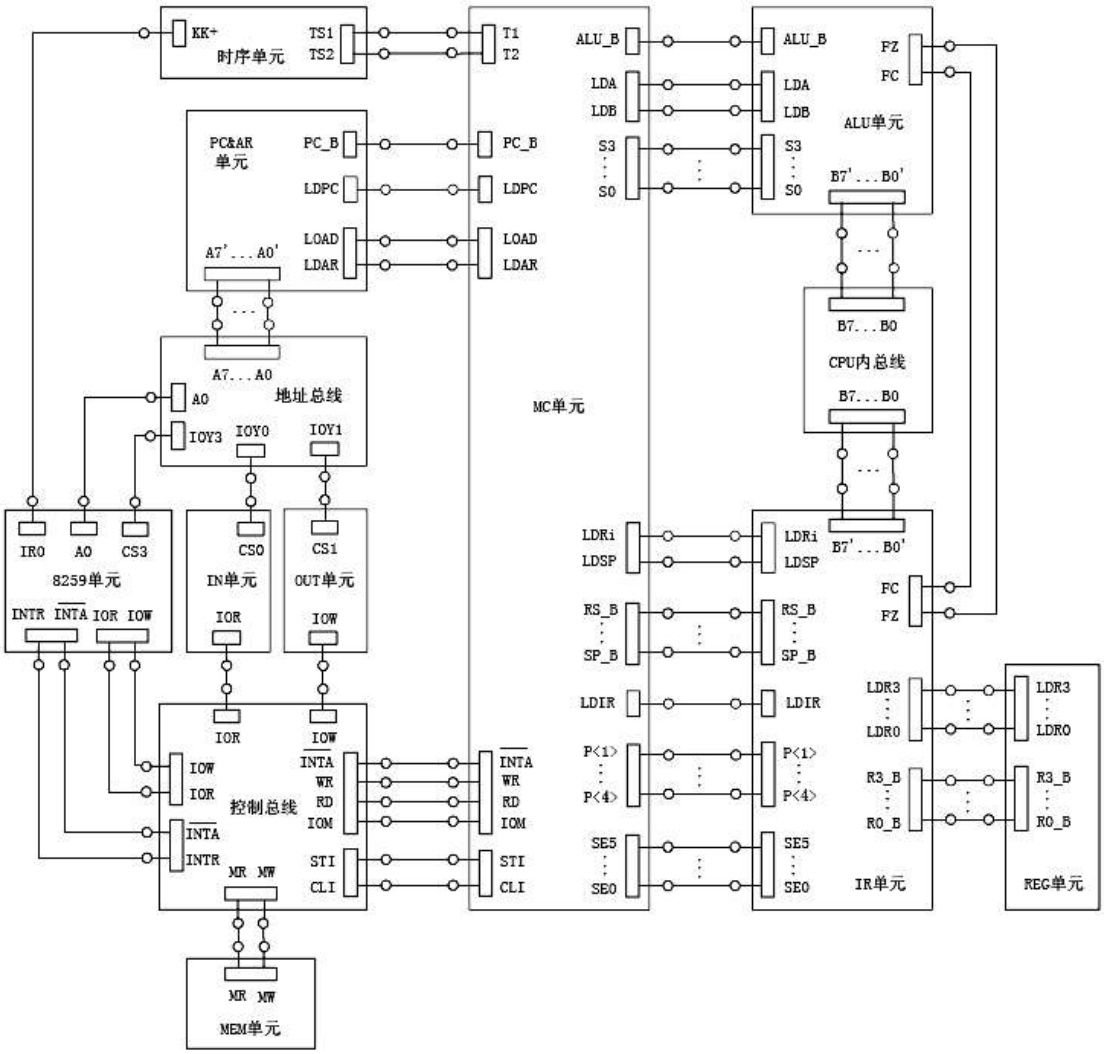


图1-6-6 实验接线图

(2) 手动校验微程序

① 接线方法和编程一样。

② 将MC 单元编程开关置为‘校验’档，时序单元状态开关置为‘单步’档，ADDR 单元状态开关置为‘置数’档。

③ 使用ADDR 单元的低6 位SA5…SA0 给出微地址MA5…MA0，微地址可以通过MC 单元的MA5…MA0 微地址灯显示。相应地址单元的数据将会被读出，并在MC 单元的M23…M0二十四位LED 灯显示。重复本步，检查2816 芯片中相应地址单元的数据是否和表1-6-5 中的二进制数据相同，如果不同，则说明写入操作失败，应重新写入。

(3) 手动写入机器程序

① 按图1-4-6 接好线。

② 将MEM 单元编程开关置为‘编程’档，时序单元状态开关置为‘单步’档，ADDR 单元状态开关置为‘置数’档。

③ 在ADDR 单元给出地址SA7…SA0，地址值通过ADDR 单元的8 位LED 灯观测。

④ 在CON 单元SD27…SD20 开关置待写入数据，值通过数据总线的D7…D0 八位LED 灯观测。

⑤ 启动时序电路（按动一次TS 按钮），将数据写入SRAM 6116 地址对应的单元中。

⑥ 重复③、④、⑤三步，将本实验机器程序写入主存。

(4) 手动校验机器程序

① 接线方法和编程一样。

② 将MEM 单元编程开关置为‘校验’档，时序单元状态开关置为‘单步’档，ADDR 单元状态开关置为‘置数’档。

③ 在ADDR 单元给出地址SA7…SA0，地址值通过ADDR 单元的8 位LED 观测。相应地址单元的数据将会被读出，且在数据总线的D7…D0 八位LED 灯显示。

2) 联机写入和校验

联机软件提供了微程序和机器程序下载功能，以代替手动读写微程序和机器程序，但是微程序和机器程序得以指定的格式写入到以TXT 为后缀的文件中，本次实验程序如下，程序中分号‘；’为注释符，分号后面的内容在下载时将被忽略掉：

注意：$P之后以及“；”之前的空格须为4个，微程序同

```
; //*****************************************//
; //                                         //
; // 带中断处理能力的模型机实验指令文件      //
; //                                         //
; //                                         //
; //*****************************************//
; //***** Start Of Main Memory Data *****//
    $P 00 60    ; LDI R0,13H        将立即数13 装入R0
    $P 01 13
    $P 02 30    ; OUT C0H,R0        将R0 中的内容写入端口C0 中，即写
    $P 03 C0    ;                   ICW1，边沿触发，单片模式，需要ICW4
    $P 04 60    ; LDI R0,30H        将立即数30 装入R0
    $P 05 30
    $P 06 30    ; OUT C1H,R0        将R0 中的内容写入端口C1 中，即写
    $P 07 C1    ;                   ICW2，中断向量为30-37
    $P 08 60    ; LDI R0,03H        将立即数03 装入R0
    $P 09 03
    $P 0A 30    ; OUT C1H,R0        将R0 中的内容写入端口C1 中，即写
    $P 0B C1    ; ICW4,             非缓冲，86 模式，自动EOI
    $P 0C 60    ; LDI R0,FEH        将立即数FE 装入R0
    $P 0D FE
    $P 0E 30    ; OUT C1H,R0        将R0 中的内容写入端口C1 中，即写
    $P 0F C1    ;                   OCW1，只允许IR0 请求
    $P 10 63    ; LDI SP,A0H        初始化堆栈指针为A0
    $P 11 A0
    $P 12 70    ; STI CPU           开中断
    $P 13 20    ; IN R0,00H         从端口00（IN 单元）读入计数初值
    $P 14 00
    $P 15 41    ; LOOP：MOV R1,R0   移动数据，并等待中断
```

```
    $P 16 E0    ; JMP LOOP          跳转，并等待中断
    $P 17 15
        ;               以下为中断服务程序：
    $P 20 80    ; CLI CPU           关中断
    $P 21 61    ; LDI R1,01H        将立即数01 装入R1
    $P 22 01
    $P 23 04    ; ADD R0,R1         将R0 和R1 相加，即计数值加1
    $P 24 30    ; OUT 40H,R0        将计数值输出到端口40（OUT 单元）
    $P 25 40
    $P 26 70    ; STI CPU           开中断
    $P 27 B0    ; IRET              中断返回
    $P 30 20    ;                   IR0 的中断入口地址20
; //***** End Of Main Memory Data *****//
; //** Start Of MicroController Data **//
    $M 00 0001C1    ; NOP
    $M 01 000102    ; 中断测试, P<4>
    $M 02 006D43    ; PC->AR, PC 加1
    $M 03 107070    ; MEM->IR, P<1>
    $M 04 002405    ; RS->B
    $M 05 04B201    ; A 加B->RD
    $M 06 002407    ; RS->B
    $M 07 013201    ; A 与B->RD
    $M 08 106009    ; MEM->AR
    $M 09 183001    ; IO->RD
    $M 0A 106010    ; MEM->AR
    $M 0B 000001    ; NOP
    $M 0C 103001    ; MEM->RD
    $M 0D 200601    ; RD->MEM
    $M 0E 005341    ; A->PC
    $M 0F 0000CB    ; NOP, P<3>
    $M 10 280401    ; RS->IO
    $M 11 103001    ; MEM->RD
    $M 12 200413    ; RS->MEM
    $M 13 001A14    ; SP->A
    $M 14 06C201    ; A 加1->SP
    $M 15 064216    ; A 减1->SP
    $M 16 006A17    ; SP->AR
    $M 17 103001    ; MEM->RD
    $M 18 064219    ; A 减1->SP
    $M 19 006A1A    ; SP->AR
    $M 1A 105141    ; MEM->PC
    $M 1B 005341    ; A->PC
```

```
$M 1C 10101D    ; MEM->A
$M 1D 10608C    ; MEM->AR, P<2>
$M 1E 10601F    ; MEM->AR
$M 1F 101020    ; MEM->A
$M 20 10608C    ; MEM->AR, P<2>
$M 22 406A23    ; INTA#, SP->AR
$M 23 200C24    ; PC->MEM
$M 24 001A25    ; SP->A
$M 25 06C226    ; A 加1->SP
$M 26 406027    ; INTA#, 入口->AR
$M 27 105142    ; MEM->PC
$M 28 101029    ; MEM->A
$M 29 00282A    ; RI->B
$M 2A 04E22B    ; A 加B->AR
$M 2B 04928C    ; A 加B->A, P<2>
$M 2C 10102D    ; MEM->A
$M 2D 002C2E    ; PC->B
$M 2E 04E22F    ; A 加B->AR
$M 2F 04928C    ; A 加B->A, P<2>
$M 30 001604    ; RD->A
$M 31 001606    ; RD->A
$M 32 006D48    ; PC->AR, PC 加1
$M 33 006D4A    ; PC->AR, PC 加1
$M 34 003401    ; RS->RD
$M 35 000035    ; NOP
$M 36 006D51    ; PC->AR, PC 加1
$M 37 000181    ; STI
$M 38 0001C1    ; CLI
$M 39 006A12    ; SP->AR
$M 3A 001A15    ; SP->A
$M 3B 001A18    ; SP->A
$M 3C 006D5C    ; PC->AR, PC 加1
$M 3D 006D5E    ; PC->AR, PC 加1
$M 3E 006D68    ; PC->AR, PC 加1
$M 3F 006D6C    ; PC->AR, PC 加1
; //** End Of MicroController Data **//
```

选择联机软件的“【转储】—【装载】”功能，在打开文件对话框中选择上面所保存的文件，软件自动将机器程序和微程序写入指定单元。

选择联机软件的“【转储】—【刷新指令区】”可以读出下位机所有的机器指令和微指令，并在指令区显示，对照文件检查微程序和机器程序是否正确，如果不正确，则说明写入操作失败，应重新写入，可以通过联机软件单独修改某个单元的指令，以修改微指令为例，先用鼠标左键单击指令区的‘微存’TAB 按钮，然后再单击需修改单元的数据，此时该单元变为编辑框，输入6 位数据并回车，编辑框消失，并以红

色显示写入的数据。

3. 运行程序。

方法一：本机运行

将MC 单元的编程开关置为‘运行’档，MEM 单元的编程开关也置为‘运行’档，按动CON 单元的总清按钮CLR，将使程序计数器PC、地址寄存器AR 和微程序地址为00H，程序可以从头开始运行，暂存器A、B，指令寄存器IR 和OUT 单元也会被清零。

将时序单元的状态开关置为‘连续’档，按动一次TS 按钮，即可连续运行指令，按动KK+开关，每按动一次，检查OUT 单元显示的数是否在原有基础加1（第一次是在IN 单元值的基础加1）。

方法二：联机运行

进入软件界面，选择菜单命令“【实验】—【综合性实验】”，打开综合性实验数据通路图，选择相应的功能命令，即可联机运行、监控、调试程序。

按动CON 单元的总清按钮CLR，然后通过软件运行程序，在数据通路图和微程序流中观测程序的执行过程。在微程序流程图中观测：选择‘单周期’运行程序，当模型机执行完MOV指令后，按下KK+开关，不要松开，可见控制总线INTR 指示灯亮，继续‘单周期’运行程序，直到模型机的CPU 向8259 发送完第一个INTA ，然后松开KK+开关，INTR 中断请求被8259锁存，CPU 响应中断。仔细分析中断响应时现场保护的过程，中断返回时现场恢复的过程。

每响应一次中断，检查OUT 单元显示的数是否在原有基础加1（第一次是在IN 单元值的基础加1）。

1.7 典型I/O 接口8253 扩展设计实验

1.7..1 实验目的

(1) 掌握CPU 外扩接口芯片的方法。

(2) 掌握8253 定时器/计数器原理及其应用编程。

1.7..2 实验设备

PC 机一台，TD-CM3+实验系统一套。

1.7..3 实验原理

1. 8253 芯片引脚说明

(1) 8253 的引脚分配图如图5-1-1 所示。

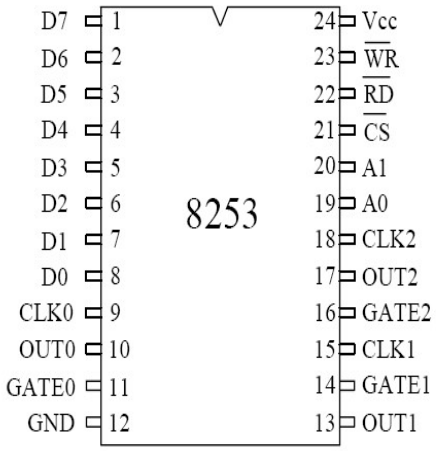


图1-7-1 8253 芯片引脚说明

(2) 芯片引脚说明

- D7～D0 为数据线。
- $\overline{CS}$为片选信号，低电平有效。
- A0、A1 用来选择三个计数器及控制寄存器。
- $\overline{RD}$为读信号，低电平有效。它控制8253 送出数据或状态信息至CPU。
- $\overline{WR}$为写信号，低电平有效。它控制把CPU 输出的数据或命令信号写到8253。
- CLKn、GATEn、OUTn 分别为三个计数器的时钟、门控信号及输出端。

$\overline{CS}$、A0、A1、$\overline{RD}$、$\overline{WR}$五个引脚的电平与8253 操作关系如表1-7-1 所示。

表1-7-1 引脚电平与8253 芯片的操作关系

$\overline{CS}$	$\overline{RD}$	$\overline{WR}$	A1	A0	寄存器选择和操作
0	1	0	0	0	写入寄存器 # 0
0	1	0	0	1	写入寄存器 # 1
0	1	0	1	0	写入寄存器 # 2
0	1	0	1	1	写入控制寄存器
0	0	1	0	0	读计数器 # 0
0	0	1	0	1	读计数器 # 1
0	0	1	1	0	读计数器 # 2
0	0	1	X	1	无操作（ 3 态）
1	X	X	1	X	禁止 （ 3 态）
0	0	1	X	X	无操作（ 3 态）

2．8253 芯片外部连接

对于CPU 外扩接口芯片，其重点是要设计接口芯片数据线、地址线和控制线与CPU 的挂接，图1-7-2 是8253 接口芯片的典型扩展接法。这里的模型计算机可以直接应用前面的复杂模型要，其I/O 地址空间分配情况如表1-7-2 所示。

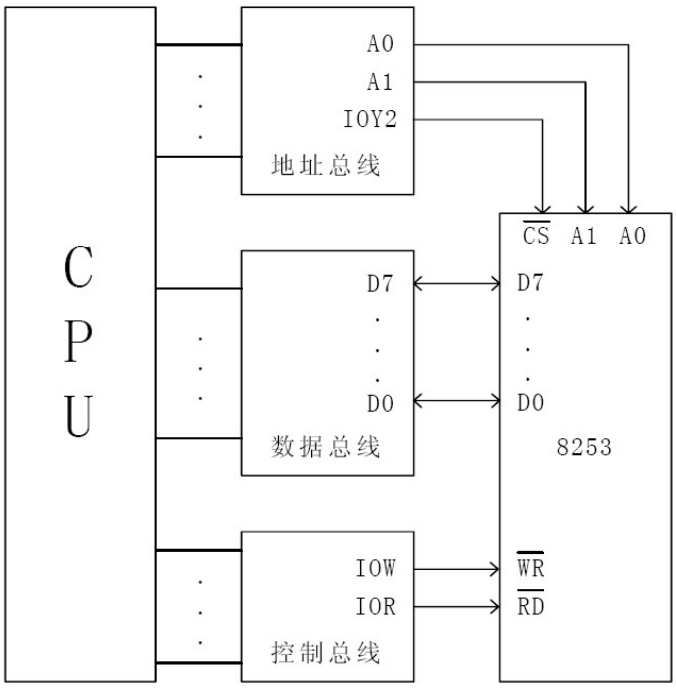


图1-7-2 8253 和CPU 挂接图

表1-7-2 I/O 地址空间分配

A7 A6	选定	地址空间
00	IOY0	00-3F
01	IOY1	40-7F
10	IOY2	80-BF
11	IOY3	C0-FF

我们可以应用复杂模型机指令系统的IN、OUT 指令来对外扩的8253 芯片进行操作。实验箱上8253 的数据线D7…D0 已接到系统的数据总线，GATE0 已接为高电平，其余都以排针形式引出。应用复杂模型机的指令系统，实现以下功能：对8253 进行初始化，使其以IN 单元数据N为计数初值，在OUT 端输出方波，8253 的输入时钟为时序单元的Φ。

根据实验要求编写机器程序如下：

```
; //******** Start Of Main Memory Data *********//
    $P 00 21     ; IN R1,00H                  IN->R1
    $P 01 00
    $P 02 C0     ; LAD R0,30H                 30  单元数据送R0（直接寻址）
    $P 03 30
    $P 04 30     ; OUT 83H,R0        R0 送83H 端口（写控制字）
    $P 05 83
    $P 06 34     ; OUT 80H,R1         R1  送80H 端口（写0#通道低字节）
```

```
        $P 07 80
        $P 08 50     ; HLT                停机
        $P 30 16     ;                    控制字
    ; //******** End Of Main Memory Data *********//
```

1.7..4 实验步骤

(1) 在复杂模型机实验接线图的基础上，再增加本实验8253 部分的接线（虚线框内部分）接线图如图1-7-3 所示。

(2) 本实验只用了计数器#0 通道，将它设置成方波速率发生器（方式3）。CLK0 接至信号源Φ 上；GATE0=1（已常接高），计数允许。OUT0 即为方波输出端。其中，30H 单元存放的数16H 为8253 的控制字，它的功能为选择计数器0，只读/写最低的有效字节，选择方式3，采用二进制。IN 单元的开关置的数N 为计数值，即输出是N 个CLK脉冲的方波。

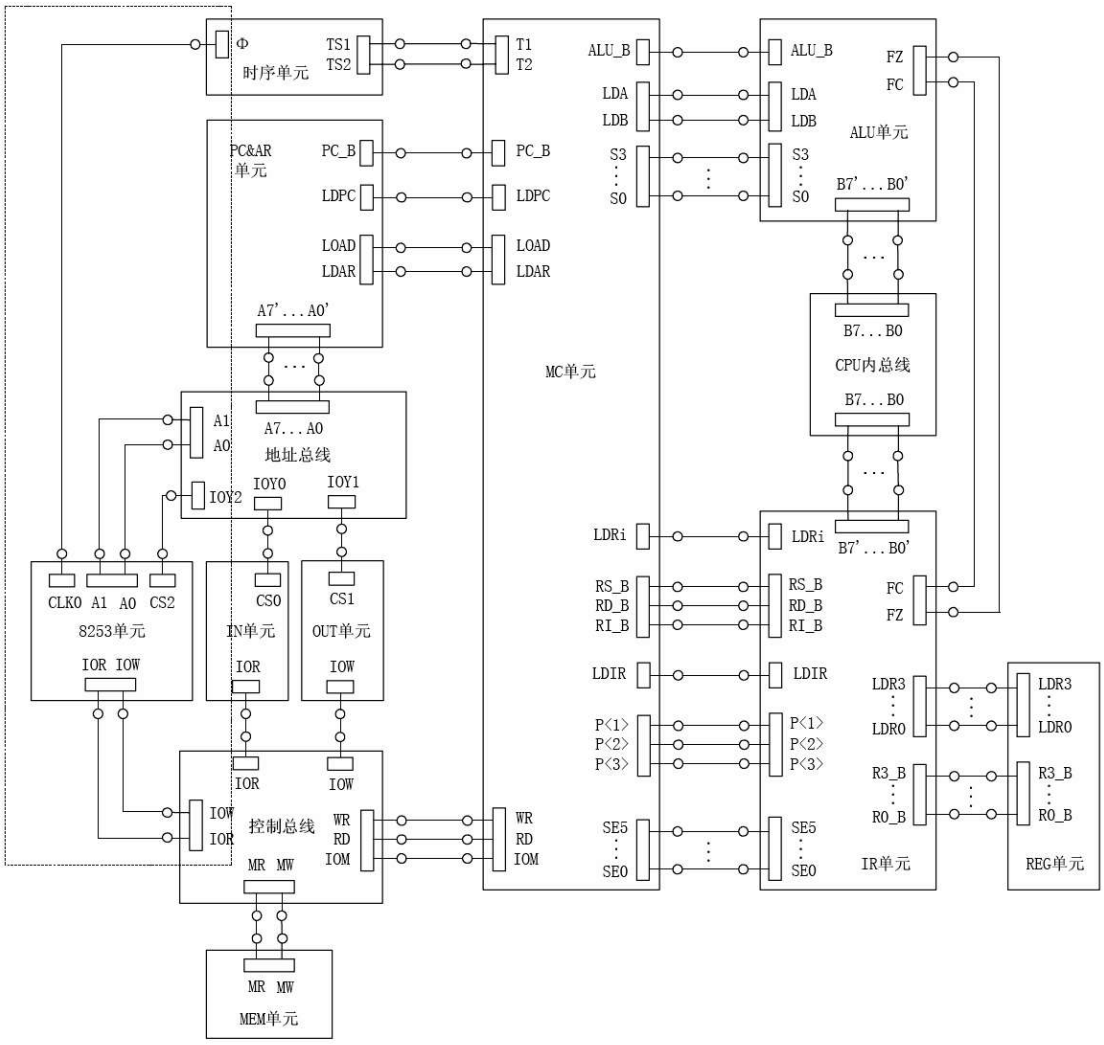


图1-7-3 实验接线图

(3) 微程序沿用复杂模型机的微代码程序，程序写入方法见实验1.6‘复杂模型机设计实验’。

(4) 运行上述程序，分两种情况:本机方式或联机方式，本机方式运行程序时，要借助示波器来观测8253 的输入和输出波形。而在联机方式时，可用CM3+联机操作软件的示波器功能测8253 的OUT0 端和时序单元信号源Φ 波形。进入软件界面，选择菜单命令“【实验】—【综合性实验】”，打开综合性实验数据通路图，选择相应的功能命令，即可联机运行、调试程序。当机器指令执行到HLT 指令时，停止运行

程序，再选择菜单命令“【波形】—【打开】”，打开示波器窗口，选择菜单命令“【波形】—【运行】”，启动逻辑示波器，就可观测到OUT0 端和时序单元信号源Φ 端的波形。将开关置不同的计数值，按下CON 单元的CLR，再运行机器指令后，可观察到OUT0 端输出波形的频率变化。

第 2 章　设计性实验

2.1 超前进位加法器设计实验

2.1.1 实验目的

(1) 掌握超前进位加法器的原理及其设计方法。

(2) 熟悉CPLD 应用设计及EDA 软件的使用。

2.1.2 实验设备

PC 机一台，TD-CM3+实验系统一套。

2.1.3 实验原理

加法器是执行二进制加法运算的逻辑部件，也是CPU 运算器的基本逻辑部件（减法可以通过补码相加来实现）。加法器又分为半加器和全加器（FA），不考虑低位的进位，只考虑两个二进制数相加，得到和以及向高位进位的加法器为半加器，而全加器是在半加器的基础上又考虑了低位过来的进位信号。

表2-1-1 1 位全加器真值表

输入	输出
A B Ci	S Co
0 0 0	0 0
0 0 1	1 0
0 1 0	1 0
0 1 1	0 1
1 0 0	1 0
1 0 1	0 1
1 1 0	0 1
1 1 1	1 1

A、B 为2 个1 位的加数，Ci 为来自低位的进位，S 为和，Co 为向高位的进位，根据表2-1-1所示的真值表，可得到全加器的逻辑表达式为：

$$S = A\overline{BCi} + \overline{A}B\overline{Ci} + \overline{AB}Ci + ABCi$$

$$Co = AB + ACi + BCi$$

根据逻辑表达式，可得到如图2-1-1 所示的逻辑电路图。

有了1 位全加器，就可以用它来构造多位加法器，加法器根据电路结构的不同，可以分为串行加法器和并行加法器两种。串行加法器低位全加器产生的进位要依次串行地向高位进位，其电路简单，占用资源较少，但是串行加法器每位和以及向高位的进位的产生都依赖于低位的进位，导致完成加法运算的延迟时间较长，效率并不高。

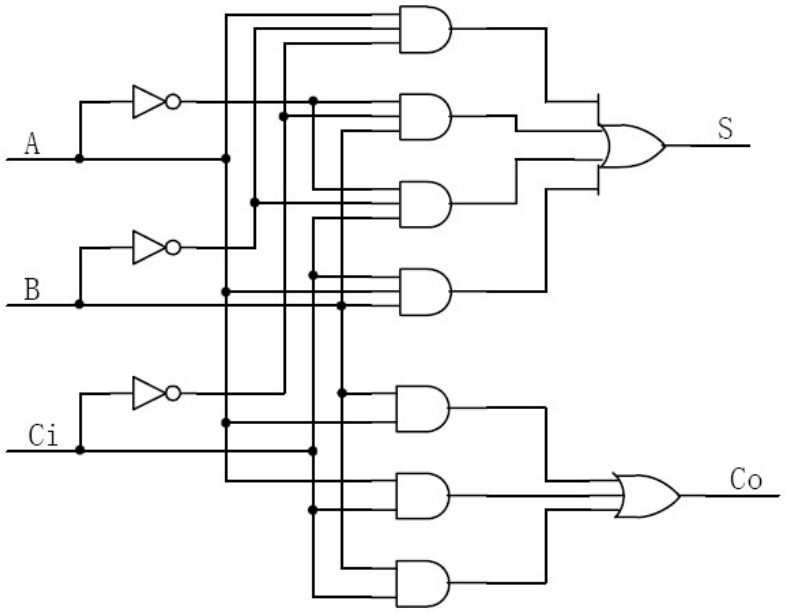


图2-1-1　1位全加器（FA）逻辑电路图

串行加法器运算速度慢，其根本原因是每一位的结果都要依赖于低位的进位，因而可以通过并行进位的方式来提高效率。只要能设计出专门的电路，使得每一位的进位能够并行地产生而与低位的运算情况无关，就能解决这个问题。可以对加法器进位的逻辑表达式做进一步的推导：

$C_0 = 0$

$C_{i+1} = A_iB_i + A_iC_i + B_iC_i = A_iB_i + (A_i + B_i)C_i$

设

$g_i = A_iB_i$

$p_i = A_i + B_i$

则有：

$$C_{i+1} = g_i + p_iC_i$$
$$= g_i + p_i(g_{i-1} + p_{i-1}C_{i-1})$$
$$= g_i + p_i(g_{i-1} + p_{i-1}(g_{i-2} + p_{i-2}C_{i-2}))$$
$$\cdots$$
$$= g_i + p_i(g_{i-1} + p_{i-1}(g_{i-2} + p_{i-2}(\cdots(g_0 + p_0C_0)\cdots)))$$
$$= g_i + p_ig_{i-1} + p_ip_{i-1}g_{i-2} + \cdots + p_ip_{i-1}\cdots p_1g_0 + p_ip_{i-1}\cdots p_1p_0C_0$$

由于g_i、p_i 只和A_i、B_i 有关，这样C_{i+1} 就只和A_i、A_{i-1}、…、A_0，B_i、B_{i-1}、…、B_0 及C_0有关。所以各位的进位C_i、C_{i-1}、…、C_1 就可以并行地产生，这种进位就叫超前进位。

根据上面的推导，随着加法器位数的增加，越是高位的进位逻辑电路就会越复杂，逻辑器件使用也就越多。事实上我们可以继续推导进位的逻辑表达式，使得某些基本逻辑单元能够复用，且能照顾到进位位的并行产生。

定义

$$G_{i,j} = g_i + p_ig_{i-1} + p_ip_{i-1}g_{i-2} + \cdots + p_ip_{i-1}\cdots p_{j+1}g_j$$

$$P_{i,j} = p_ip_{i-1}\cdots p_{j+1}p_j$$

则有

$G_{i,i} = g_i$

$P_{i,i} = p_i$

$G_{i,j} = G_{i,k} + P_{i,k}G_{k-1,j}$

$P_{i,j} = P_{i,k}P_{k-1,j}$

$C_{i+1} = G_{i,j} + P_{i,j}C_j$

从而可以得到表2-1-2 所示的算法，该算法为超前进位算法的扩展算法，这里实现的是一个8 位加法器的算法。

表2-1-2 超前进位扩展算法

<table>
<tr><td>$G_{1,0} = g_1 + p_1g_0$
$P_{1,0} = p_1p_0$</td><td rowspan="2">$G_{3,0} = G_{3,2} + P_{3,2}G_{1,0}$
$P_{3,0} = P_{3,2}P_{1,0}$</td><td rowspan="4">$G_{7,0} = G_{7,4} + P_{7,4}G_{3,0}$
$P_{7,0} = P_{7,4}P_{3,0}$</td></tr>
<tr><td>$G_{3,2} = g_3 + p_3g_2$
$P_{3,2} = p_3p_2$</td></tr>
<tr><td>$G_{5,4} = g_5 + p_5g_4$
$P_{5,4} = p_5p_4$</td><td rowspan="2">$G_{7,4} = G_{7,6} + P_{7,6}G_{5,4}$
$P_{7,4} = P_{7,6}P_{5,4}$</td></tr>
<tr><td>$G_{7,6} = g_7 + p_7g_6$
$P_{7,6} = p_7p_6$</td></tr>
<tr><td colspan="3">$C_8 = G_{7,0} + P_{7,0}C_0$</td></tr>
</table>

从上表可以看出，本算法的核心思想是把8 位加法器分成两个4 位加法器，先求出低4 位加法器的各个进位，特别是向高4 位加法器的进位C_4。然后，高4 位加法器把C_4 作为初始进位，使用低4 位加法器相同的方法来完成计算。每一个4 位加法器在计算时，又分成了两个2位的加法器。如此递归，如图2-1-2 所示。

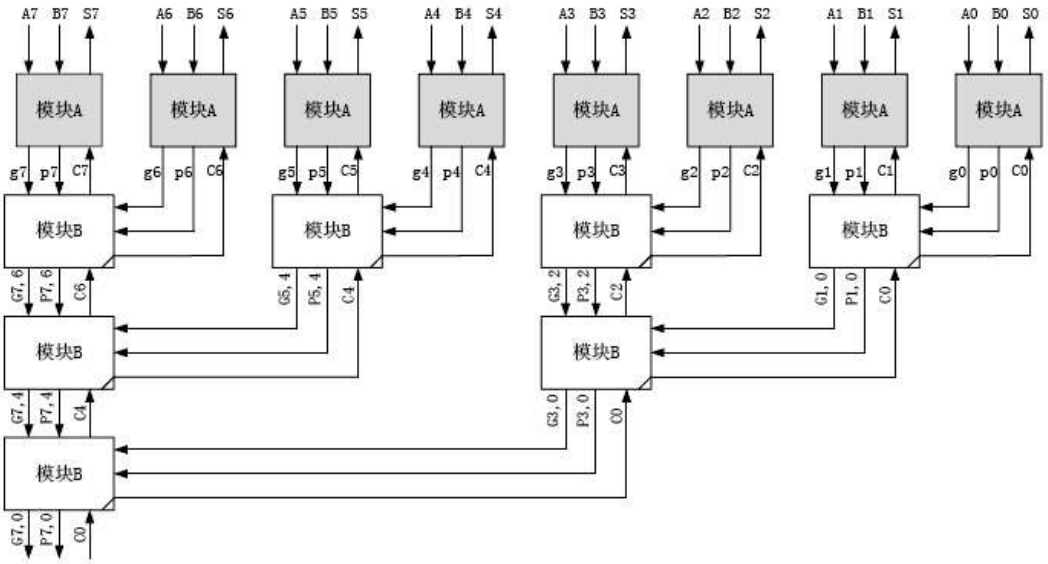


图2-1-2 超前进位扩展算法示意图

这样，在超前进位扩展算法的逻辑电路实现中，需要设计两种电路。模块A 逻辑电路需要完成如下计算逻辑，其原理图如图2-1-3 所示。

$G_{i,i} = A_iB_i$

$P_{i,i} = A_i + B_i$

$S_i = \overline{A}\overline{B}\overline{C}i + \overline{A}B\overline{C}i + A\overline{B}\overline{C}i + ABCi$

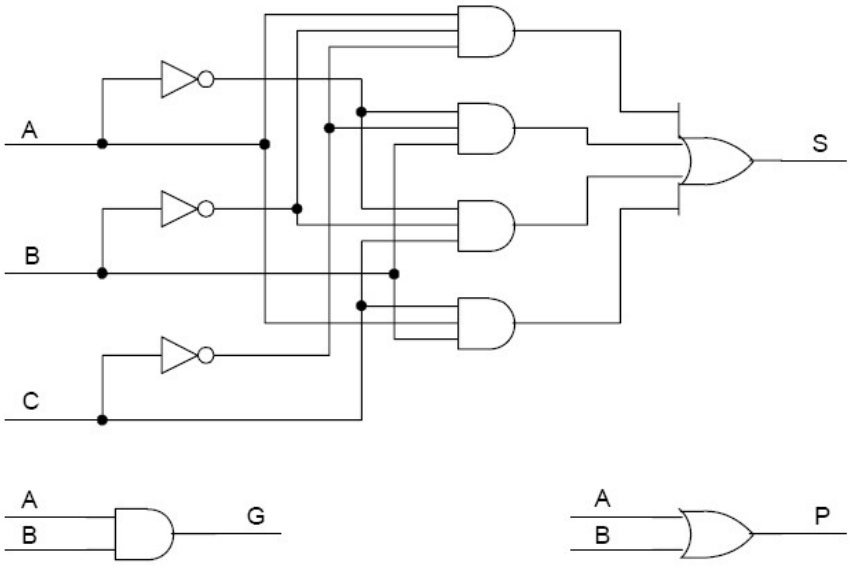


图2-1-3 模块A 原理图

模块B 逻辑电路需要完成如下计算逻辑，其原理图如图2-1-4 所示。

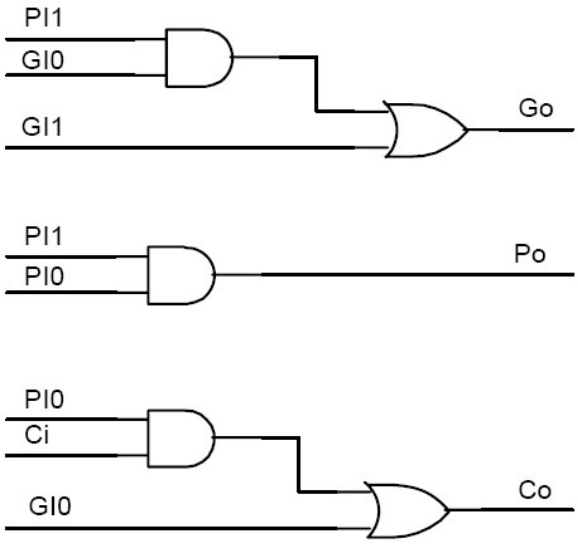


图2-1-4 模块B 原理图

$G_{i,j} = G_{i,k} + P_{i,k}G_{k-1,j}$

$P_{i,j} = P_{i,k}P_{k-1,j}$

$C_{i+1} = G_{i,j} + P_{i,j}C_j$

按图2-1-2 将这两种电路连接起来，就可以得到一个8 位的超前进位的加法器。

从图中可以看到$G_{i,i}$ 和$P_{i,i}$ 既参与了每位上进位的计算，又参与了下一级$G_{i,i}$ 和$P_{i,i}$ 的计算。这样就复用了这些电路，使得需要的总逻辑电路数大大减少。超前进位加法器的运算速度较快，但是，与串行进位加法器相比，逻辑电路比较复杂，使用的逻辑器件较多，这些是为提高运算速度付出的代价。

本实验在FPGA 单元上进行，FPGA 单元由由两大部分组成，一是LED 显示灯，两组16只，供调试时观测数据，LED 灯为正逻辑，1 时亮，0 时灭。另外是一片Altera Cyclone EP1C6Q240及其外围电路。FPGA有240 个引脚，分成四个块，即BANK1…BANK4，将每个块的通用I/O 脚加以编号，就形成A01…A34、B01…B35等I/O 号，如图2-1-5 所示。FPGA 单元排针的丝印分为两部分，一是I/O 号，以A、B、C、D 打头，如A15，一是芯片引脚号，是纯数字，如21，它们表示的是同一个引脚。在Quartus II 软件中分配I/O 时用的是引脚号，而在实验接线图中，都以I/O 号来描述。

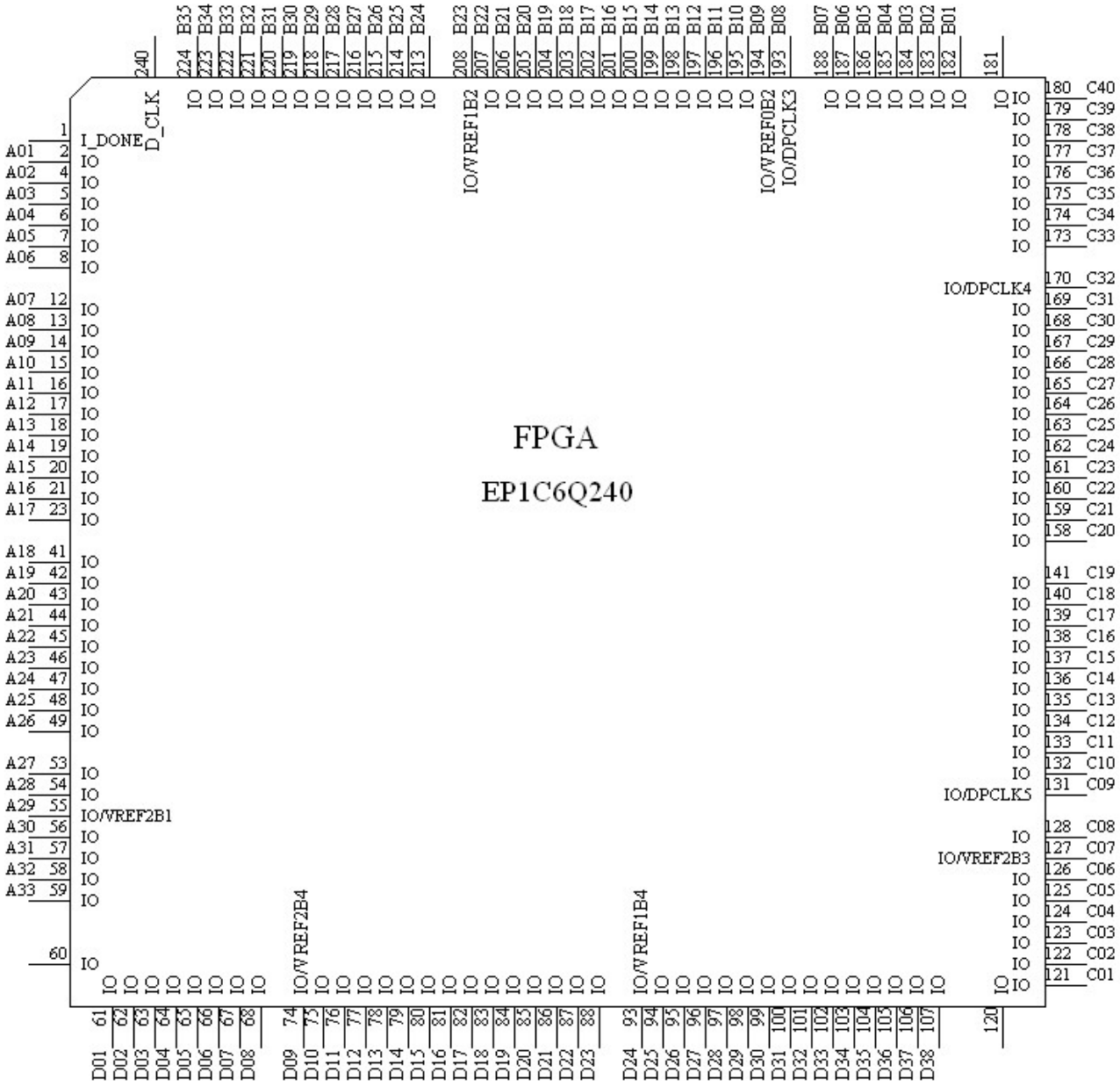


图2-1-5 EP1C6Q240 引脚分配图

EP1C6Q240 共有178 个I/O 脚，本单元引出134 个，其中84 个以排针形式引出，供实验使用，其余50 个以双列扩展插座形式给出，并标记为JP，JP 座的I/O 分配如图2-1-6 所示。

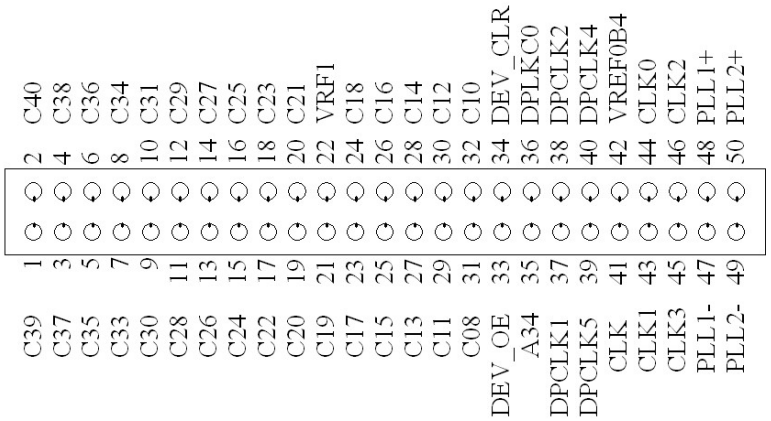


图2-1-6 JP 座I/O 分配图

2.1.4 实验步骤

(1) 根据上述加法器的逻辑原理使用Quartus II 软件编辑相应的电路原理图并进行编译，其在EP1C6 芯片中对应的引脚如图2-1-7 所示，框外文字表示I/O 号，框内文字表示该引脚的含义（本实验例程见‘安装路径\Design\Adder\Adder.qpf’工程）。

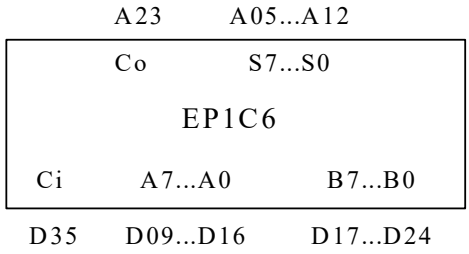


图2-1-7 引脚分配图

(2) 关闭实验系统电源，按图2-1-8 连接实验电路，图中将用户需要连接的信号用圆圈标明。

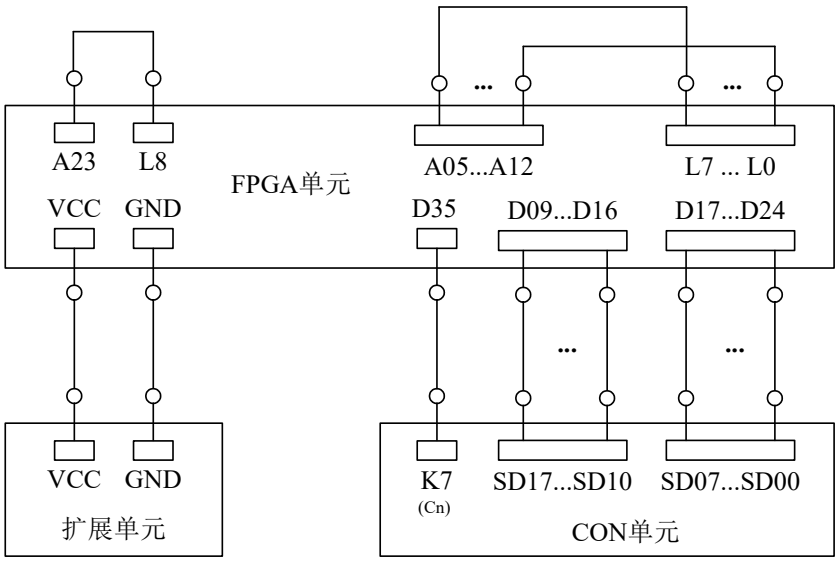


图2-1-8 实验接线图

(3) 打开实验系统电源，将生成的POF 文件下载到EP1C6 中去。

(4) 以CON 单元中的SD17…SD10 八个二进制开关为被加数A，SD07…SD00 八个二进制开关为加数B，

K7 用来模拟来自低位的进位信号，相加的结果在FPGA 单元的L7…L0 八个LED灯显示，相加后向高位的进位用FPGA 单元的L8 灯显示。给A 和B 置不同的数，观察相加的结果。

2.2 阵列乘法器设计实验

2.2.1 实验目的

(1) 掌握乘法器的原理及其设计方法。

(2) 熟悉FPGA 应用设计及EDA 软件的使用。

2.2.2 实验设备

PC 机一台，TD-CM3+实验系统一套。

2.2.3 实验原理

硬件乘法器常规的设计是采用“串行移位”和“并行加法”相结合的方法，这种方法并不需要很多的器件，然而“加法-移位”的方法毕竟太慢。随着大规模集成电路的发展，采用高速的单元阵列乘法器，无论从计算机的计算速度，还是从提高计算效率，都是十分必要的。阵列乘法器分带符号和不带符号的阵列乘法器，本节只讨论不带符号阵列乘法。高速组合阵列乘法器，采用标准加法单元构成乘法器，即利用多个一位全加器（FA）实现乘法运算。

对于一个4 位二进制数相乘，有如下算式：

					A3	A2	A1	A0
×					B3	B2	B1	B0
					A3B0	A2B0	A1B0	A0B0
				A3B1	A2B1	A1B1	A0B1	
			A3B2	A2B2	A1B2	A0B2		
+		A3B3	A2B3	A1B3	A0B3			
	P7	P6	P5	P4	P3	P2	P1	P0

这个4 × 4 阵列乘法器的原理如图2-2-1 所示。

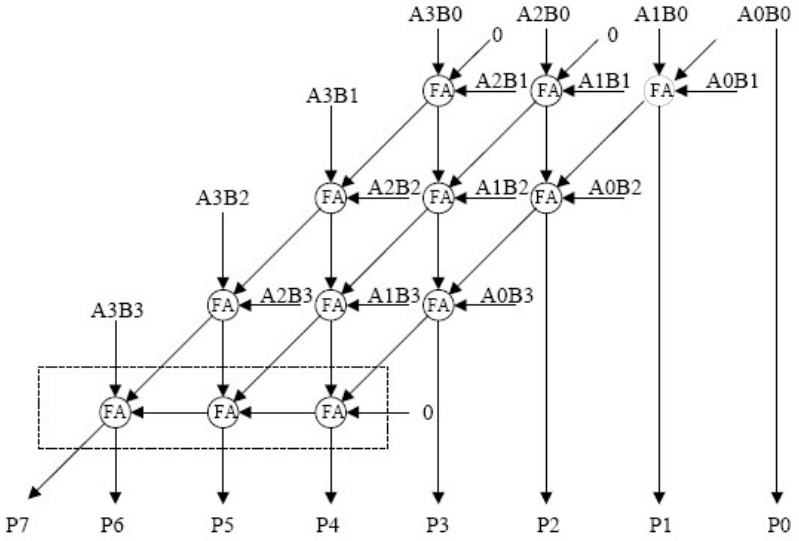


图2-2-1 4 4 阵列乘法器原理图

FA（全加器）的斜线方向为进位输出，竖线方向为和输出。图中阵列的最后一行构成了一个串行进

位加法器。由于FA 一级是无需考虑进位的，它的进位被暂时保留下来不往前传递，因此同一极中任意一位FA 加法器的进位输出与和输出几乎是同时形成的，与“串行移位”相比可大大减少同级间的进位传递延迟，所以送往最后一行串行加法器的输入延迟仅与FA 的级数（行数）有关，即与乘数位数有关。本实验用FPGA 来设计一个4×4 位加法器，且全部采用原理图方式实现。

2.2.4 实验步骤

(1) 根据上述阵列乘法器的原理，使用Quartus II 软件编辑相应的电路原理图并进行编译，其在EP1C6芯片中对应的引脚如图2-2-2 所示，框外文字表示I/O 号，框内文字表示该引脚的含义（本实验例程见‘安装路径\Design\Multiply\Multiply.qpf’工程）。

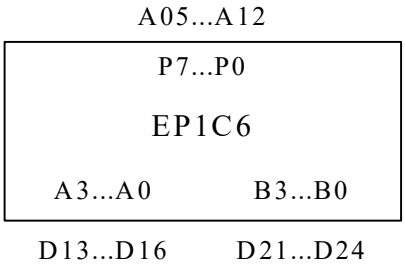


图2-2-2 引脚分配图

(2) 关闭实验系统电源，按图2-2-3 连接实验电路，图中将用户需要连接的信号用圆圈标明。(3) 打开实验系统电源，将生成的POF 文件下载到EP1C6 中去，FPGA 单元介绍见实验2.1。

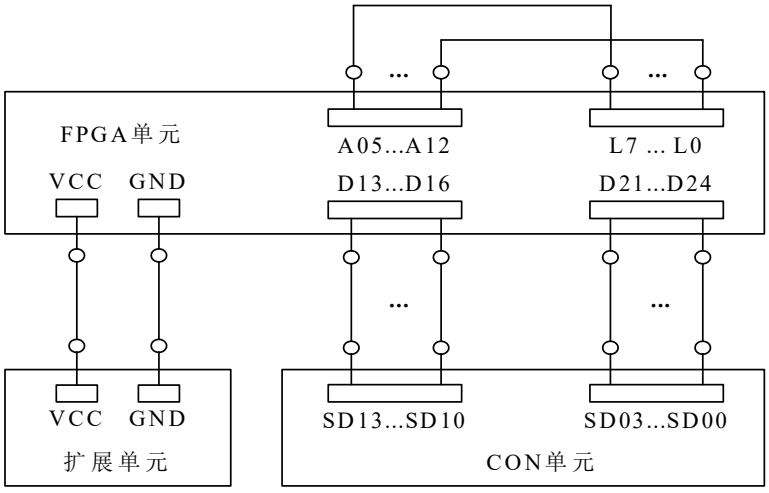


图2-2-3 阵列乘法器实验接线图

(4) 以CON 单元中的SD13…SD10 四个二进制开关为乘数A，SD13…SD10 四个二进制开关为被乘数B，而相乘的结果在FPGA 单元的L7…L0 八个LED 灯显示。给A 和B 置不同的数，观察相乘的结果。

2.3 Cache 控制器设计实验

2.3. 1 实验目的

(1) 掌握Cache 控制器的原理及其设计方法。

(2) 熟悉FPGA 应用设计及EDA 软件的使用。

2.3.2 实验设备

PC 机一台，TD-CM3+实验系统一套。

2.3.3 实验原理

本实验采用的地址变换是直接映象方式，这种变换方式简单而直接，硬件实现很简单，访问速度也比较快，但是块的冲突率比较高。其主要原则是：主存中一块只能映象到Cache 的一个特定的块中。

假设主存的块号为B，Cache 的块号为b，则它们之间的映象关系可以表示为：

$$b = B \bmod C_b$$

其中，C_b 是Cache 的块容量。设主存的块容量为M_b，区容量为M_e，则直接映象方法的关系如图2-3-1 所示。把主存按Cache 的大小分成区，一般主存容量为Cache 容量的整数倍，主存每一个分区内的块数与Cache 的总块数相等。直接映象方式只能把主存各个区中相对块号相同的那些块映象到Cache 中同一块号的那个特定块中。例如，主存的块0 只能映象到Cache 的块0 中，主存的块1 只能映象到Cache 的块1 中，同样，主存区1 中的块C_b（在区1 中的相对块号是0）也只能映象到Cache 的块0 中。根据上面给出的地址映象规则，整个Cache 地址与主存地址的低位部分是完全相同的。

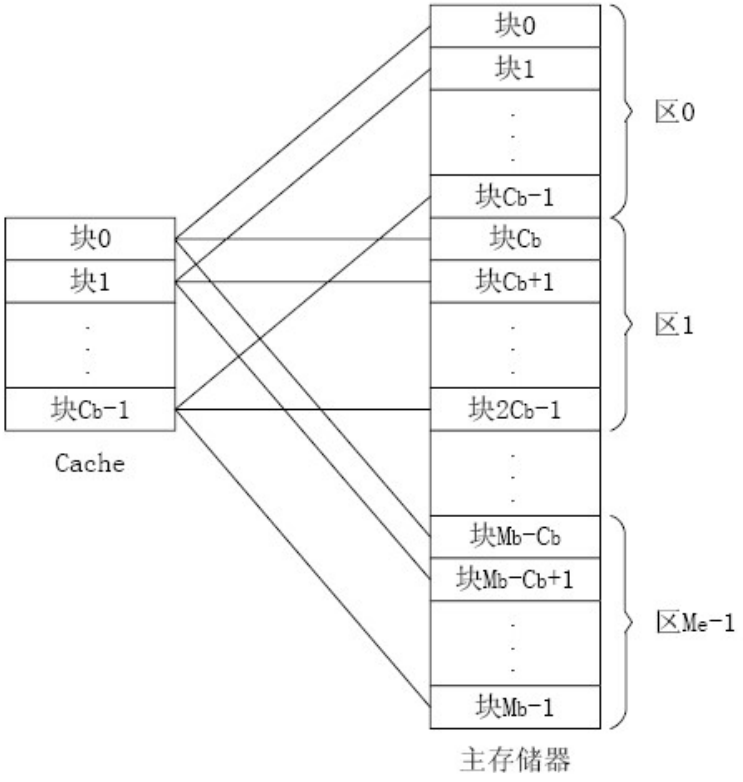


图2-3-1 直接相联映象方式

直接映象方式的地址变换过程如图2-3-2 所示，主存地址中的块号B 与Cache 地址中的块号b 是完全相同的。同样，主存地址中的块内地址W 与Cache 地址中的块内地址w 也是完全相同的，主存地址比Cache 地址长出来的部分称为区号E。

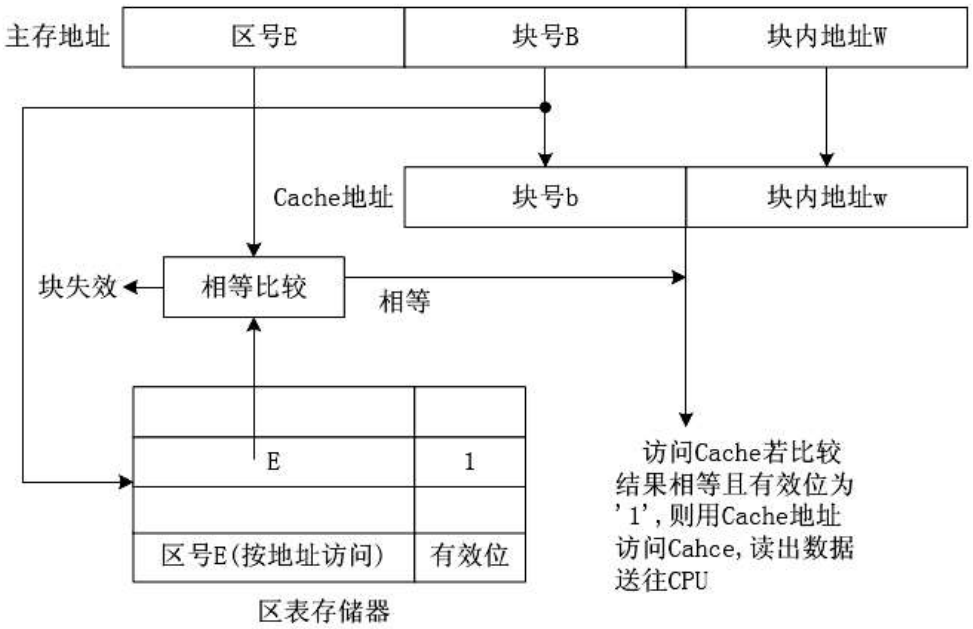


图2-3-2 直接相联地址变换

在程序执行过程中，当要访问Cache 时，为了实现主存块号到Cache 块号的变换，需要有一个存放主存区号的小容量存储器，这个存储器的容量与Cache 的块数相等，字长为主存地址中区号E 的长度，另外再加一个有效位。

在主存地址到Cache 地址的变换过程中，首先用主存地址中的块号去访问区号存储器（按地址访问）。把读出来的区号与主存地址中的区号E 进行比较，根据比较结果和与区号在同一存储字中的有效位情况作出处理。如果区号比较结果相等，有效位为‘1’，则Cache 命中，表示要访问的那一块已经装入到Cache 中了，这时Cache 地址（与主存地址的低位部分完全相同）是正确的。用这个Cache 地址去访问Cache，把读出来的数据送往CPU。其他情况均为Cache没有命中，或称为Cache 失效，表示要访问的那个块还没有装入到Cache 中，这时，要用主存地址去访问主存储器，把读出来的一个字送往CPU，同时，把包括被访问字在内的一块都从主存储器中读出来。

本实验要在FPGA 中实现Cache 及其地址变换逻辑（也叫Cache 控制器），采用直接相联地址变换，只考虑CPU 从Cache 读数据，不考虑CPU 写回数据的情况，Cache 和CPU 以及存储器的关系如图2-3-3 所示。

Cache 控制器顶层模块如图2-3-4 所示，主存地址为A7…A0,共8 位，区号E 取3 位，这样Cache 地址还剩5 位，所以Cache 容量为32 个单元，块号B 取3 位，那么Cache 分为8 块，块内地址W 取2 位，则每块为4 个单元。图2-3-4 中，WCT 为写Cache 块表信号，CLR 为系统总清零信号，A7…A0 为CPU 访问内存的地址，M 为Cache 失效信号，CA4…CA0 为Cache 地址，

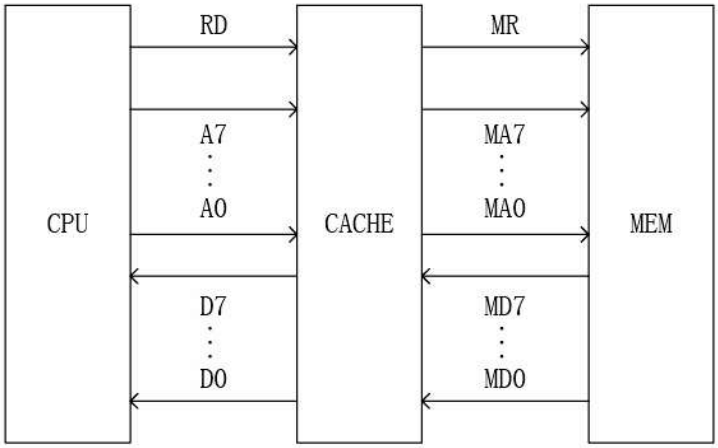


图2-3-3 Cache 系统图

MD7…MD0 为主存送Cache 的数据，D7…D0 为Cache 送CPU 数据，T2 为系统时钟，RD 为CPU 访问内存读信号，LA1 和LA0 为块内地址。

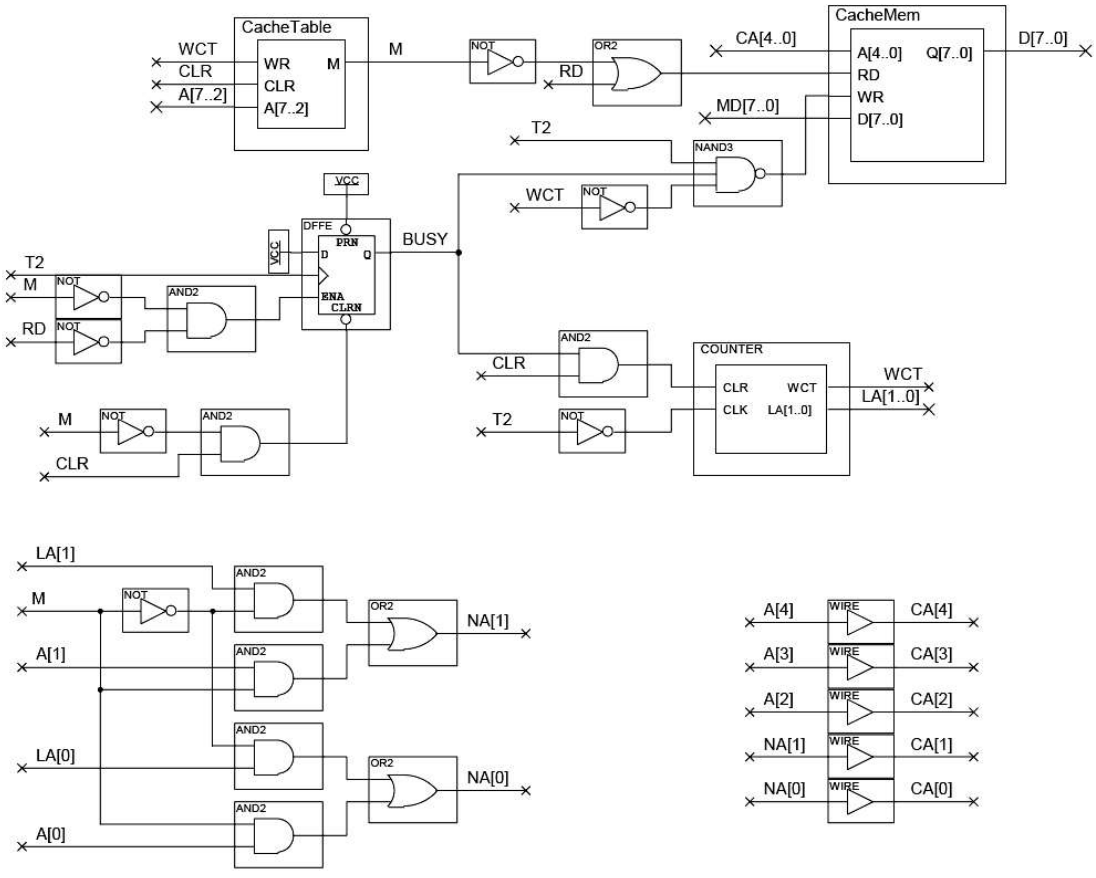


图2-3-4 Cache 控制器顶层模块图

在QuartusII 软件中先实现一个8 位的存储单元（见例程中的MemCell.bdf），然后用这个8位的存储单元来构成一个32 X 8 位的Cache（见例程中的CacheMem.bdf）,这样就实现了Cache的存储体。

再实现一个4 位的存储单元（见例程中的TableCell.bdf），然后用这个4 位的存储单元来构成一个8 X 4 位的区表存储器，用来存放区号和有效位（见例程中的CacheTable.bdf）,在这个文件中，还实现了一个区号比较器，如果主存地址的区号E 和区表中相应单元中的区号相等，且有效位为1，则Cache 命中，否则Cache 失效，标志为M，M 为0 时表示Cache 失效。当Cache 命中时，就将Cache 存储体中相应单元的数据送往CPU，这个过程比较简单。当Cache 失效时，就将主存中相应块中的数据读出写入Cache 中，

这样Cache 控制器就要产生访问主存储器的地址和主存储器的读信号，由于每块占四个单元，所以需要连续访问四次主存，这就需要一个低地址发生器，即一个2 位计数器（见例程中的Counter.vhd），将低2 位和CPU 给出的高6 位地址组合起来，形成访问主存储器的地址。M 就可以做为主存的读信号，这样，在时钟T2 的控制下，就可以将主存中相应的块写入到Cache 的相应块中，最后再修改区表（见例程中的（CacheCtrl.bdf）。

2.3.4 实验步骤

(1) 使用Quartus II 软件编辑实现相应的逻辑并进行编译，直到编译通过，Cache 控制器在EP1C6芯片中对应的引脚如图2-3-5 所示，框外文字表示I/O 号，框内文字表示该引脚的含义（本实验例程见‘安装路径\Design\CacheCtrl\CacheCtrl.qpf’工程）。

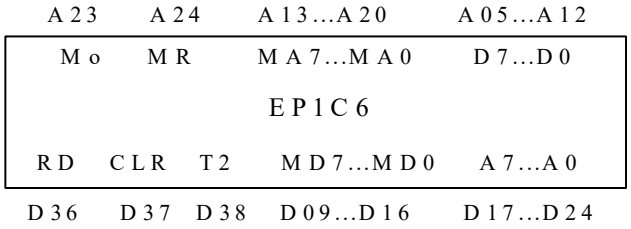


图2-3-5 引脚分配图

(2) 关闭实验系统电源，按图2-3-6 连接实验电路，并检查无误，图中将用户需要连接的信号用圆圈标明。

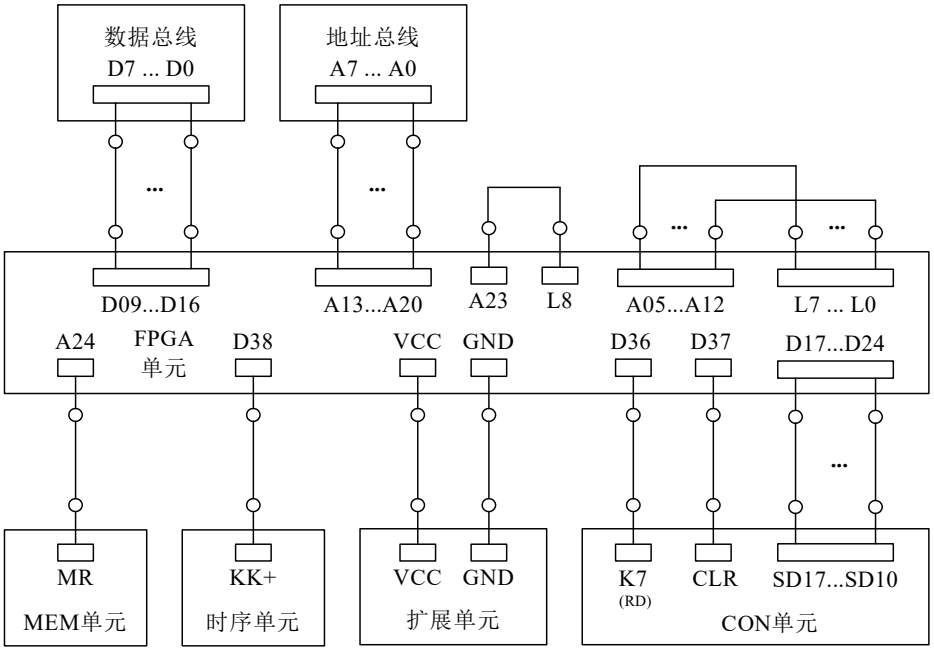


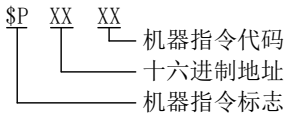
图2-3-6 实验接线图

(3) **打开实验系统电源，将生成的SOF（JTAG模式）或POF（AS模式）文件下载到EP1C6中去，FPGA单元介绍见实验2.1**。

(4) 将MEM 单元的编程开关置为‘运行’档，CLR 信号由CON 单元的CLR 模拟给出，按动CON 单元的CLR 按钮，清空区表。

(5) 预先往主存写入数据：联机软件提供了机器程序下载功能，以代替手动读写主存，机器程序以指定的格式写入到以TXT 为后缀的文件中，机器指令的格式如下：

机器指令格式说明:



如$P 1F 11，表示机器指令的地址为1FH，指令值为11H，本次实验只初始化00-0FH 共16个单元，初始数据如下，程序中分号‘；’为注释符，分号后面的内容在下载时将被忽略掉。

```
; //*************************************//
; //                                     //
; //      Cache 控制器实验指令文件       //
; //                                     //
; //*************************************//

; //***** Start Of Main Memory Data ******//
  $P 00 11    ; 数据
  $P 01 22
  $P 02 33
  $P 03 44
  $P 04 55
  $P 05 66
  $P 06 77
  $P 07 88
  $P 08 99
  $P 09 AA
  $P 0A BB
  $P 0B CC
  $P 0C DD
  $P 0D EE
  $P 0E FF
  $P 0F 00
; //****** End Of Main Memory Data *******//
```

用联机软件的“【转储】—【装载】”功能将该格式（*.TXT）文件装载入实验系统。装入过程中，在软件的输出区的‘结果’栏会显示装载信息，如当前正在装载的是机器指令还是微指令，还剩多少条指令等。

(6) 联机软件在启动时会读取所有机器指令和微指令，在指令区显示，软件启动后，也可以选择菜单命令“【转储】—【刷新指令区】”读取下位机指令，并在指令区显示。点击指令区的 ‘主存’TAB 按钮，两列数据中显示了主存的所有数据，第一列为主存地址，第二列为该地址中的数据。对上面文件检查机器程序是否正确，如果不正确，则说明写入操作失败，应重新写入，可以通过联机软件单独修改某个单元的指令，单击需修改单元的数据，此时该单元变为编辑框，输入2 位数据并回车，编辑框消失，写入数据以红色显示。

7) CPU 访问主存地址由CON 单元的SD17…SD10 模拟给出，如00000001。CPU 访问主存的读信号由CON 单元的K7 模拟给出，置K7 为低，可以观察到FPGA 单元上的L8 指示灯亮，L0…L7 指示灯灭，表示Cache 失效。此时按动KK+按钮四次，注意数据总线和地址总线上的指示灯的变化情况，地址会依次加一，数据总线上显示的是当前主存数据，按动四次KK+按钮后， L8 指示灯变灭，L0…L7 上显示的值即为Cache 送往CPU 的数据。

(8) 重新给出主存访问地址，如00000011，L8 指示灯变灭，表示Cache 命中，说明第0 块数据已写入Cache。

(9) 记住01H 单元的数据，然后通过联机软件，修改01H 单元的数据，重新给出主存访问地址00000001，再次观察L0-L7 指示灯表示的值是01H 单元修改前的值，说明送往CPU 的数据是由Cache 给出的。

(10) 重新给出大于03H 地址，体会Cache 控制器的工作过程。

2.4 硬布线控制器模型机设计实验

2.4.1 实验目的

(1) 掌握硬布线控制器的组成原理、设计方法。

(2) 了解硬布线控制器和微程序控制器的各自优缺点。

2.4.2 实验设备

PC 机一台，TD-CM3+实验系统一套。

2.4.3 实验原理

硬布线控制器本质上是一种由门电路和触发器构成的复杂树形网络，它将输入逻辑信号转换成一组输出逻辑信号，即控制信号。硬布线控制器的输入信号有：指令寄存器的输出、时序信号和运算结果标志状态信号等，输出的就是所有各部件需要的各种微操作信号。

硬布线控制器的设计思想是：在硬布线控制器中，操作控制器发出的各种控制信号是时间因素和空间因素的函数。各个操作定时的控制构成了操作控制信号的时间特征，而各种不同部件的操作所需要的不同操作信号则构成了操作控制信号的空间特征。硬布线控制器就是把时间信号和操作信号组合，产生具有定时特点的控制信号。

简单模型机的控制器是微程序控制器，本实验中的模型机将用硬布线取代微程序控制器，其余部件和简单模型机的一样，所以其数据通路图也和简单模型机的一样，见图1-4-3，机器指令也和简单模型机的机器指令一样，如下所示。

助记符	机器指令码	说明
IN	0010 0000	IN→R0
ADD	0000 0000	R0 + R0→R0
OUT	0011 0000	R0→OUT
JMP addr	1110 0000 ********	addr→ PC
HLT	1110 0000	停机

根据指令要求，得出用时钟进行驱动的状态机描述，即得出其有限状态机，如图2-4-1 所示。

下面分析每个状态中的基本操作：

S0：空操作，系统复位后的状态

S1：PC->AR,PC+1

S2：MEM->BUS,BUS->IR

S3：R0->BUS,BUS->A

S4：R0->BUS,BUS->B

S5：A 加B->BUS,BUS->R0

S6：IN->BUS,BUS->R0

S7：R0->BUS,BUS->OUT

S8：空操作

S9：PC->AR,PC+1

S10：MEM->BUS,BUS->PC

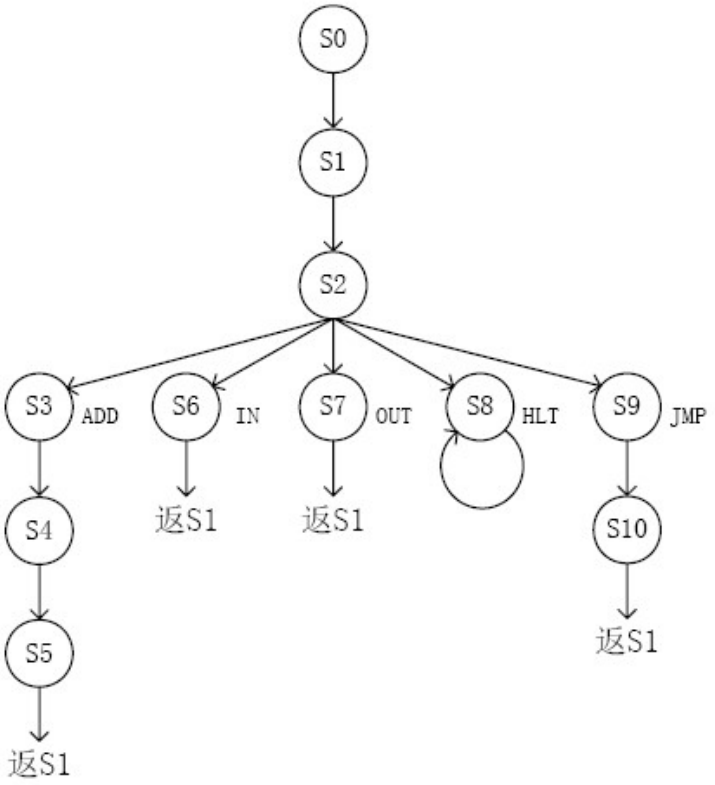


图2-4-1 状态机描述

设计一段机器程序，要求从IN 单元读入一个数据，存于R0，将R0 和自身相加，结果存于R0，再将R0 的值送OUT 单元显示。

2.4.4 实验步骤

(1) 分析每个状态所需的控制信号，并汇总成表，如表2-4-1 所示。

表2-4-1 控制信号表

状态号	控制信号
S0	0 0 0 0 0 0 0 0 0 1 0 0 1 1 0 1 0
S1	0 0 0 0 0 0 0 0 0 1 1 0 1 1 0 0 1
S2	0 1 0 0 0 0 0 0 0 1 0 1 1 1 0 1 0
S3	0 0 0 0 0 0 0 1 0 1 0 0 1 0 0 1 0
S4	0 0 0 0 0 0 0 0 1 1 0 0 1 0 0 1 0
S5	0 0 0 1 0 0 1 0 0 1 0 0 0 1 1 1 0
S6	0 1 1 0 0 0 0 0 0 1 0 0 1 1 1 1 0
S7	1 0 1 0 0 0 0 0 0 1 0 0 1 0 0 1 0
S8	0 0 0 0 0 0 0 0 0 1 0 0 1 1 0 1 0
S9	0 0 0 0 0 0 0 0 0 1 1 0 1 1 0 0 1
S10	0 1 0 0 0 0 0 0 0 0 0 0 1 1 0 1 1

控制信号由左至右，依次为：WR，RD，IOM，S3，S2，S1，S0，LDA，LDB，LOAD，LDAR，LDIR，ALU_B，R0_B，LDR0，PC_B，LDPC。

(2) 用VHDL语言来设计本实验的状态机，使用Quartus II 软件编辑VHDL 文件并进行编译，硬布线控制器在EP1C6 芯片中对应的引脚如图2-4-2 所示（本实验例程见‘安装路径

\Design\Controller\Controller.qpf’工程）。

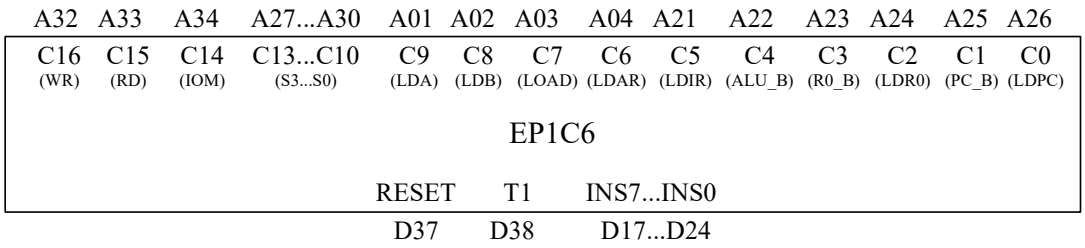


图2-4-2 引脚分配图

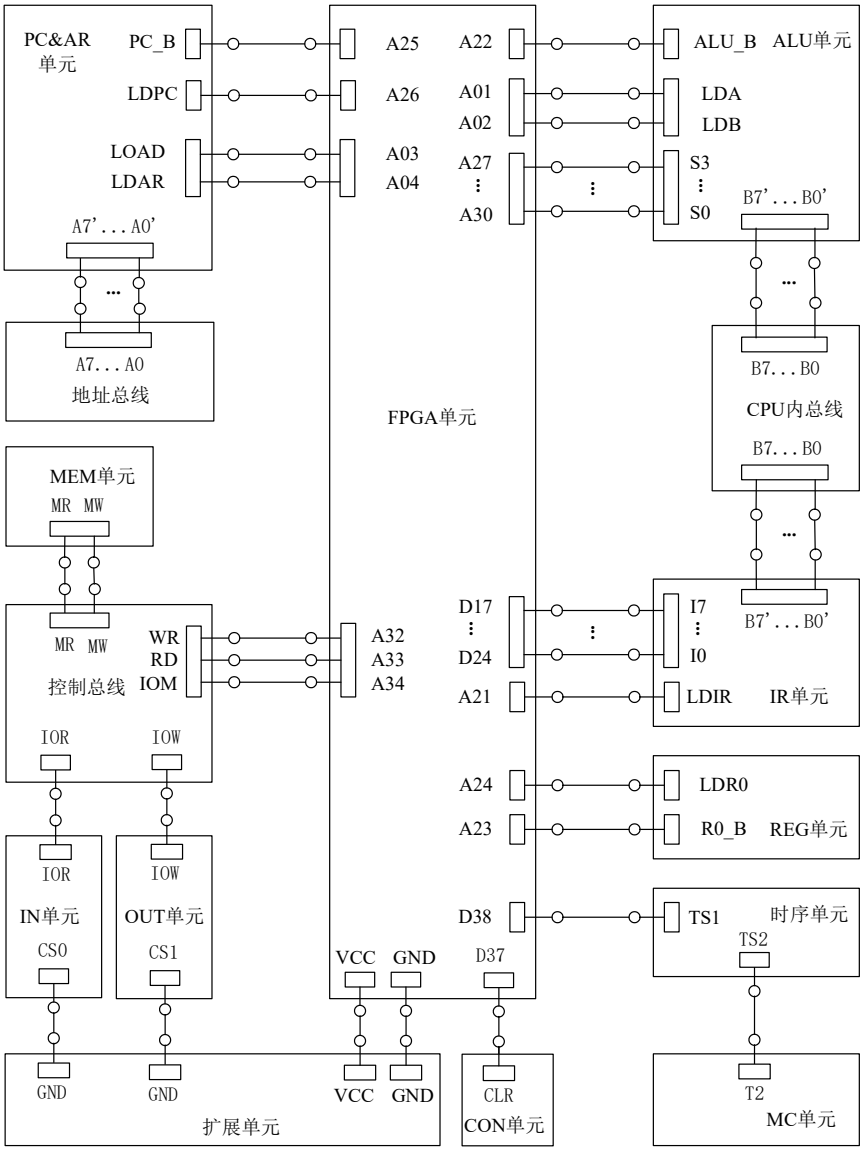


图2-4-3 实验接线图

(3) 关闭实验系统电源，按图2-4-3 连接实验电路。

(4) 打开实验系统电源，**将生成的 SOF（JTAG 模式）或 POF（AS 模式）文件下载到 FPGA 单元的 EP1C6 中去**。

(5) 用本实验定义的机器指令系统，可具体编写多种应用程序，下面给出的是本次实验的例程，其程序的文件名以.TXT 为后缀。程序中分号‘；’为注释符，分号后面的内容在下载时将被忽略掉：

```
; //****************************************//
```

```
; //                                          //
; //      硬布线控制器模型机实验指令文件       //
; //                                          //
; //****************************************** //
; //**** Start Of Main Memory Data **** //
    $P 00 20    ; START: IN R0      从IN 单元读入数据送R0
    $P 01 00    ; ADD R0,R0         R0 和自身相加，结果送R0
    $P 02 30    ; OUT R0            R0 的值送OUT 单元显示
    $P 03 E0    ; JMP START         跳转至START
    $P 04 00    ;
    $P 05 50    ; HLT               停机
; //***** End Of Main Memory Data *****//
```

(6) 进入软件界面，装载机器指令，选择菜单命令“【实验】—【简单模型机】”，打开简单模型机数据通路图，按动CON 单元的总清按钮CLR，使程序计数器PC 地址清零，控制器状态机回到S0，程序从头开始运行，选择相应的功能命令，即可联机运行、监控、调试程序。

(7) 当模型机执行完JMP 指令后，检查OUT 单元显示的数是否为IN 单元值的2 倍，按下CON 单元的总清按钮CLR，改变IN 单元的值，再次执行机器程序，从OUT 单元显示的数判别程序执行是否正确。

附录1 软件使用说明

软件运行环境

操作系统：Windows 98/NT/2000/XP

最低配置：

CPU：奔腾300MHz

内存：64MB

显示卡：标准VGA，256 色显示模式以上

硬盘：20MB 以上

光驱：标准CD-ROM

安装软件

安装操作如下：

可以通过“资源管理器”，找到光盘驱动器本软件安装目录下的‘安装CM3P.EXE’，双击执行它，按屏幕提示进行安装操作。

“TD-CM3+”软件安装成功后，在“开始”的“程序”里将出现“CM3P”程序组，点击“CM3P”即可执行程序。

卸载软件

联机软件提供了自卸载功能，使您可以方便地删除“TD-CM3+”的所有文件、程序组或快捷方式。单击【开始】/【程序】打开“CM3P”的程序组，然后运行“卸载”项，就可执行卸载功能，按照屏幕提示操作即可以安全、快速地删除“TD-CM3+”。

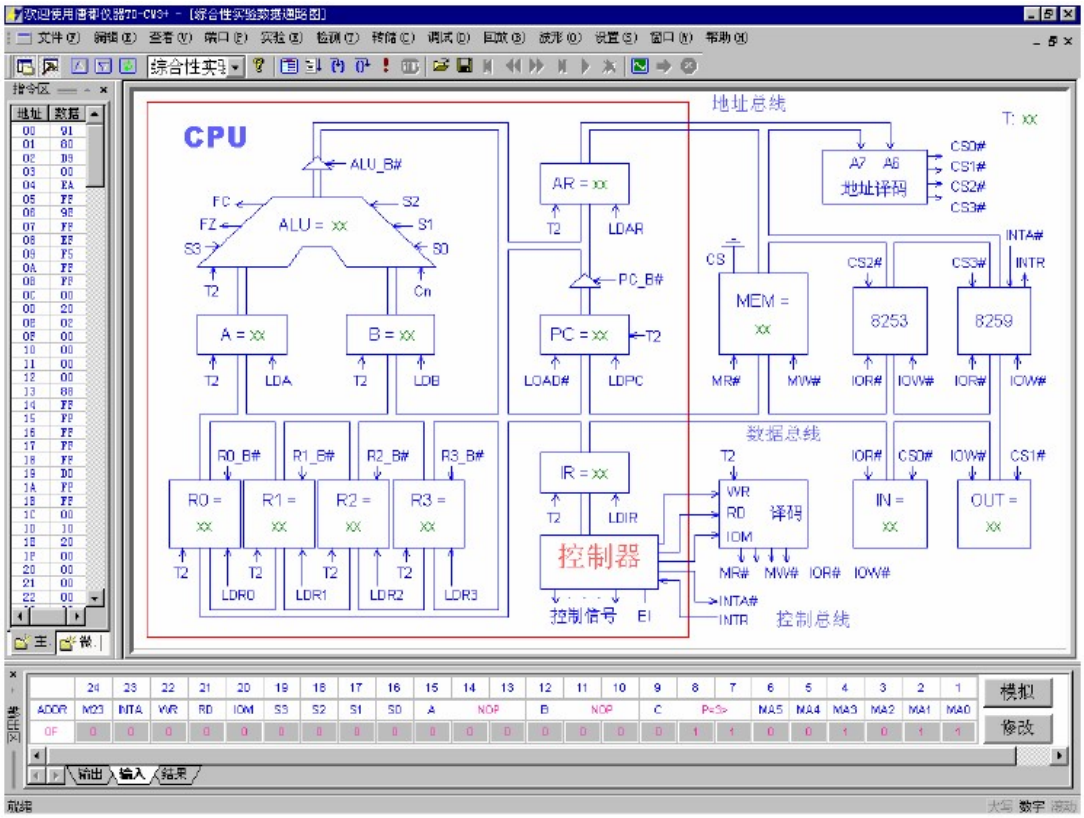
（一） 界面窗口介绍

主界面如附图1-1 所示，由指令区、输出区和图形区三部分组成。

指令区：

分为机器指令区和微指令区，指令区下方有两个Tab 按钮，可通过按钮在两者之间切换。机器指令区：分为两列，第一列为主存地址（00—FF，共256 个单元），第二列为每个地址所对应的数值。串口通讯正常且串口无其它操作，可以直接修改指定单元的内容，用鼠标单击要修改单元的数据，此时单元格会变成一个编辑框，即可输入数据，编辑框只接收两位合法的16 进制数，按回车键确认，或用鼠标点击别的区域，即可完成修改工作。按下ESC 键可取消修改，编辑框会自动消失,恢复显示原来的值，也可以通过上下方向键移动编辑框。

微指令区：分为两列，第一列为微控器地址（00—3F，共64 个单元），第二列为每个地址所对应的微指令，共6 字节。修改微指令操作和修改机器指令一样，只不过微指令是6 位，而机器指令是2 位。



附图1-1 软件主界面

输出区：

输出区由输出页、输入页和结果页组成。

输出页：在数据通路图打开，且该通路中用到微程序控制器，运行程序时，输出区用来实时显示当前正在执行的微指令和下条将要执行的微指令的24 位微码及其微地址。当前正在执行微指令的显示可通过菜单命令“【设置】—【当前微指令】”进行开关。

输入页：可以对微指令进行按位输入及模拟，鼠标左键单击ADDR 值，此时单元格会变成一个编辑框，即可输入微地址，输入完毕后回车，编辑框消失，后面的24 位代表当前地址的24位微码，微码值用红色显示，鼠标左键单击微码值可使该值在0 和1 之间切换。在数据通路图打开时，按动‘模拟’按钮，可以在数据通路中模拟该微指令的功能，按动‘修改’按钮则可以将当前显示的微码值下载到下位机。

结果页：用来显示一些提示信息或错误信息，保存和装载程序时会在这一区域显示一些提示信息。在系统检测时，也会在这一区域显示检测状态和检测结果。

图形区：

可以在此区域编辑指令，显示各个实验的数据通路图、示波器界面等。

（二）菜单功能介绍

1. 文件菜单项：

文件菜单提供了以下命令：

①. 新建（N）：

在CM3P 中建立一个新文档。在文件新建对话框中选择您所要建立的新文件的类型。

②. 打开（O）

在一个新的窗口中打开一个现存的文档。您可同时打开多

个文档。您可用窗口菜单在多个打开的文档将活动文档保存到它的当前的文件名和目录下。当您第一次保存文档时，CM3P 显示另存为对话框以便您命名您的文档。如果在保存之前，您想改变当前文档的文件名和目录，您可选用另存为命令。



⑤．另存为（A）…

保存并命名活动文档。CM3P 会显示另存为对话框以便您命名您的文档。

⑥．打印（P）…

打印一个文档。在此命令提供的打印对话框中，您可以指明要打印的页数范围、副本数、目标打印机，以及其它打印机设置选项。

⑦．打印预览（V）

按要打印的格式显示活动文档。当您选择此命令时，主窗口就会被一个打印预览窗口所取代。这个窗口可以按它们被打印时的格式显示一页或两页。打印预览工具栏提供选项使您可选择一次查看一页或两页，在文档中前后移动，放大和缩小页面，以及开始一个打印作业。

⑧．打印设置（R）…

选择一台打印机和一个打印机连接。在此命令提供的打印设置对话框中，您可以指定打印机及其连接。

⑨．最近使用文件

您可以通过此列表，直接打开最近打开过的文件，共四个。

⑩．退出（X）

结束CM3P 的运行阶段。您也可使用在应用程序控制菜单上的关闭命令。

2．编辑菜单项：



①．撤消（U）

撤消上一步编辑操作。

②．剪切（T）

将当前被选取的数据从文档中删除并放置于剪贴板上。如当前没有数据被选取时，此命令则不可用。

③．复制（C）

将被选取的数据复制到剪切板上。如当前无数据被选取时，此命令则不可用。

④．粘贴（P）

将剪贴板上内容的一个副本插入到插入点处。如剪贴板是空的，此命令则不可用。

3．查看菜单项：

查看菜单提供了以下命令：

①．工具栏（T）

显示和隐藏工具栏，工具栏包括了CM3P 中一些最普通命令的按钮。当工具　　栏被显示时，在菜单

项目的旁边会出现一个打勾记号。



②．指令区（W）

显示和隐藏指令区，当指令区被显示时，在菜单项目的旁边会出现一个打勾记号。

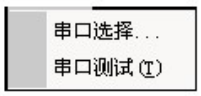
③．输出区（O）

显示和隐藏输出区，当输出区被显示时，在菜单项目的旁边会出现一个打勾记号。

④．状态栏（S）

显示和隐藏状态栏。状态栏描述了被选取的菜单项目或被按下的工具栏按钮，以及键盘的锁定状态将要执行的操作。当状态栏被显示时，在菜单项目的旁边会出现一个打勾记号。

4．端口菜单项：



端口菜单提供了以下命令：

①．端口选择…

选择通讯端口，选择该命令时会弹出附图1-2 所示对话框。该命令会自动检测当前系统可用的串口号，并列于组合框中，选择某一串口后，按确定键，对选定串口进行初始化操作，并进行联机测试，报告测试结果，如果联机成功，则会将指令区初始化。



附图1-2 串口选择对话框

②．端口测试（T）

对当前选择的串口进行联机通讯测试，并报告测试结果，只测一次，如果联机成功，则会将指令区初始化。如串不能正常初始化，此命令则不可用。

5．实验菜单项：



实验菜单提供了以下命令：

①．运算器实验

打开运算器实验数据通路图，如果该通路图已经打，则把通路激活并置于最前面显示。

②．存储器实验

打开存储器实验数据通路图，如果该通路图已经打，则把通路激活并置于最前面显示。

③．微控器实验

打开微控器实验数据通路图，如果该通路图已经打，则把通路激活并置于最前面显示。

④．简单模型机

打开简单模型机数据通路图，如果该通路图已经打，则把通路激活并置于最前面显示。

⑤．综合性实验

打开综合性实验数据通路图，如果该通路图已经打，则把通路激活并置于最前面显示。

6．检测菜单项：检测菜单提供了以下命令：

①．连线检测（C）

a、 简单模型机

对简单模型机的连线进行检测，并在‘输出区’的‘结果页’显示相关信息。



b、 复杂模型机

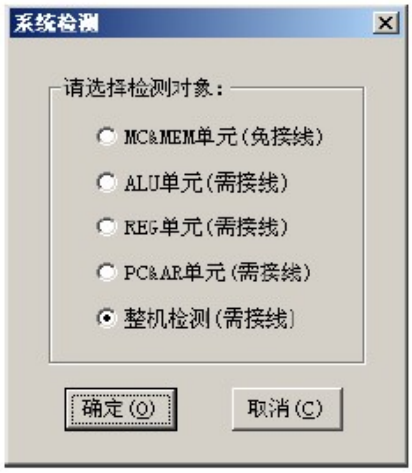
对复杂模型机的连线进行检测，并在‘输出区’的‘结果页’显示相关信息。

②. 系统检测（T）…

启动系统检测，可以进行部件或是整机检测，进行微控器或存储器检测时不需要连线，其余的都需要连线。选择该命令时会弹出如附图1-3 所示对话框，选择一个对象后按‘确定’键就可以启动系统检测，和连线检测不一样的是，连线检测只报告连线错误，而系统检测不仅报告连线错误，而且还报告系统中可能存在的错误，比如哪些芯片可能已被损坏，错误报告中芯片的编号信息见用户手册的系统布局详图。

③. 停止检测（S）

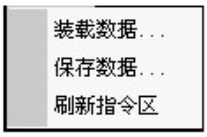
停止系统检测。



附图1-3 系统检测对话框

7. 转储菜单项：

转储菜单提供了以下命令：



①. 装载数据…

将上位机指定文件中的数据装载到下位机中，您选择该命令会弹出打开文件对话框。

可以打开任意路径下的*.TXT 文件，如果指令文件合法，系统将把这些指令装载到下位机中，装载指令时，系统提供了一定的检错功能，如果指令文件中有错误的指令，将会导致系统退出装载，并提示错误的指令行。

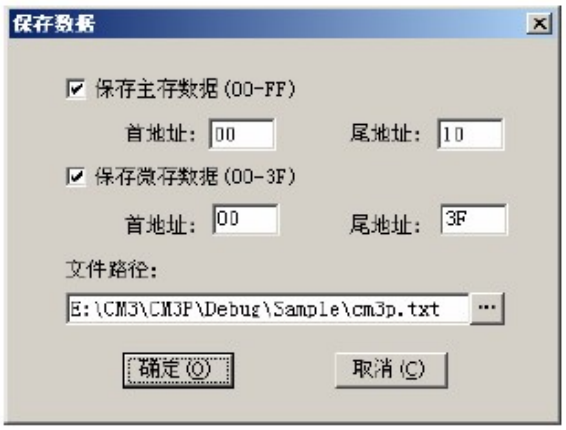
指令文件中指令书写格式如下：



例如机器指令$P00FF,“$”为标记号，“P”代表机器指令，“00”为机器指令的地址，“FF”为该地址中的数据。微指令$M00AA77FF,“$”为标记号，“M”代表微指令，“00”为机器指令的地址，“AA77FF”为该地址中的数据。

②．保存数据…

将下位机中（主存，微控器）的数据保存到上位机中，选择该命令会弹出一个保存对话框，如附图1-4所示。



附图1-4 保存数据对话框

可以选择保存机器指令，此时首尾地址输入框将会变亮，否则首尾地址输入框将会变灰，在允许输入的情况下您可以指定需要保存的首尾地址，微指令也是如此，数据到保存指定路径的*.TXT 格式文件中。

③．刷新指令区

从下位读取所有机器指令和微指令，并在指令区显示。

8．调试菜单项：



调试菜单提供了以下命令：

①．微程序流程图…

当微控器实验、简单模型机和综合性实验中任一数据通路图打开时，可用此命令来打开指定的微程序流程图，选择该命令会弹出打开文件对话框。

②．单节拍

向下位机发送单节拍命令，下位机完成一个节拍的工作。

③．单周期

向下位机发送单周期命令，下位机完成一个机器周期的工作。

④．单步机器指令

向下位机发送单步机器指令命令，下位机运行一条机器指令。

⑤．连续运行

向下位机发送连续运行命令，下位机将会进入连续运行状态。

⑥．停止运行

如果下位机处于连续运行状态，此命令可以使得下位机停止运行。

9．回放菜单项：

回放菜单提供了以下命令。

①．打开…

打开现存的数据文件。

②．保存…

保存当前的数据到数据文件。

③．首端

跳转到首页。



④. 向前

向前翻一页。

⑤. 向后

向后翻一页。

⑥. 末端

跳转到末页。

⑦. 播放

连续向后翻页。

⑧. 停止播放

停止连续向后翻页。

10. 波形菜单项：



波形菜单提供了以下命令：

①. 打开（O）

打开示波器窗口。

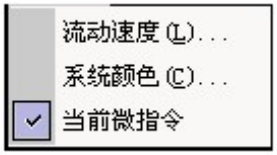
②. 运行（R）

启动示波器，如果下位机正运行程序则不启动。

③. 停止（S）

停止处于启动状态的示波器。

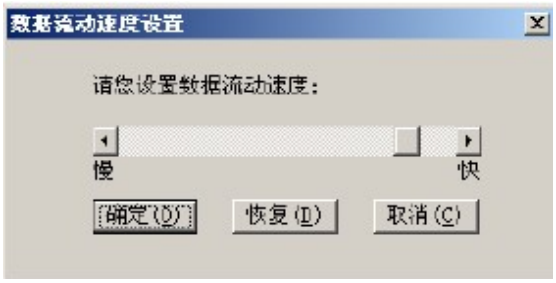
11. 设置菜单项：



设置菜单提供了以下命令：

①. 流动速度（L）…

设置数据通路图中数据的流动速度，选择该命令会弹出一个流动速度设置对话框，如附图1-5 所示。拖动滑动块至适当位置，点击‘确定’按钮即完成设置。



附图1-5 流动速度设置对话框

②. 系统颜色（C）…

设置数据通路图、微程序流程图和示波器的显示颜色，选择该命令会弹出一个设置对话框，如附图1-6 所示。

分为三页，分别为通路图、微流图和示波器，按动每页的TAB 按钮，可在三页之间切换。选择某项要设置的对象，然后按下‘更改’按钮，或直接用鼠标左键点击要设置对象的颜色框，可弹出颜色选择对话框，选定好颜色后，点击‘应用’按钮相应对象的颜色就会被修改掉。



附图1-6 系统颜色设置对话框

③. 当前微指令

设置‘输出区’的‘输出页’是否显示当前微指令，当前微指令用灰色显示，并在地址栏标记为‘C’，下条将要执行的微指令标记为‘N’。

12. 窗口菜单项：



窗口菜单提供了以下命令。这些命令使您能在应用程序窗口中安排多个文档的多个视图：

①. 新建窗口（N）

打开一个具有与活动的窗口相同内容的新窗口。您可同时打开数个文档窗口以显示文档的不同部分或视图。如果您对一个窗口的内容做了改动，所有其它包含同一文档的窗口也会反映出这些改动。当您打开一个新的窗口，这个新窗口就成了活动的窗口并显示于所有其它打开窗口之上。

②. 层叠（C）

按相互重叠形式来安排多个打开的窗口。

③. 平铺（T）

按互不重叠形式来安排多个打开的窗口。

④. 排列图标（A）

在主窗口的底部安排被最小化的窗口的图标。如果在主窗口的底部有一个打开的窗口，则有可能会看不见某些或全部图标，因为它们在这个文档窗口的下面。

⑤. 窗口选择

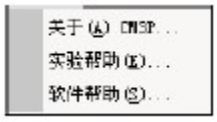
CM3P 在窗口菜单的底部显示出当前打开的文档窗口的清单。有一个打勾记号出现在活动的窗口的文档名前。从该清单中挑选一个文档可使其窗口成为活动窗口。

13. 帮助菜单项：

帮助菜单提供以下的命令，为您提供使用这个应用程序的帮助：

①. 关于（A）CM3P…

显示您的CM3P 版本的版权通告和版本号码。



②. 实验帮助（E）…

显示实验帮助的开场屏幕。从此开场屏幕，您可跳到关于CM3P 所提供实验的参考资料。

③. 软件帮助（S）…

显示软件帮助的开场屏幕。从此开场屏幕，您可跳到关于使用CM3P 设备的参考资料。

工具栏命令按钮介绍：

 显示或隐藏指令区。

 显示或隐藏输出区。

 保存下位机数据。

 向下位机装载数据。

 刷新指令区数据。

 打开实验帮助。

 打开微程序流程图。

 单节拍运行。

 单周期运行。

 单机器指令运行。

 连续运行。

 停止运行。

 打开实验数据文件。

 保存实验数据。

 跳转到首页。

 向前翻页。

 向后翻页。

 跳转到末页。

 连续向后翻页。

 停止向后翻页。

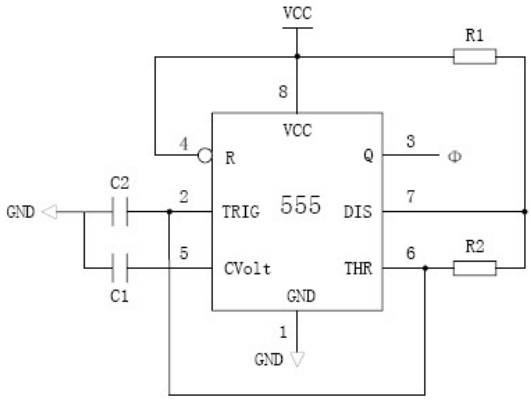
 打开示波器窗口。

 启动示波器。

 停止示波器。

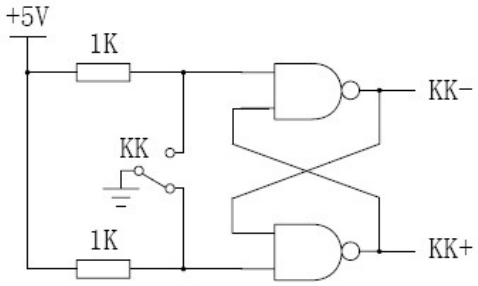
附录2 时序单元介绍

时序单元可以提供三种时序：Φ、KK 和TS。Φ为555 构成的多谐振荡器的输出，其原理如附图2-1 所示，频率大约为60Hz-70Hz，占空比约为50%，其中C1 取0.01uF，C2 取1uF，R1 取1KΩ，R2 取10KΩ。



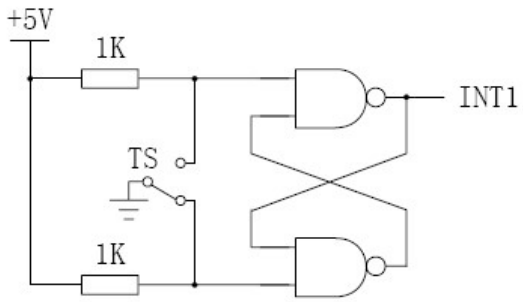
附图2-1 555 多谐振荡器原理图

每按动一次KK 按钮，在KK+和KK-端将分别输出一个上升沿和下降沿单脉冲。其原理如附图2-2 所示：

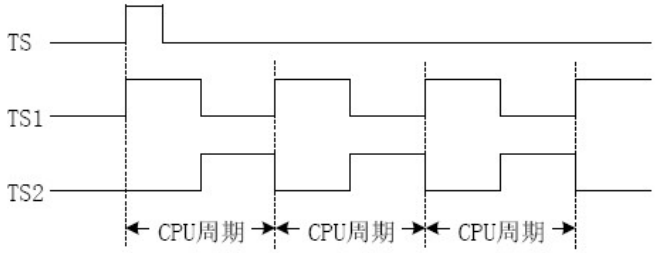


附图2-2 KK 单脉冲电路原理图

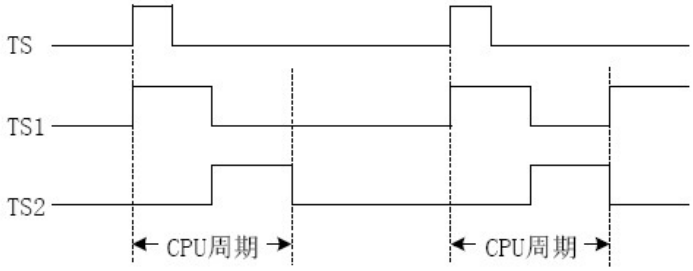
每按动一次TS 按钮，将对单片机INT1 产生一次中断请求，其原理如附图2-3 如示，单片机根据时序开关档位的不同，在TS1 和TS2 端输出不同的波形。当开关处于‘连续’档时，TS1 和TS2 输出的是附图2-4 所示的连续时序。开关处于‘单步’档时，TS1 和TS2 只输出一个CPU 周期的波形，如附图2-5。开关处于‘单拍’档时，TS1 和TS2 交替出现，如附图2-6 所示。



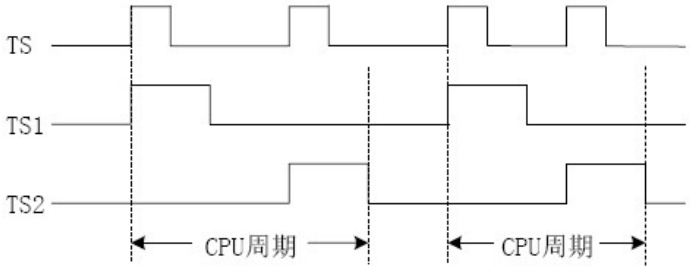
附图2-3 TS 时序请求原理图



附图2-4 连续时序



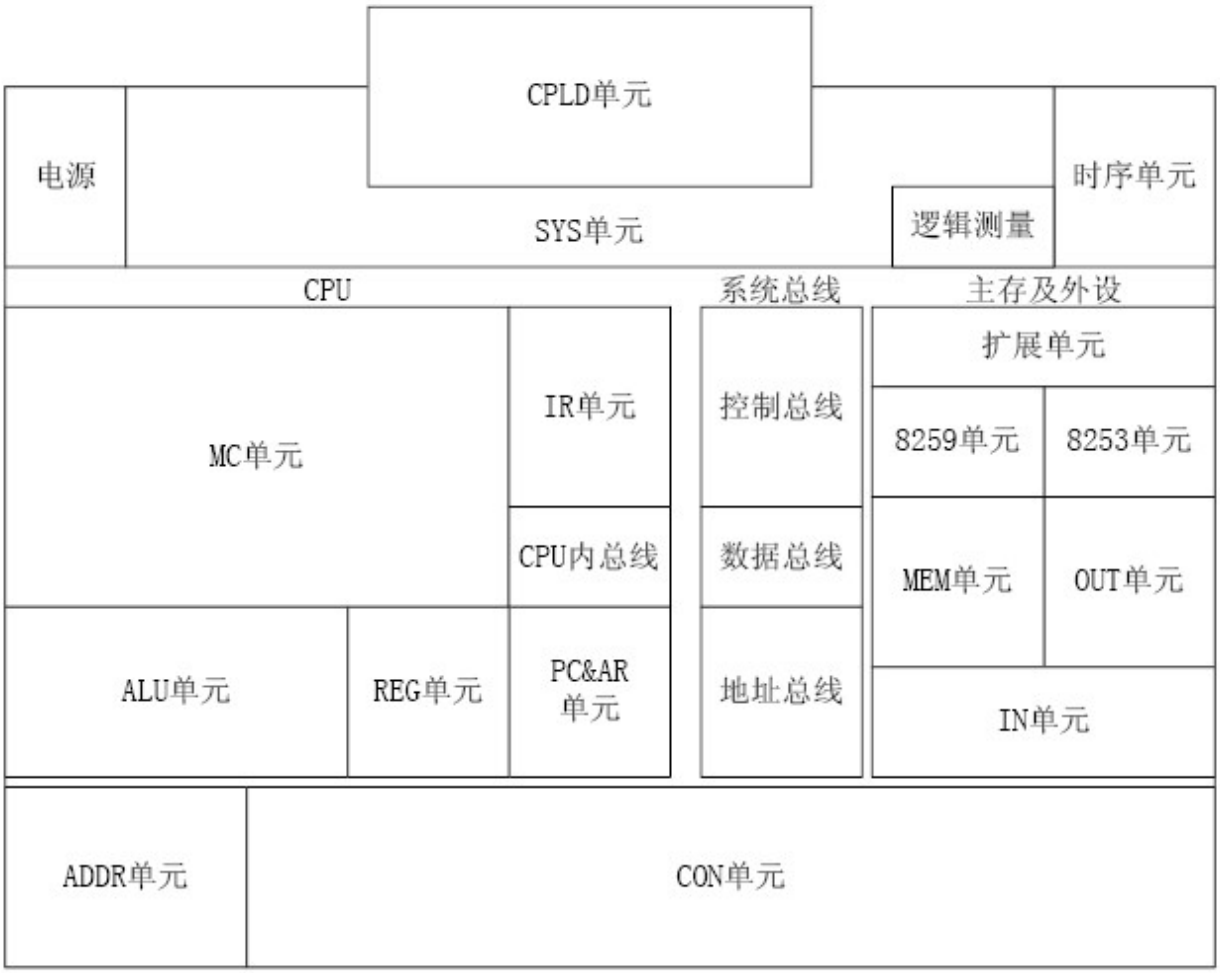
附图2-5 单步时序



附图2-6 单拍时序

当TS1 和TS2 输出连续波形时，有三种方法可以停止输出，一种是将时序状态开关拨至非连续档，按动CON 单元的CLR 按钮或是系统单元的复位按钮，CON 单元的CLR 按钮和SYS 单元的复位按钮的区别是，CLR 按钮完成对各单元清零，复位按钮完成对单片机复位。

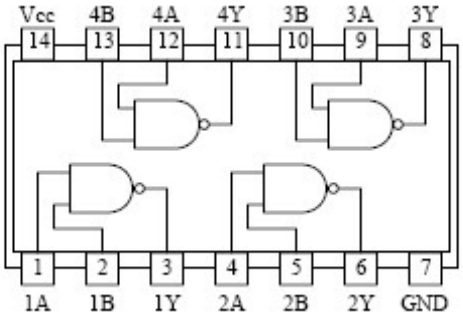
附录3 系统布局图


CPLD单元
电源
SYS单元
逻辑测量
时序单元
CPU
系统总线
主存及外设
扩展单元
MC单元
IR单元
控制总线
8259单元
8253单元
CPU内总线
数据总线
MEM单元
OUT单元
ALU单元
REG单元
PC&AR
单元
地址总线
IN单元
ADDR单元
CON单元

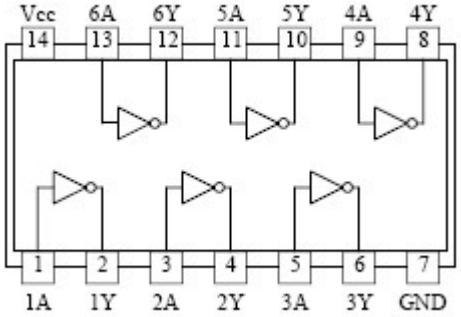
附录4 实验用芯片介绍

本附录介绍一些实验电路中用到的中大规模数字功能器件，以供教学实验参考，其中Q_0 为时钟脉冲的上升沿之前Q 的输出，1 为高电平，0 为低电平，Z 为高阻态。

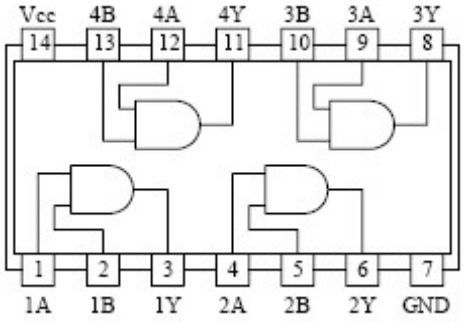
74LS00



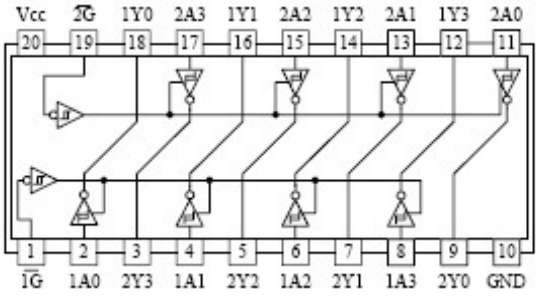
74LS04



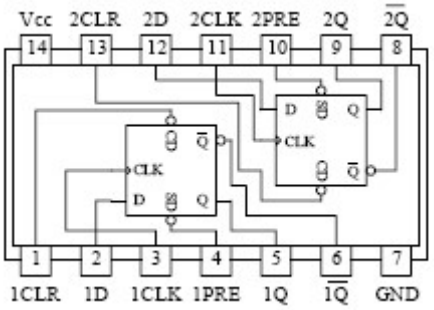
74LS08



74LS240

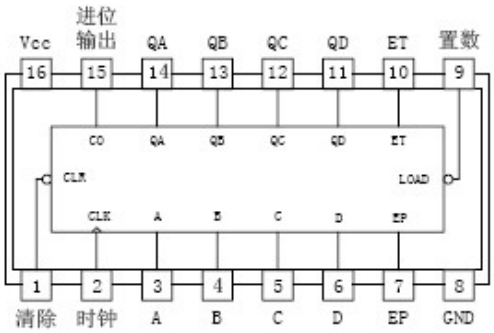


74LS74



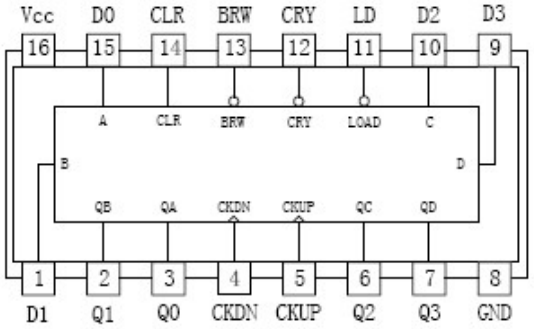
PRC	CLR	CLK	D	Q	$\overline{Q}$
0	1	X	X	1	0
1	0	X	X	0	1
0	0	X	X	1*	1*
1	1	↑	1	1	0
1	1	↑	0	0	1
1	1	0	X	Q_0	$\overline{Q_0}$

74LS161



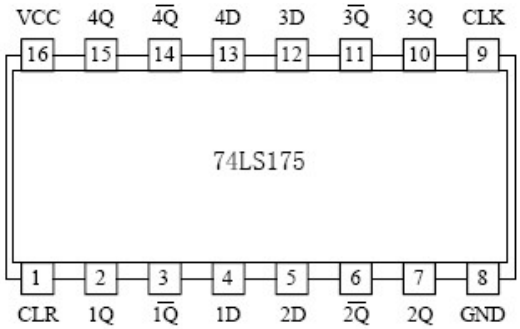
输入					输出					
清除	置数	时钟	使能		QA	QB	QC	QD	进位输出	工作
			EP	ET						
1	1	↑	1	1	——				—	计数
1	0		X	X	A	B	C	D	—	置数
0↓	X	X	X	X	0	0	0	0	—	清除
1	X	X	X	1	1	1	1	1	1	—

74LS193



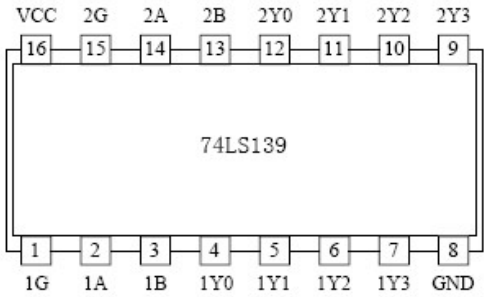
CKUP	CKDN	CLR	LD	功能
↑	1	0	1	上行计数
1	↑	0	1	下行计数
X	X	1	X	清除
X	X	0	0	置数

74LS175



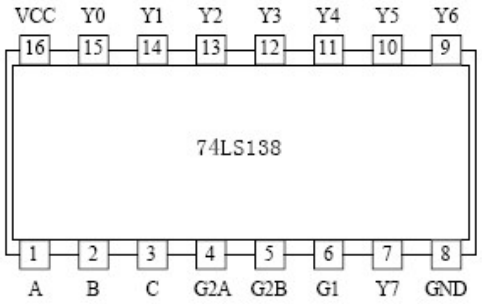
CLR	CLK	D	Q	$\overline{Q}$
0	X	X	0	1
0	↑	1	1	0
1	↑	0	0	1
1	0	X	Q_0	$\overline{Q_0}$

74LS139



G	B	A	Y0	Y1	Y2	Y3
1	X	X	1	1	1	1
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0

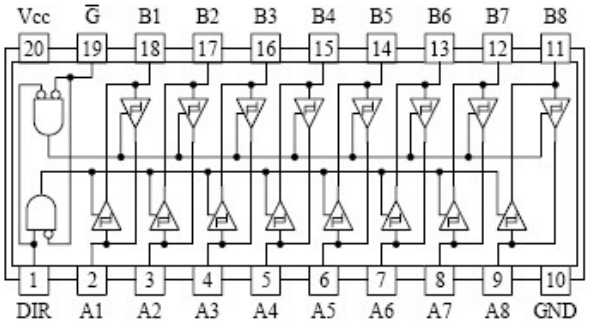
74LS138



G1	G*	C	B	A	Y0	Y1	Y2	Y3	Y4	Y5	Y6	Y7
X	1	X	X	X	1	1	1	1	1	1	1	1
0	X	X	X	X	1	1	1	1	1	1	1	1
1	0	0	0	0	0	1	1	1	1	1	1	1
1	0	0	0	1	1	0	1	1	1	1	1	1
1	0	0	1	0	1	1	0	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1	1	1
1	0	1	0	0	1	1	1	1	0	1	1	1
1	0	1	0	1	1	1	1	1	1	0	1	1
1	0	1	1	0	1	1	1	1	1	1	0	1
1	0	1	1	1	1	1	1	1	1	1	1	0

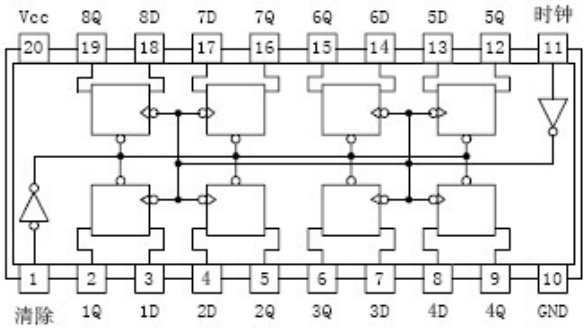
G* = G2A + GAB

74LS245



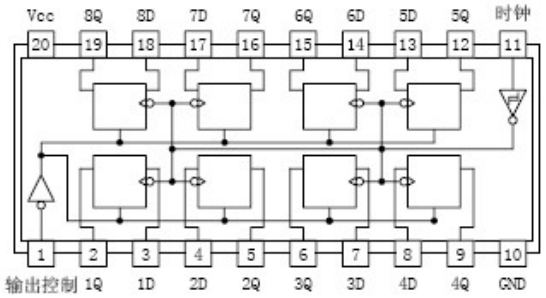
使能 G	方向控制 DIR	操作
0	0	B->A
0	1	A->B
1	X	隔开

74LS273



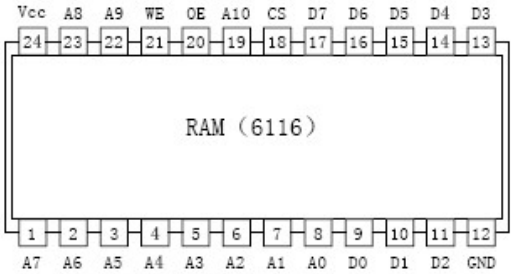
输入			输出 Q
清除	时钟	D	
0	X	X	0
1	↑	1	1
1	↑	0	0
1	0	X	Q_0

74LS374



输出控制	G	D	输出
0	↑	1	1
0	↑	0	0
0	0	X	Q_0
1	X	X	Z

SRAM 6116



CS	WE	OE	功能
1	X	X	不选择
0	1	0	读
0	0	1	写
0	0	0	写